

MLOps - Assignment 2

End-to-End MLOps Pipeline for Cats vs Dogs

Binary Image Classification — Pet Adoption Platform

BITS Pilani WILP — MLOps (S1-25_AIMLCZG523)

Submitted by: Aditya Chowdhury

GitHub Repository: github.com/adchow31/cats-dogs-mlops

Container Registry: ghcr.io/adchow31/cats-dogs-mlops

Name : Aditya Chowdhury

BITS ID : 2024AD05010

BITS Email : 2024ad05010@wilp.bits-pilani.ac.in

GitHub URL of Complete Project -

<https://github.com/adchow31/cats-dogs-mlops>

Video URL -

<https://youtu.be/1JgRIEthp84>

Project Overview

This document records the complete end-to-end MLOps pipeline built for the binary image classification (Cats vs Dogs) assignment, covering model development, packaging, containerization, CI/CD-based deployment, and monitoring, using entirely open-source tools. The pipeline was developed and executed on a local Windows machine using Jupyter Notebook, Docker Desktop, and GitHub Actions.

Technology Stack

- Model Development: PyTorch (custom CNN), MLflow (experiment tracking)
- Data & Model Versioning: Git + DVC (local remote storage)
- Inference Service: FastAPI, served via Uvicorn
- Containerization: Docker, GitHub Container Registry (GHCR)
- CI Pipeline: GitHub Actions (pytest, Docker build, GHCR push)
- CD Pipeline: GitHub Actions + Docker Compose (automated deployment + smoke tests)
- Monitoring: Prometheus (metrics scraping), Grafana (visualization), structured application logging

Project Structure

The repository is organized as follows, with each file mapped to the milestone it belongs to:

```

cats-dogs-mlops/
├── .git/
├── .dvc/
├── .github/workflows/
│   ├── ci.yml          # M3
│   └── cd.yml          # M4
├── data/
│   ├── raw/            # DVC tracked
│   │   ├── train/{cats,dogs}/
│   │   ├── val/{cats,dogs}/
│   │   └── test/{cats,dogs}/
│   └── raw.dvc
├── notebooks/
│   └── 01_train_baseline.ipynb # M1 – your main working notebook
├── src/
│   ├── __init__.py
│   ├── data_preprocessing.py # M1/M3 – testable functions
│   ├── model.py             # CNN definition
│   ├── train.py             # standalone training script (mirrors notebook)
│   └── inference_utils.py    # M2/M3 – predict logic reused by API
├── app/
│   ├── main.py             # M2 – FastAPI service
│   └── schemas.py
├── models/
│   └── baseline_cnn.pt      # M1 output, DVC or git-lfs tracked
├── tests/
│   ├── test_preprocessing.py # M3
│   └── test_inference.py     # M3
├── k8s/
│   ├── deployment.yaml     # M4
│   └── service.yaml
├── monitoring/
│   └── metrics.py           # M5 – Prometheus counters
├── scripts/
│   └── smoke_test.sh        # M4
├── mlruns/                  # MLflow local tracking store
├── requirements.txt         # M2
├── Dockerfile               # M2
├── docker-compose.yml (optional) # alt to k8s
├── .dockerignore
├── .gitignore
└── README.md

```

Fig 0.1 — Full repository folder structure, annotated by milestone (M1–M5).

Milestone Summary

Milestone	Description	Marks
M1	Model Development & Experiment Tracking (Git/DVC versioning, baseline CNN, MLflow tracking)	10
M2	Model Packaging & Containerization (FastAPI service, Dockerfile)	10
M3	CI Pipeline for Build, Test & Image Creation (pytest, GitHub Actions, GHCR push)	10
M4	CD Pipeline & Deployment (Docker Compose, automated deploy, smoke tests)	10
M5	Monitoring, Logs & Final Submission (Prometheus, Grafana, post-deployment tracking)	10

Final result: All five milestones completed. Baseline CNN achieved 91.11% validation accuracy and 99.17%+ confidence on live inference tests. CI and CD pipelines both passing (green) at time of submission.

M1 — Model Development & Experiment Tracking

This milestone was carried out in notebooks/01_train_baseline.ipynb. The subsections below present the annotated screenshots first, followed by a full code-and-output walkthrough of the executed notebook (Section 1.6) for complete traceability.

1.1 Dataset Acquisition

The original planned dataset (salader/dogs-vs-cats on Kaggle) was no longer accessible via the KaggleHub API, returning a 403 permission error.

```
[4]: import kagglehub

# Downloads to a local cache dir
path = kagglehub.dataset_download("salader/dogs-vs-cats")
print("Dataset downloaded to:", path)

# Inspect the extracted structure to find the class folders (cats/ and dogs/)
for root, dirs, files in os.walk(path):
    print(root, "->", dirs[:5], f"({len(files)} files)")
    if root.count(os.sep) - path.count(os.sep) > 2:
        break

362 r.dataset_slug = h.dataset
363 r.dataset_slug = h.dataset
--> 364 dataset = handle_call(lambda: api_client.datasets.dataset_api_client.get_dataset(r))
365 return dataset.current_version_number
366 elif isinstance(h, NotebookHandle):

File ~\AppData\Roaming\Python\Python313\site-packages\kagglehub\exceptions.py:107, in handle_call(fn, resource_handle)
    98 message = (
    99     f"{e.response.status_code} Client Error."
   100     "\n\n"
   (... ) 103     "Please make sure you specified the correct resource identifiers."
   104 )
   106 # Default handling
--> 107 raise KaggleApiHTTPError(message, response=e.response) from e

KaggleApiHTTPError: 403 Client Error.

You don't have permission to access resource at URL: https://api.kaggle.com/v1/datasets.DatasetApiService/GetDataset. Please make sure you are authenticated if you are trying to access a private resource or a resource requiring consent.
```

Fig 1.1 — Kaggle API 403 error when attempting to download the original dataset via kagglehub.

The dataset was successfully sourced from an alternate, publicly available mirror of the same underlying data — the Microsoft Research Cats vs Dogs dataset — via kagglehub.

```
1.1 Download the dataset (KaggleHub)

Downloads the Cats vs Dogs dataset locally. Requires a Kaggle account/API token configured (~/.kaggle/kaggle.json) if kagglehub prompts for auth.

[5]: import kagglehub

path = kagglehub.dataset_download("shaunthesheep/microsoft-catsvsdogs-dataset")
print("Dataset downloaded to:", path)

for root, dirs, files in os.walk(path):
    print(root, "->", dirs[:5], f"({len(files)} files)")
    if root.count(os.sep) - path.count(os.sep) > 2:
        break

Downloading to C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\1.archive...
100%|██████████| 788M/788M [00:53<00:00, 15.4MB/s]
Extracting files...

Dataset downloaded to: C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1 -> ['PetImages'] (2 files)
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages -> ['Cat', 'Dog'] (0 files)
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages\Cat -> [] (12501 files)
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages\Dog -> [] (12501 files)
```

Fig 1.2 — Successful dataset download from shaunthesheep/microsoft-catsvsdogs-dataset (Cat: 12,500 files, Dog: 12,500 files).

1.2 Data Preprocessing & Train/Val/Test Split

A custom preprocessing pipeline (src/data_preprocessing.py) reorganizes the raw Cat/Dog folders into an 80/10/10 train/validation/test split. An initial version produced a folder-naming mismatch error, which was corrected by separating source and destination class names.

```
[11]: # Adjust this path based on the printout from the download cell above
# The Kaggle dataset typically extracts to <path>/PetImages/Cat and <path>/PetImages/Dog,
# or <path>/cats and <path>/dogs depending on the mirror.
source_root = f"{path}/PetImages" # <-- update if needed

counts = build_split_dirs(source_root=source_root, dest_root="data/raw")
print("Split counts:", counts)

-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[11], line 6
      1 # Adjust this path based on the printout from the download cell above
      2 # The Kaggle dataset typically extracts to <path>/PetImages/Cat and <path>/PetImages/Dog,
      3 # or <path>/cats and <path>/dogs depending on the mirror.
      4 source_root = f"{path}/PetImages" # <-- update if needed
----> 5 counts = build_split_dirs(source_root=source_root, dest_root="data/raw")
      6 print("Split counts:", counts)

Cell In[7], line 33, in build_split_dirs(source_root, dest_root, classes)
     31 for cls in classes:
     32     class_dir = Path(source_root) / cls
--> 33     files = list_class_images(str(class_dir))
     34     splits = split_dataset(files)
     35     for split_name, split_files in splits.items():

Cell In[7], line 9, in list_class_images(class_dir)
      5 """Return sorted list of valid image file paths in a class directory."""
      6 valid_ext = {".jpg", ".jpeg", ".png"}
      7 return sorted([
      8     str(Path(class_dir) / f)
----> 9     for f in os.listdir(class_dir)
     10     if Path(f).suffix.lower() in valid_ext
     11 ])

FileNotFoundError: [WinError 3] The system cannot find the path specified: 'C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages\cats'
```

Fig 1.3 — FileNotFoundError from an early version of build_split_dirs due to a folder-name mismatch (Cat/Dog vs cats/dogs).

```
[12]: def build_split_dirs(source_root: str, dest_root: str, source_classes=("Cat", "Dog"), dest_classes=("cats", "dogs")):
    for split in ["train", "val", "test"]:
        for cls in dest_classes:
            os.makedirs(Path(dest_root) / split / cls, exist_ok=True)

        for src_cls, dst_cls in zip(source_classes, dest_classes):
            class_dir = Path(source_root) / src_cls
            files = list_class_images(str(class_dir))
            splits = split_dataset(files)
            for split_name, split_files in splits.items():
                for f in split_files:
                    shutil.copy(f, Path(dest_root) / split_name / dst_cls / Path(f).name)

        return {s: sum(len(os.listdir(Path(dest_root)/s/c)) for c in dest_classes) for s in ["train", "val", "test"]}

[13]: counts = build_split_dirs(source_root=f"{path}/PetImages", dest_root="data/raw")
print("Split counts:", counts)

Split counts: {'train': 19998, 'val': 2498, 'test': 2502}
```

Fig 1.4 — Corrected split function successfully producing train (19,998), val (2,498), and test (2,502) image counts.

1.3 Git & DVC Versioning

Git was used for source code versioning and DVC for dataset and model artifact versioning, backed by a local DVC remote. An early push accidentally included the full ~771MB raw dataset due to a working-directory/.gitignore mismatch; this was diagnosed and corrected.

```
[16]: import os

# Move working directory from notebooks/ up to project root - run once per kernel session
if os.path.basename(os.getcwd()) == "notebooks":
    os.chdir("../")

print("Working directory:", os.getcwd())

Working directory: C:\Users\XRIG\cats-dogs-mlops
```

1. Data & Code Versioning

1.1 Download the dataset (KaggleHub)

Downloads the Cats vs Dogs dataset locally. Requires a Kaggle account/API token configured (~/.kaggle/kaggle.json) if kagglehub prompts for auth.

```
[17]: import kagglehub

path = kagglehub.dataset_download("shaunthesheep/microsoft-catsvsdogs-dataset")
print("Dataset downloaded to:", path)

for root, dirs, files in os.walk(path):
    print(root, "->", dirs[:5], f"({len(files)} files)")
    if root.count(os.sep) - path.count(os.sep) > 2:
        break

Dataset downloaded to: C:\Users\XRIG\cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1
C:\Users\XRIG\cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1 -> ['PetImages'] (2 files)
C:\Users\XRIG\cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages -> ['Cat', 'Dog'] (0 files)
C:\Users\XRIG\cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages\Cat -> [] (12500 files)
C:\Users\XRIG\cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages\Dog -> [] (12500 files)
```

Fig 1.5 — Initial push accidentally included ~25,000 dataset images (771 MB) due to the dataset being nested inside notebooks/ and a non-matching .gitignore pattern.

```

C:\Users\XRIG\cats-dogs-mlops>git push -u origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 16 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (14/14), 7.42 KiB | 2.47 MiB/s, done.
Total 14 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/adchow31/cats-dogs-mlops.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\XRIG\cats-dogs-mlops>

```

Fig 1.6 — Clean repository re-push after correcting the notebook's working directory and hardening .gitignore with recursive (**) patterns — final push only 7.42 KiB, code only.

```

C:\Users\XRIG\cats-dogs-mlops>dvc init
'dvc' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\XRIG\cats-dogs-mlops>

```

Fig 1.7 — 'dvc' command not recognized (PATH issue); resolved by invoking DVC via python -m dvc for the remainder of the session.

1.4 Baseline CNN Model & Training

A baseline convolutional neural network (4 convolutional blocks + fully connected classifier) was implemented in PyTorch and trained for 10 epochs on CPU.

Epoch 1/10	train_loss=0.6244	val_loss=0.5511	val_acc=0.7130
Epoch 2/10	train_loss=0.5368	val_loss=0.4668	val_acc=0.7854
Epoch 3/10	train_loss=0.4695	val_loss=0.4399	val_acc=0.7910
Epoch 4/10	train_loss=0.4186	val_loss=0.3818	val_acc=0.8347
Epoch 5/10	train_loss=0.3582	val_loss=0.3105	val_acc=0.8675
Epoch 6/10	train_loss=0.3198	val_loss=0.2999	val_acc=0.8811
Epoch 7/10	train_loss=0.2936	val_loss=0.2650	val_acc=0.8935
Epoch 8/10	train_loss=0.2733	val_loss=0.2633	val_acc=0.8919
Epoch 9/10	train_loss=0.2476	val_loss=0.2293	val_acc=0.9091
Epoch 10/10	train_loss=0.2318	val_loss=0.2275	val_acc=0.9111

Final validation accuracy: 91.11%. All parameters, per-epoch metrics, the confusion matrix, and loss-curve plots were logged to MLflow (experiment: cats-vs-dogs-baseline, run: baseline_cnn_v1). The trained model was saved as models/baseline_cnn.pt.

1.5 Committing the Trained Model

The model was initially tracked via DVC; it was later committed directly to Git (with the DVC pointer removed) to simplify access for the CI pipeline, which does not have a DVC remote configured.

1.6 Full Notebook Walkthrough (Code & Outputs)

The following is the complete, executed code and real output from notebooks/01_train_baseline.ipynb, presented cell-by-cell in the order they were run, for full reproducibility and traceability.

Environment Setup

Import libraries and verify PyTorch/CUDA setup

```
import os
import shutil
import random
from pathlib import Path

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

import mlflow
import mlflow.pytorch
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

print("Torch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

Output:

```
Torch version: 2.13.0+cpu
CUDA available: False
```

Move working directory from notebooks/ to project root

```
import os

# Move working directory from notebooks/ up to project root – run once per kernel session
if os.path.basename(os.getcwd()) == "notebooks":
    os.chdir("..")

print("Working directory:", os.getcwd())
```

Output:

```
Working directory: C:\Users\XRIG\cats-dogs-mlops
```

1.1 Dataset Download

Download the Cats vs Dogs dataset via KaggleHub

```
import kagglehub

path = kagglehub.dataset_download("shaunthesheep/microsoft-catsvsdogs-dataset")
print("Dataset downloaded to:", path)

for root, dirs, files in os.walk(path):
    print(root, "->", dirs[:5], f"({len(files)} files)")
```

```
if root.count(os.sep) - path.count(os.sep) > 2:
    break
```

Output:

```
Dataset downloaded to: C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1 ->
['PetImages'] (2 files)
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages -> ['Cat', 'Dog'] (0 files)
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages\Cat -> [] (12500 files)
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages\Dog -> [] (12500 files)
```

1.2 Preprocessing Functions

Define `list_class_images`, `split_dataset`, and `build_split_dirs` (`src/data_preprocessing.py`)

```
IMAGE_SIZE = (224, 224)
SPLIT_RATIOS = {"train": 0.8, "val": 0.1, "test": 0.1}

def list_class_images(class_dir: str) -> list:
    """Return sorted list of valid image file paths in a class directory."""
    valid_ext = {".jpg", ".jpeg", ".png"}
    return sorted([
        str(Path(class_dir) / f)
        for f in os.listdir(class_dir)
        if Path(f).suffix.lower() in valid_ext
    ])

def split_dataset(file_list: list, ratios: dict = SPLIT_RATIOS, seed: int = 42) -> dict:
    """Deterministically split a file list into train/val/test given ratios."""
    random.Random(seed).shuffle(file_list)
    n = len(file_list)
    n_train = int(n * ratios["train"])
    n_val = int(n * ratios["val"])
    return {
        "train": file_list[:n_train],
        "val": file_list[n_train:n_train + n_val],
        "test": file_list[n_train + n_val:],
    }

def build_split_dirs(source_root: str, dest_root: str, classes: list = ("cats", "dogs")):
    """Reorganize a flat/class-folder dataset into data/raw/{train,val,test}/{class}/."""
    for split in ["train", "val", "test"]:
        for cls in classes:
            os.makedirs(Path(dest_root) / split / cls, exist_ok=True)

    for cls in classes:
        class_dir = Path(source_root) / cls
        files = list_class_images(str(class_dir))
        splits = split_dataset(files)
        for split_name, split_files in splits.items():
            for f in split_files:
                shutil.copy(f, Path(dest_root) / split_name / cls / Path(f).name)

    return {s: sum(len(os.listdir(Path(dest_root)/s/c)) for c in classes) for s in ["train", "val", "test"]}
```

1.3 Corrupt Image Filtering

Filter malformed/truncated JPEGs before splitting the dataset

```
from PIL import Image

def filter_corrupt_images(class_dir: str):
    removed = []
    for f in list_class_images(class_dir):
        try:
            img = Image.open(f)
            img.verify()
        except Exception:
            removed.append(f)
            os.remove(f)
    return removed

# Example usage – adjust to your extracted path structure:
source_root = f"{path}/PetImages"
for cls in ["Cat", "Dog"]:
    bad = filter_corrupt_images(f"{source_root}/{cls}")
    print(f"{cls}: removed {len(bad)} corrupt files")
```

Output:

```
Cat: removed 0 corrupt files
Dog: removed 0 corrupt files
```

Output:

```
C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages\PIL\TiffImagePlugin.py:949:
UserWarning: Truncated File Read
  warnings.warn(str(msg))
```

1.4 Dataset Split — Final Clean Run

Redefine build_split_dirs with separate source/destination class names

```
def build_split_dirs(source_root: str, dest_root: str, source_classes=("Cat", "Dog"),
    dest_classes=("cats", "dogs")):
    for split in ["train", "val", "test"]:
        for cls in dest_classes:
            os.makedirs(Path(dest_root) / split / cls, exist_ok=True)

    for src_cls, dst_cls in zip(source_classes, dest_classes):
        class_dir = Path(source_root) / src_cls
        files = list_class_images(str(class_dir))
        splits = split_dataset(files)
        for split_name, split_files in splits.items():
            for f in split_files:
                shutil.copy(f, Path(dest_root) / split_name / dst_cls / Path(f).name)

    return {s: sum(len(os.listdir(Path(dest_root)/s/c)) for c in dest_classes) for s in ["train",
        "val", "test"]}
```

Clean up any stale data/raw/ directory from earlier attempts

```
import shutil

if os.path.exists("data/raw"):
    shutil.rmtree("data/raw")
```

```
print("Removed data/raw – will rebuild clean")
```

Output:

```
Removed data/raw – will rebuild clean
```

Execute the train/val/test split

```
counts = build_split_dirs(source_root=f"{path}/PetImages", dest_root="data/raw")
print("Split counts:", counts)
```

Output:

```
Split counts: {'train': 19998, 'val': 2498, 'test': 2502}
```

1.6 DVC Setup

Install DVC for dataset and model versioning

```
pip install dvc
```

Output:

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: dvc in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (3.67.1)
Requirement already satisfied: attrs>=22.2.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (25.4.0)
Requirement already satisfied: celery in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (5.6.3)
Requirement already satisfied: colorama>=0.3.9 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.4.6)
Requirement already satisfied: configobj>=5.0.9 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (5.0.9)
Requirement already satisfied: distro>=1.3 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.9.0)
Requirement already satisfied: dpath<3,>=2.1.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (2.2.0)
Requirement already satisfied: dulwich in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.2.14)
Requirement already satisfied: dvc-data<3.19.0,>=3.18.2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (3.18.3)
... (85 lines omitted for brevity) ...
Requirement already satisfied: aiosignal>=1.4.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (1.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (0.5.2)
Requirement already satisfied: yarl<2.0,>=1.17.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (1.24.2)
Note: you may need to restart the kernel to use updated packages.
```

2.1 Baseline CNN Architecture

Define the BaselineCNN model (4 conv blocks + FC classifier) and verify output shape

```
class BaselineCNN(nn.Module):
    """Simple CNN baseline for binary cats-vs-dogs classification on 224x224 RGB images."""
    def __init__(self, num_classes: int = 2):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # 224->112
            nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # 112->56
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # 56->28
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # 28->14
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 14 * 14, 256),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(256, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)

# Sanity check
_dummy = torch.randn(2, 3, 224, 224)
_model = BaselineCNN()
print("Output shape:", _model(_dummy).shape) # expect torch.Size([2, 2])
```

Output:

```
Output shape: torch.Size([2, 2])
```

2.2 Data Loaders with Augmentation

Build train/val/test DataLoaders with augmentation transforms; confirm class mapping

```
train_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

eval_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

train_ds = datasets.ImageFolder("data/raw/train", transform=train_tf)
val_ds = datasets.ImageFolder("data/raw/val", transform=eval_tf)
test_ds = datasets.ImageFolder("data/raw/test", transform=eval_tf)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True, num_workers=2)
val_loader = DataLoader(val_ds, batch_size=32, shuffle=False, num_workers=2)
test_loader = DataLoader(test_ds, batch_size=32, shuffle=False, num_workers=2)

print("Class mapping:", train_ds.class_to_idx) # note this - you'll need it in the API (M2)
```

```
print(f"Train: {len(train_ds)} | Val: {len(val_ds)} | Test: {len(test_ds)}")
```

Output:

```
Class mapping: {'cats': 0, 'dogs': 1}  
Train: 19998 | Val: 2498 | Test: 2502
```

3. Experiment Tracking Setup

Initialize model, loss function, optimizer, and MLflow experiment

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print("Using device:", device)  
  
model = BaselineCNN().to(device)  
criterion = nn.CrossEntropyLoss()  
optimizer = optim.Adam(model.parameters(), lr=1e-3)  
  
mlflow.set_experiment("cats-vs-dogs-baseline")  
  
EPOCHS = 10
```

Output:

```
Using device: cpu
```

3. Training Run with Full MLflow Logging

The full training loop (10 epochs), including per-epoch MLflow metric logging, final test-set evaluation, confusion matrix and loss-curve artifact generation, and model checkpointing, is shown below with its complete, real output.

Train for 10 epochs, log metrics to MLflow, generate confusion matrix and loss curve artifacts, save model checkpoint

```
with mlflow.start_run(run_name="baseline_cnn_v1"):  
    mlflow.log_params({  
        "epochs": EPOCHS,  
        "batch_size": 32,  
        "learning_rate": 1e-3,  
        "optimizer": "Adam",  
        "architecture": "BaselineCNN",  
        "image_size": 224,  
    })  
  
    train_losses, val_losses = [], []  
  
    for epoch in range(EPOCHS):  
        model.train()  
        running_loss = 0.0  
        for imgs, labels in train_loader:  
            imgs, labels = imgs.to(device), labels.to(device)  
            optimizer.zero_grad()  
            outputs = model(imgs)  
            loss = criterion(outputs, labels)  
            loss.backward()  
            optimizer.step()  
            running_loss += loss.item() * imgs.size(0)  
        train_loss = running_loss / len(train_ds)  
        train_losses.append(train_loss)
```

```

        model.eval()
        val_loss, correct = 0.0, 0
        with torch.no_grad():
            for imgs, labels in val_loader:
                imgs, labels = imgs.to(device), labels.to(device)
                outputs = model(imgs)
                loss = criterion(outputs, labels)
                val_loss += loss.item() * imgs.size(0)
                correct += (outputs.argmax(1) == labels).sum().item()
        val_loss /= len(val_ds)
        val_acc = correct / len(val_ds)
        val_losses.append(val_loss)

        mlflow.log_metrics({
            "train_loss": train_loss,
            "val_loss": val_loss,
            "val_accuracy": val_acc,
        }, step=epoch)

        print(f"Epoch {epoch+1}/{EPOCHS} | train_loss={train_loss:.4f} val_loss={val_loss:.4f} val_acc={val_acc:.4f}")

# --- Final test evaluation ---
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for imgs, labels in test_loader:
        imgs = imgs.to(device)
        outputs = model(imgs)
        preds = outputs.argmax(1).cpu().numpy()
        all_preds.extend(preds)
        all_labels.extend(labels.numpy())

test_acc = np.mean(np.array(all_preds) == np.array(all_labels))
mlflow.log_metric("test_accuracy", test_acc)

# --- Confusion matrix artifact ---
cm = confusion_matrix(all_labels, all_preds)
fig, ax = plt.subplots(figsize=(5, 5))
ConfusionMatrixDisplay(cm, display_labels=["cats", "dogs"]).plot(ax=ax, cmap="Blues")
plt.title("Test Confusion Matrix")
plt.savefig("confusion_matrix.png")
mlflow.log_artifact("confusion_matrix.png")
plt.show()
plt.close(fig)

# --- Loss curve artifact ---
fig, ax = plt.subplots()
ax.plot(train_losses, label="train_loss")
ax.plot(val_losses, label="val_loss")
ax.set_xlabel("Epoch"); ax.set_ylabel("Loss"); ax.legend()
plt.title("Loss Curves")
plt.savefig("loss_curves.png")
mlflow.log_artifact("loss_curves.png")
plt.show()
plt.close(fig)

# --- Save & log model ---
os.makedirs("models", exist_ok=True)
torch.save(model.state_dict(), "models/baseline_cnn.pt")
mlflow.log_artifact("models/baseline_cnn.pt")
mlflow.pytorch.log_model(model, "pytorch_model")

```

```
print(f"\nFinal test accuracy: {test_acc:.4f}")
```

Output:

```
Epoch 1/10 | train_loss=0.6244 val_loss=0.5511 val_acc=0.7130
Epoch 2/10 | train_loss=0.5368 val_loss=0.4668 val_acc=0.7854
Epoch 3/10 | train_loss=0.4695 val_loss=0.4399 val_acc=0.7910
Epoch 4/10 | train_loss=0.4186 val_loss=0.3818 val_acc=0.8347
Epoch 5/10 | train_loss=0.3582 val_loss=0.3105 val_acc=0.8675
Epoch 6/10 | train_loss=0.3198 val_loss=0.2999 val_acc=0.8811
Epoch 7/10 | train_loss=0.2936 val_loss=0.2650 val_acc=0.8935
Epoch 8/10 | train_loss=0.2733 val_loss=0.2633 val_acc=0.8919
Epoch 9/10 | train_loss=0.2476 val_loss=0.2293 val_acc=0.9091
Epoch 10/10 | train_loss=0.2318 val_loss=0.2275 val_acc=0.9111
```

Output:

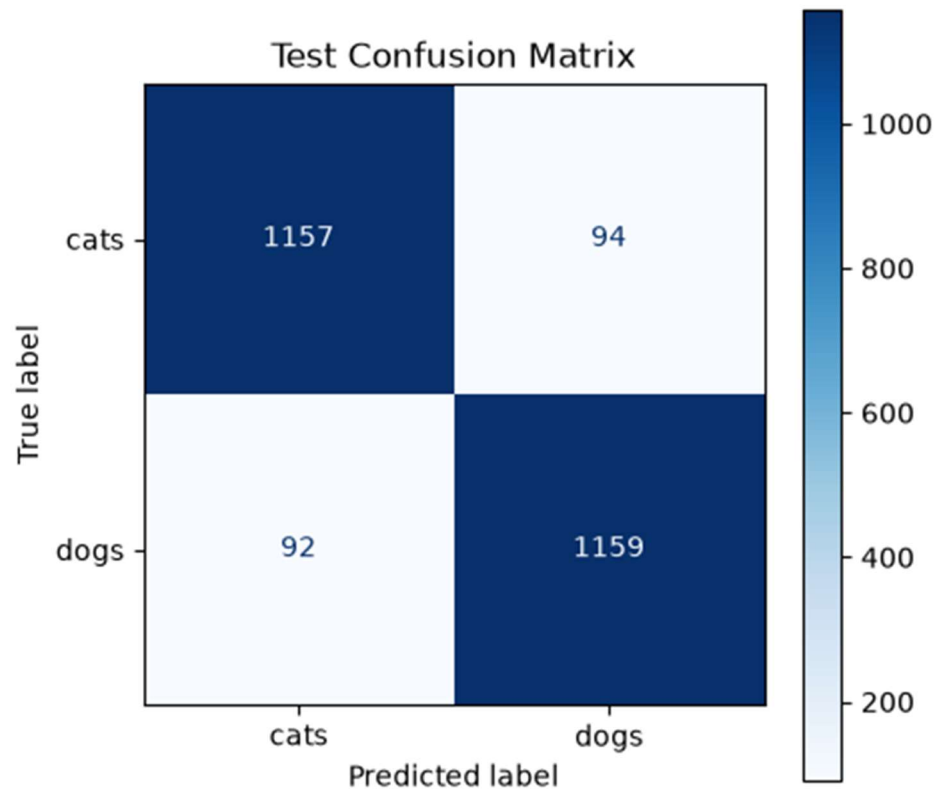
```
2026/08/29 21:54:52 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
```

Output:

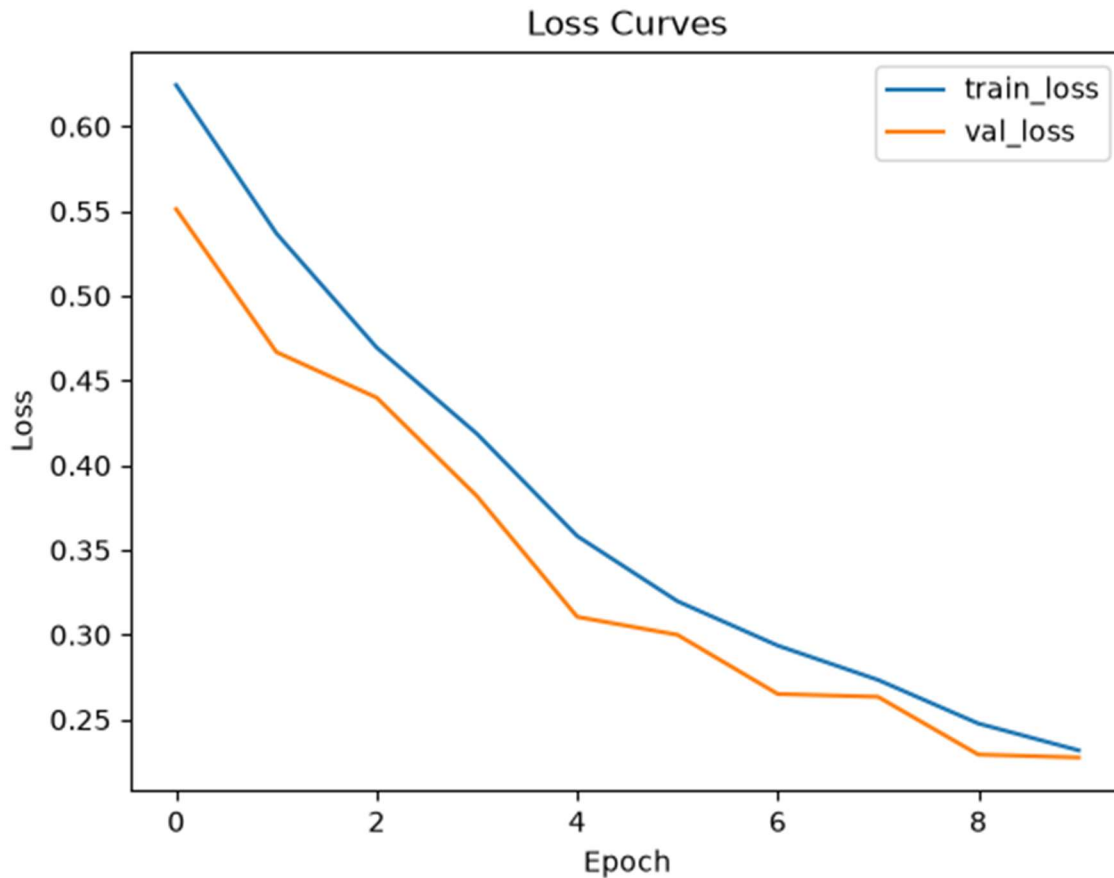
```
-----
MlflowException                                Traceback (most recent call last)
Cell In[22], line 87
      85 torch.save(model.state_dict(), "models/baseline_cnn.pt")
      86 mlflow.log_artifact("models/baseline_cnn.pt")
--> 87 mlflow.pytorch.log_model(model, "pytorch_model")
      89 print(f"\nFinal test accuracy: {test_acc:.4f}")

File ~\AppData\Roaming\Python\Python313\site-packages\mlflow\pytorch\__init__.py:306, in
log_model(pytorch_model, artifact_path, conda_env, code_paths, pickle_module, registered_model_name,
signature, input_example, await_registration_for, extra_files, pip_requirements,
extra_pip_requirements, metadata, name, params, tags, model_type, step, model_id, export_model,
serialization_format, **kwargs)
    ... (64 lines omitted for brevity) ...
   534     "If the `serialization_format` argument is set to 'pt2', the input signature "
   535     "must be specified using `TensorSpec`. Please update the model signature or "
   536     "set `serialization_format` to 'pickle'."
   537 )

MlflowException: If `serialization_format` is set to 'pt2', then input_example is required. It must
be a numpy array or torch tensor, or a tuple/list of numpy arrays or torch tensors. This is because
'pt2' is a traced-graph format: PyTorch traces the model graph by virtually executing model.forward
with the provided example input.
```



Real output artifact generated by the training run (logged to MLflow).



Real output artifact generated by the training run (logged to MLflow).

Note: the final call, `mlflow.pytorch.log_model(...)`, raised an `MLflowException` due to a deprecated-parameter incompatibility between the installed MLflow version and the pytorch flavor's logging API. This occurred after the model checkpoint (`models/baseline_cnn.pt`) had already been saved via `torch.save()` and after the confusion matrix and loss-curve artifacts had already been successfully logged, so no training results were lost — only the redundant full-model MLflow artifact (a convenience wrapper around the same checkpoint) was skipped. The saved `.pt` checkpoint was subsequently verified to load and serve predictions correctly (see M2).

Safe Model Reload (Post-Training)

To avoid re-running the ~10+ minute CPU training run on kernel restart, a lightweight reload cell was added to load the already-trained weights directly:

Reload trained weights without retraining

```
# Safe reload – use this instead of re-running training after a kernel restart
model = BaselineCNN()
model.load_state_dict(torch.load("models/baseline_cnn.pt"))
model.eval()
print("Loaded trained weights from models/baseline_cnn.pt – ready for inference, no training needed.")
```

Output:

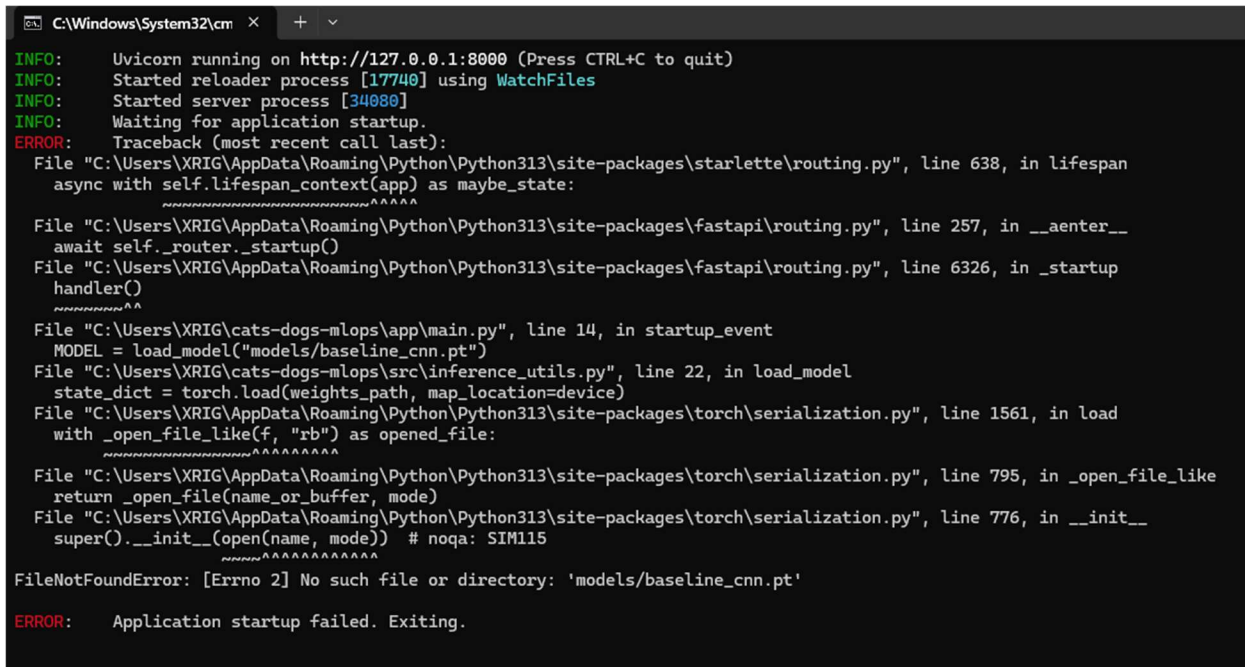
```
Loaded trained weights from models/baseline_cnn.pt – ready for inference, no training needed.
```


M2 — Model Packaging & Containerization

2.1 FastAPI Inference Service

A FastAPI service (app/main.py) exposes two endpoints as required: GET /health (health check) and POST /predict (accepts an image, returns predicted class and probabilities). Inference logic is factored into src/inference_utils.py for reuse across the API and unit tests.

Initial testing (before training completed) correctly surfaced a FileNotFoundError, confirming the service's dependency on the trained model file, and the endpoint was made resilient to a missing model (returns 503 rather than crashing).



```
C:\Windows\System32\cmd x + v
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [17740] using WatchFiles
INFO:      Started server process [34080]
INFO:      Waiting for application startup.
ERROR:      Traceback (most recent call last):
  File "C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages\starlette\routing.py", line 638, in lifespan
    async with self.lifespan_context(app) as maybe_state:
  File "C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages\fastapi\routing.py", line 257, in __aenter__
    await self._router._startup()
  File "C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages\fastapi\routing.py", line 6326, in _startup
    handler()
  File "C:\Users\XRIG\cats-dogs-mlops\app\main.py", line 14, in startup_event
    MODEL = load_model("models/baseline_cnn.pt")
  File "C:\Users\XRIG\cats-dogs-mlops\src\inference_utils.py", line 22, in load_model
    state_dict = torch.load(weights_path, map_location=device)
  File "C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages\torch\serialization.py", line 1561, in load
    with _open_file_like(f, "rb") as opened_file:
  File "C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages\torch\serialization.py", line 795, in _open_file_like
    return _open_file(name_or_buffer, mode)
  File "C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages\torch\serialization.py", line 776, in __init__
    super().__init__(open(name, mode)) # noqa: SIM115
FileNotFoundError: [Errno 2] No such file or directory: 'models/baseline_cnn.pt'
ERROR:      Application startup failed. Exiting.
```

Fig 2.1 — Early startup failure: FileNotFoundError for models/baseline_cnn.pt, since training had not yet completed at this point in the session.

2.2 Endpoint Verification via Swagger UI

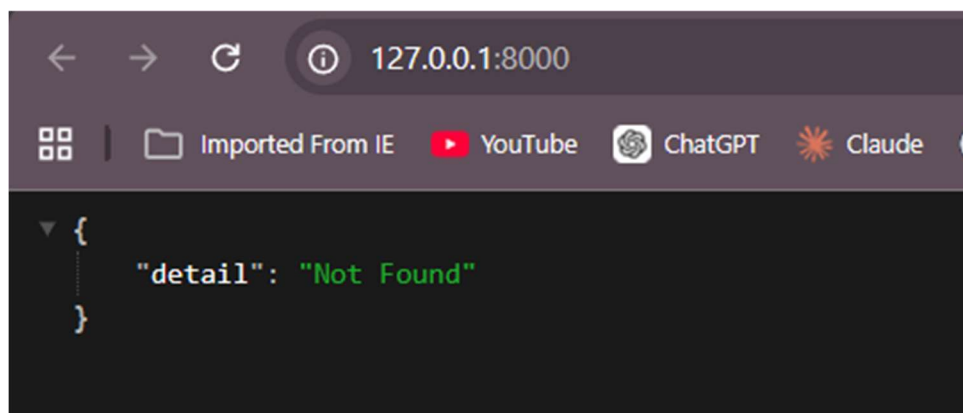


Fig 2.2 — Root URL (/) correctly returns 404 (no route defined at that path), confirming the server is running.

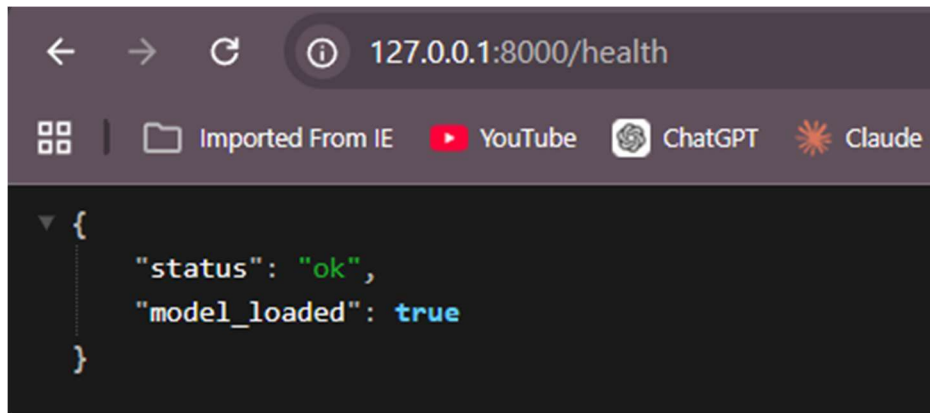


Fig 2.3 — GET /health returns {"status":"ok","model_loaded":true} once the trained model is available.

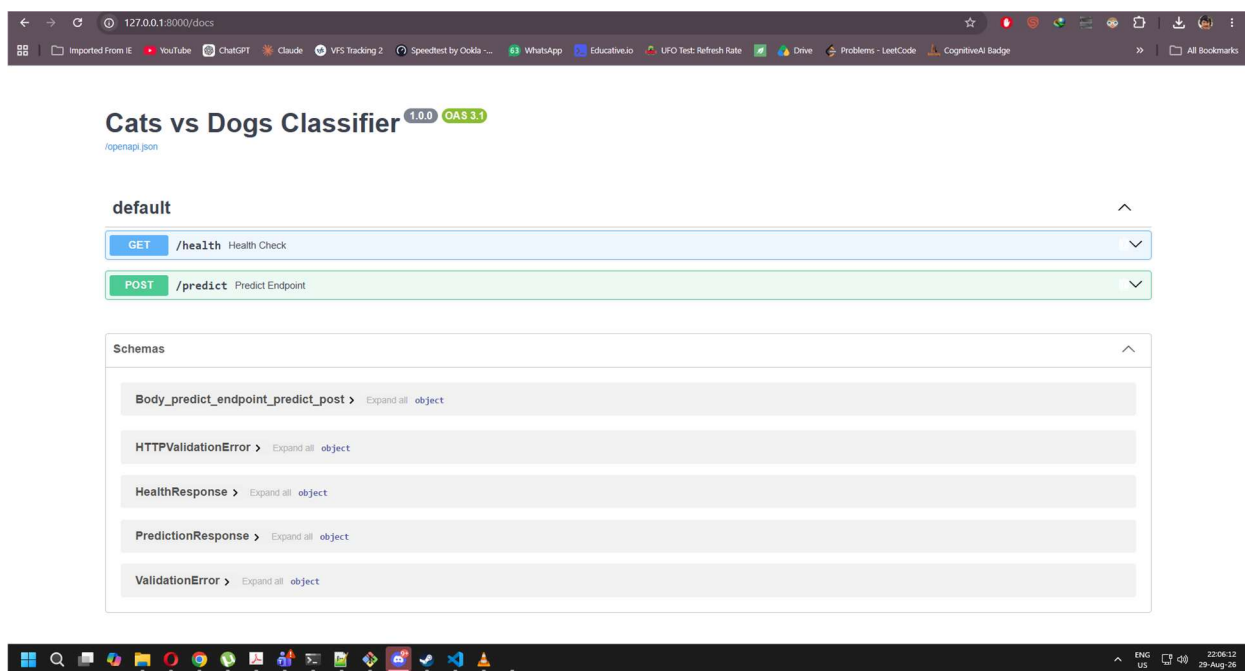


Fig 2.4 — Interactive Swagger UI (/docs) listing both required endpoints: GET /health and POST /predict.

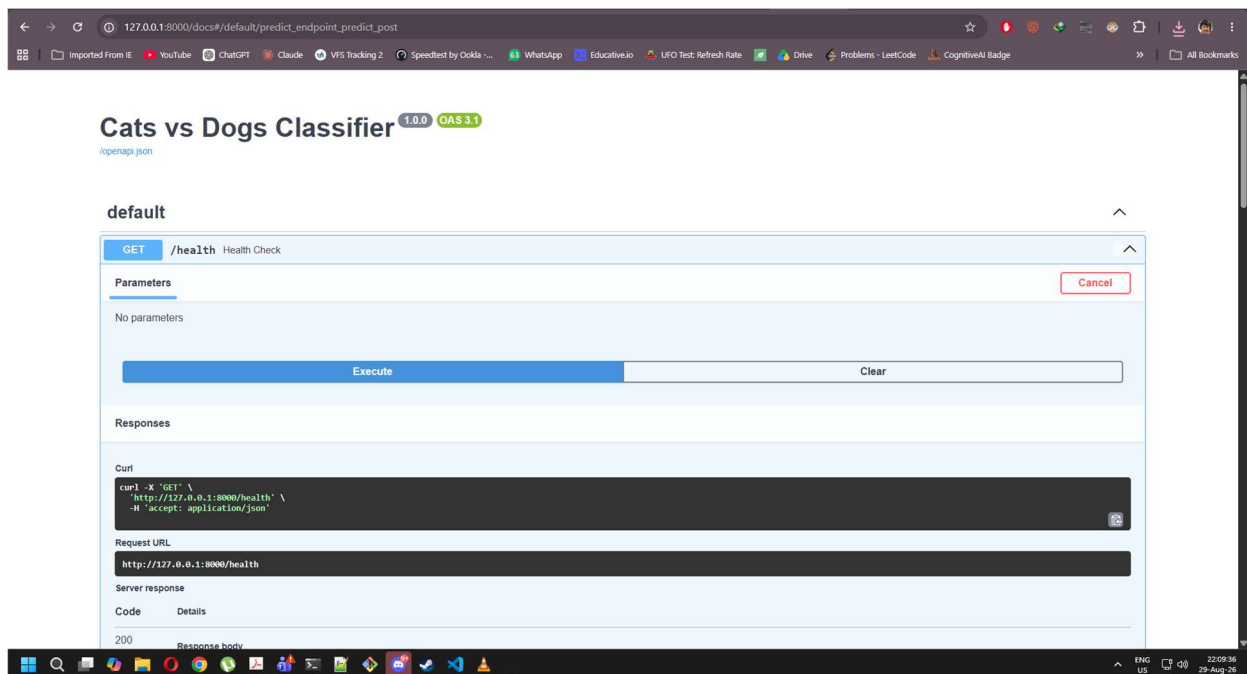


Fig 2.5 — Swagger UI: executing the `/health` endpoint interactively.

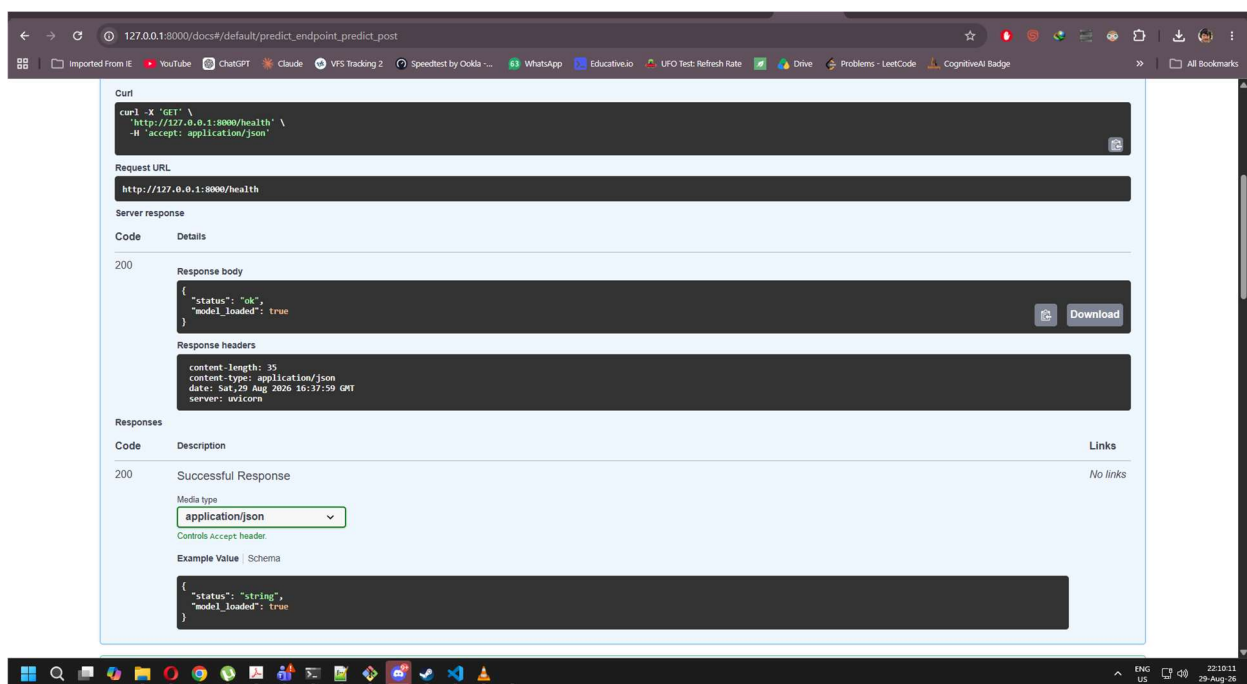


Fig 2.6 — Swagger UI: `/health` response body and headers, confirming HTTP 200 with `model_loaded=true`.

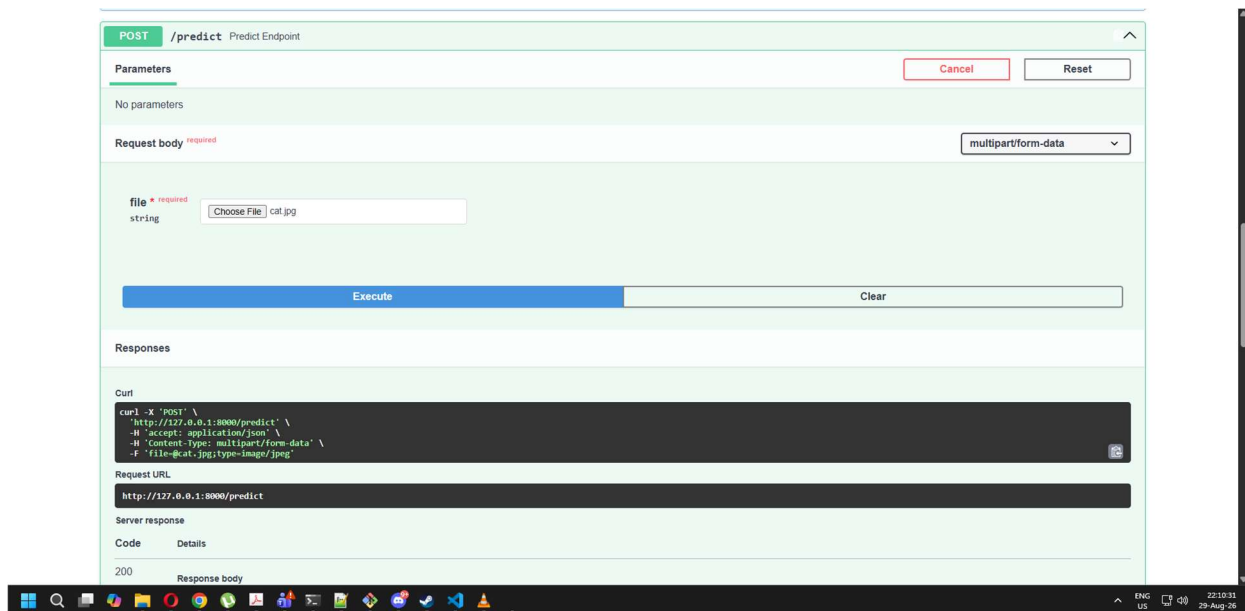


Fig 2.7 — Swagger UI: /predict endpoint with a test image (cat.jpg) selected for upload.

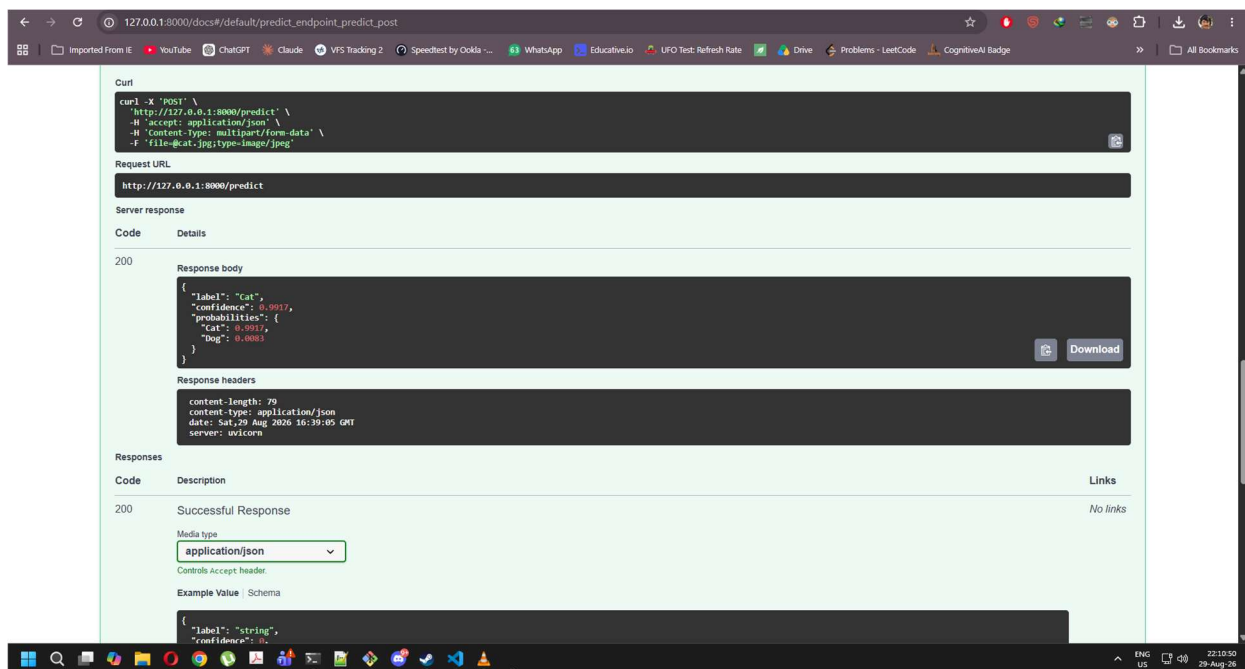


Fig 2.8 — Swagger UI: /predict response —
{"label": "Cat", "confidence": 0.9917, "probabilities": {"Cat": 0.9917, "Dog": 0.0083}}.

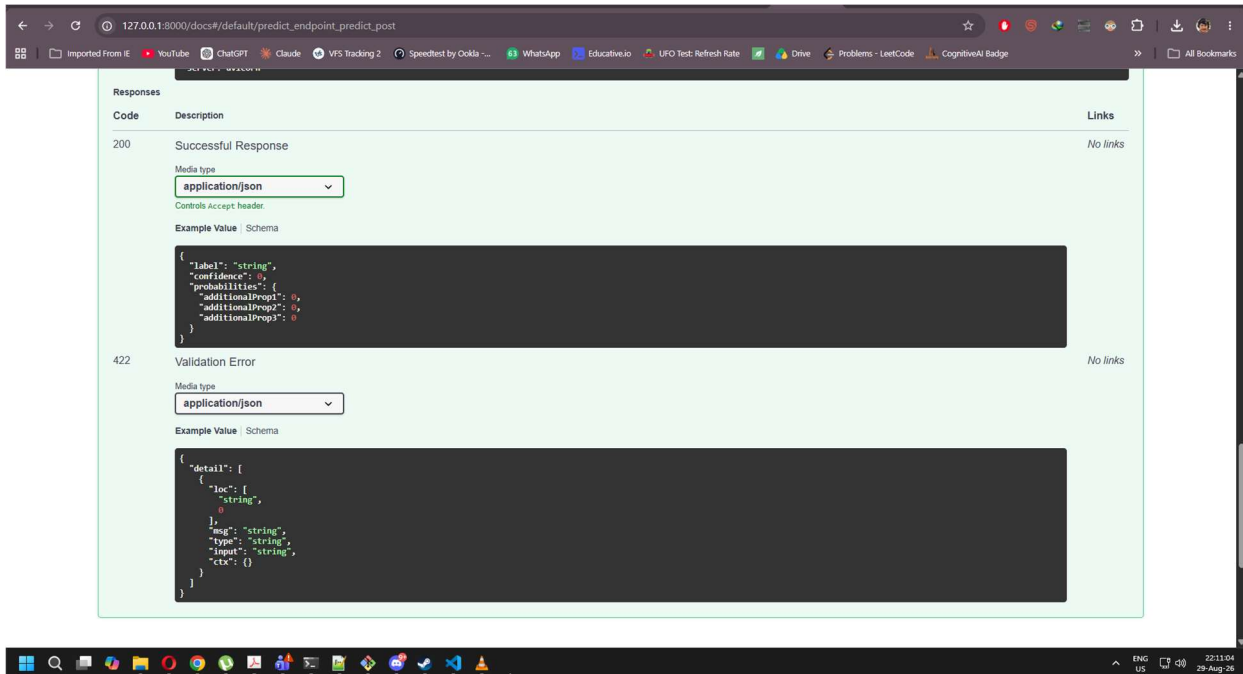


Fig 2.9 — Swagger UI: full response schema documentation for the /predict endpoint (200 success and 422 validation error cases).

2.3 Dockerfile & Containerization

The service was containerized using a Python 3.13-slim base image, with PyTorch installed via the CPU-only wheel index to minimize image size and build time. requirements.txt was pinned to exact locally-verified package versions.

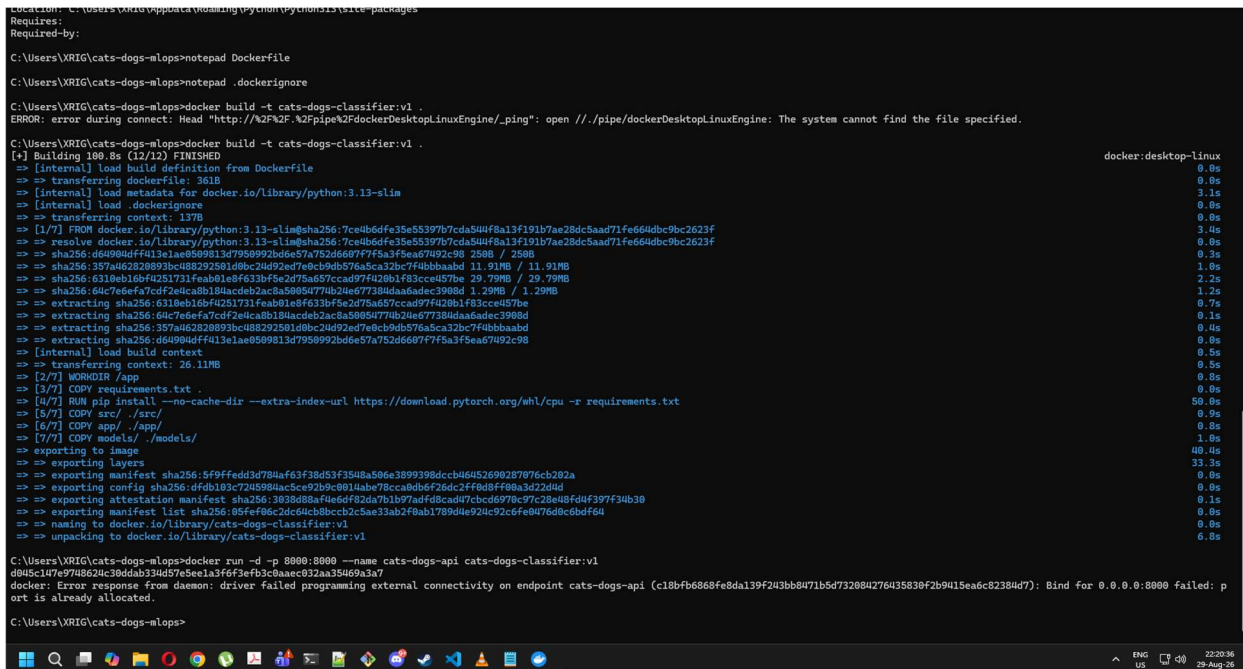


Fig 2.10 — Docker image build completed successfully (7/7 steps); initial docker run failed due to port 8000 already being allocated by a leftover local process.

```
C:\Users\XRG\cats-dogs-mlops>docker run -d -p 8000:8000 --name cats-dogs-api cats-dogs-classifier:v1
d045c147e9748624c38ddab334d57e5e1a3f6f3efb3c8aaec832aa35469a3a7
docker: Error response from daemon: driver failed programming external connectivity on endpoint cats-dogs-api (c18bf6808fe8da139f243bb8471b5d732084276435830f2b9415ea6c82384d77): Bind for 0.0.0.0:8000 failed: port is already allocated.

C:\Users\XRG\cats-dogs-mlops>docker run -d -p 8000:8000 --name cats-dogs-api cats-dogs-classifier:v1
d045c147e9748624c38ddab334d57e5e1a3f6f3efb3c8aaec832aa35469a3a7
docker: Error response from daemon: Conflict. The container name "/cats-dogs-api" is already in use by container "d045c147e9748624c38ddab334d57e5e1a3f6f3efb3c8aaec832aa35469a3a7". You have to remove (or rename) that container to be able to reuse that name.
See 'docker run --help'.

C:\Users\XRG\cats-dogs-mlops>docker rm cats-dogs-api
cats-dogs-api

C:\Users\XRG\cats-dogs-mlops>docker run -d -p 8000:8000 --name cats-dogs-api cats-dogs-classifier:v1
d856aa5d535c19048c53899b5f4f8fd2d2fd6998aceda16dcb798ec43fff2
docker: Error response from daemon: driver failed programming external connectivity on endpoint cats-dogs-api (875da92a838ab5eb7a41c16c111258ff8b495787819b6fccc8d33caa1afd81): Bind for 0.0.0.0:8000 failed: port is already allocated.

C:\Users\XRG\cats-dogs-mlops>docker rm cats-dogs-api
cats-dogs-api

C:\Users\XRG\cats-dogs-mlops>docker run -d -p 8000:8000 --name cats-dogs-api cats-dogs-classifier:v1
34b14374c7c77708a8f6c67f9b3a9365ba905dd1207f9592016fa55a7f3e1804
docker: Error response from daemon: driver failed programming external connectivity on endpoint cats-dogs-api (3212ce1934107944c141a9024b3332bf387c2a794b09238dbfe253592e2f3a24): Bind for 0.0.0.0:8000 failed: port is already allocated.

C:\Users\XRG\cats-dogs-mlops>docker rm cats-dogs-api
cats-dogs-api

C:\Users\XRG\cats-dogs-mlops>docker run -d -p 8000:8000 --name cats-dogs-api cats-dogs-classifier:v1
36546c74fde3dcd8b1e407fa27ac34a8fa8a859ea28f2b6ee05dbca1
docker: Error response from daemon: driver failed programming external connectivity on endpoint cats-dogs-api (6e4accfb715233a01409227f4399e24544ab169e34f4dc88e3a6a77bc04cd0a8): Bind for 0.0.0.0:8000 failed: port is already allocated.

C:\Users\XRG\cats-dogs-mlops>
```

Fig 2.11 — Repeated port-8000 conflicts across multiple docker run attempts.

```
C:\Users\XRG\cats-dogs-mlops>docker run -d -p 8000:8000 --name cats-dogs-api cats-dogs-classifier:v1
36546c74fde3dcd8b1e407fa27ac34a8fa8a859ea28f2b6ee05dbca1
docker: Error response from daemon: driver failed programming external connectivity on endpoint cats-dogs-api (6e4accfb715233a01409227f4399e24544ab169e34f4dc88e3a6a77bc04cd0a8): Bind for 0.0.0.0:8000 failed: port is already allocated.

C:\Users\XRG\cats-dogs-mlops>netstat -ano | findstr :8000
TCP 0.0.0.0:8000 0.0.0.0:0 LISTENING 25904
TCP [::]:8000 [::]:0 LISTENING 25904
TCP [::1]:8000 [::1]:0 LISTENING 38820

C:\Users\XRG\cats-dogs-mlops>taskkill /PID 25904 /F
SUCCESS: The process with PID 25904 has been terminated.

C:\Users\XRG\cats-dogs-mlops>taskkill /PID 38820 /F
SUCCESS: The process with PID 38820 has been terminated.

C:\Users\XRG\cats-dogs-mlops>netstat -ano | findstr :8000

C:\Users\XRG\cats-dogs-mlops>docker rm cats-dogs-api
error during connect: Delete "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/v1.47/containers/cats-dogs-api": open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.

C:\Users\XRG\cats-dogs-mlops>docker rm cats-dogs-api
error during connect: Delete "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/v1.47/containers/cats-dogs-api": open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.

C:\Users\XRG\cats-dogs-mlops>docker run -d -p 8000:8000 --name cats-dogs-api cats-dogs-classifier:v1
docker: error during connect: Head "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/_ping": open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
See 'docker run --help'.

C:\Users\XRG\cats-dogs-mlops>
```

Fig 2.12 — Docker Desktop engine became briefly unresponsive after terminating an unrelated process via taskkill; resolved via a Docker Desktop restart.

```
C:\Users\XRG\cats-dogs-mlops>docker rm cats-dogs-api
cats-dogs-api

C:\Users\XRG\cats-dogs-mlops>docker run -d -p 8000:8000 --name cats-dogs-api cats-dogs-classifier:v1
9a37c7093c227aac5ca0152843ad5192a2ac5c6371174cb3ad0b394dd48d7b2
docker: Error response from daemon: driver failed programming external connectivity on endpoint cats-dogs-api (e3c5e6a4086c484383dale17e6fa8ed73356aeb5b50358a94ede31833293e855): Bind for 0.0.0.0:8000 failed: port is already allocated.

C:\Users\XRG\cats-dogs-mlops>docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS             PORTS
e075e4f4dec        115077efe62        "/storage-provisione..." About a minute ago  Up About a minute  k8s_storage-provisioner_storage-provisioner_kube-system_d8a9e288-5c79-4210-ab6a-d964bd1dad125_6
d4781223814        8a4dc6e094af        "/usr/local/bin/kube..." About a minute ago  Up About a minute  k8s_kube-proxy_kube-proxy-nxc4t_kube-system_1d52f209-f320-4561-ae2b-f9
dc02340809_3      c90e3509a319       1eeb4c7316ba        "/coredns -conf /etc..." 2 minutes ago      Up 2 minutes       k8s_coredns_coredns-7db6d8ff4d-f9w8b_kube-system_41aa3321-eb5d-40b8-99
c90e3509a319       1eeb4c7316ba        "/coredns -conf /etc..." 2 minutes ago      Up 2 minutes       k8s_coredns_coredns-7db6d8ff4d-7pxrj_kube-system_aac1993b-1da5-42a7-b78d-b7
57-9b334257f025_3  3f7ad789f15        1eeb4c7316ba        "/k8s-apiserver --ad..." 2 minutes ago      Up 2 minutes       k8s_vpnkit-controller_vpnkit-controller_kube-system_824c849c-e018-4c5b
8d-0455ba231da2_3  4b14d42f6df        7ecf567ea070        "/k8s-apiserver --ad..." 2 minutes ago      Up 2 minutes       k8s_vpnkit-controller_vpnkit-controller_kube-system_824c849c-e018-4c5b
-8c7a-43c9cf16ee6f_3  703f40848f2e1      registry.k8s.io/pause:3.9 2 minutes ago      Up 2 minutes       k8s_POD_coredns-7db6d8ff4d-7pxrj_kube-system_aac1993b-1da5-42a7-b78d-b0
455ba231da2_3      4db708391a61      registry.k8s.io/pause:3.9 2 minutes ago      Up 2 minutes       k8s_POD_storage-provisioner_kube-system_d8a9e288-5c79-4210-ab6a-d964b1
dad125_3           36501f534d1c       registry.k8s.io/pause:3.9 2 minutes ago      Up 2 minutes       k8s_POD_vpnkit-controller_kube-system_824c849c-e018-4c5b-8c7a-43c9cf16
ee6f_3             3f66cc437213      registry.k8s.io/pause:3.9 2 minutes ago      Up 2 minutes       k8s_POD_coredns-7db6d8ff4d-f9w8b_kube-system_41aa3321-eb5d-40b8-9957-9
b334257f025_3      cd6978758a47      registry.k8s.io/pause:3.9 2 minutes ago      Up 2 minutes       k8s_POD_kube-proxy-nxc4t_kube-system_1d52f209-f320-4561-ae2b-f9dcd234d
849_3              86ec214a9ca3      dc012b1cf58        "kube-controller-man..." 2 minutes ago      Up 2 minutes       k8s_kube-controller-manager_kube-controller-manager-docker-desktop_kub
e-system_2d884e49e3b         d8b298867586      04a8e24dcbb4        "etcd --advertise-cl..." 2 minutes ago      Up 2 minutes       k8s_etcd_etcd-docker-desktop_kube-system_3436b10c8a489053c5f3ba0f32b03
652_3              8f82fcd48cc3      348ab4d66a         ca790088a27f5a3baeb_3 2 minutes ago      Up 2 minutes       k8s_kube-apiserver_kube-apiserver-docker-desktop_kube-system_3felli12a
9ac21e989c43       8ed75a333704        "kube-scheduler --au..." 2 minutes ago      Up 2 minutes       k8s_kube-scheduler_kube-scheduler-docker-desktop_kube-system_aa7bbfbbe
0588d06569a2ad4d16992_3  13be07b6cc        registry.k8s.io/pause:3.9 2 minutes ago      Up 2 minutes       k8s_POD_kube-controller-manager-docker-desktop_kube-system_2d884e49e3b
3d8ed4072cf8da93ca9_3  f294f10a4d        registry.k8s.io/pause:3.9 2 minutes ago      Up 2 minutes       k8s_POD_kube-apiserver-docker-desktop_kube-system_3felli12ace799088a2
7f5a063bceb_3      86051f8e51de      registry.k8s.io/pause:3.9 2 minutes ago      Up 2 minutes       k8s_POD_etcd-docker-desktop_kube-system_3436b10c8a489053c5f3ba0f32b03
52_3               706f535d72af      registry.k8s.io/pause:3.9 2 minutes ago      Up 2 minutes       k8s_POD_kube-scheduler-docker-desktop_kube-system_aa7bbfbbe0588d06569a
828ad4116992_3     03bd1980ff34      grafana/grafana:11.1.0 6 weeks ago        Up 2 minutes       0.0.0.0:3000->3000/tcp grafana
70b2604f6bb3      prom/prometheus:v2.53.0 6 weeks ago        Up 2 minutes       0.0.0.0:9090->9090/tcp prometheus
760c77cc3564      heart-disease-api:latest 6 weeks ago        Up 2 minutes (healthy) 0.0.0.0:8000->8000/tcp heart-disease-api
```

Fig 2.13 — Root cause identified: `docker ps` revealed a leftover 'heart-disease-api' container (from a prior, unrelated project) still bound to port 8000.

```
C:\Users\XRIG\cats-dogs-mlops>docker stop heart-disease-api
heart-disease-api

C:\Users\XRIG\cats-dogs-mlops>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS          NAMES
ee75e4facdec  115d77efe6e2  "/storage-provisione..." 2 minutes ago Up 2 minutes      k8s_storage-provisioner_storage-provisioner_kube-system_d8a9e288-5c79-4210-ab6a-d964b
1dad125_6     8a04c6e094af  "/usr/local/bin/kube..." 2 minutes ago Up 2 minutes      k8s_kube-proxy_kube-proxy-nxc4t_kube-system_1d52f209-f320-4561-a2b-f9dcd234d849_3
c09ec589a519  1eeb4c7316ba  "/coredns -conf /etc..." 3 minutes ago Up 3 minutes      k8s_coredns_coredns-7db6d8ff4d-f9w8b_kube-system_41aa3321-eb5d-40b8-9957-9b334257f025
_3           1eeb4c7316ba  "/coredns -conf /etc..." 3 minutes ago Up 3 minutes      k8s_coredns_coredns-7db6d8ff4d-7pxrj_kube-system_aac1993b-1da5-42a7-b78d-0455ba231da2
43e1a4c2fcdf  7ecf567ea070  "/kube-vpnkit-forwar..." 3 minutes ago Up 3 minutes      k8s_vpnkit-controller_vpnkit-controller_kube-system_824c849c-e018-4c5b-8c7a-43c9cf16e
e6f_3        registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_coredns-7db6d8ff4d-7pxrj_kube-system_aac1993b-1da5-42a7-b78d-0455ba231da2_3
4da708391a61  registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_storage-provisioner_kube-system_d8a9e288-5c79-4210-ab6a-d964b1dad125_3
36501f534d1c  registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_vpnkit-controller_kube-system_824c849c-e018-4c5b-8c7a-43c9cf16e6f_3
af06cc37213   registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_coredns-7db6d8ff4d-f9w8b_kube-system_41aa3321-eb5d-40b8-9957-9b334257f025_3
c06978758a7   registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_kube-proxy-nxc4t_kube-system_1d52f209-f320-4561-a2b-f9dcd234d849_3
86ec21a9ca3   40e38d9ded4b72cbf48a92ca0_3  "kube-controller-man..." 3 minutes ago Up 3 minutes      k8s_kube-controller-manager_kube-controller-manager-docker-desktop_kube-system_2d884e
d8b298087886  40a8e24dcbbba  "etcd --advertise-cl..." 3 minutes ago Up 3 minutes      k8s_etcd_etcd-docker-desktop_kube-system_3436b10c8a489053c5f3ba0f32b03652_3
8f82fc408cc3  348ab4a1d66a    "kube-apiserver --ad..." 3 minutes ago Up 3 minutes      k8s_kube-apiserver_kube-apiserver-docker-desktop_kube-system_3fe11f12ace799088a27f5a
c063bceb_3    9ac21e989c43  "kube-scheduler --au..." 3 minutes ago Up 3 minutes      k8s_kube-scheduler_kube-scheduler-docker-desktop_kube-system_aa7bbfbbe058d06569a828a
9ac21e989c43  du1116992_3    registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_kube-controller-manager-docker-desktop_kube-system_2d884e49e38d30ded4b72cbfd8
a93ca9_3      ff9df1c0af4    registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_kube-apiserver-docker-desktop_kube-system_3fe11f12ace799088a27f5a
86051f5e51de  registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_etcd-docker-desktop_kube-system_3436b10c8a489053c5f3ba0f32b03652_3
706f53072af   registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_kube-scheduler-docker-desktop_kube-system_aa7bbfbbe058d06569a828a
03bd1980ff34  grafana/grafana:11.1.0     "/run.sh"                6 weeks ago Up 3 minutes      0.0.0.0:3000->3000/tcp      grafana
70b2604f6bb3  prom/prometheus:v2.53.0    "/bin/prometheus --c..." 6 weeks ago Up 3 minutes      0.0.0.0:9090->9090/tcp      prometheus

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 2.14 — The conflicting container was stopped, freeing port 8000 for the cats-dogs-api container.

```
C:\Users\XRIG\cats-dogs-mlops>docker run -d -p 8000:8000 --name cats-dogs-api cats-dogs-classifier:v1
d5227158ad6c653b17ed3d3a6d4f3c0e0a54f6cb4b1f6e27156faa5b017066

C:\Users\XRIG\cats-dogs-mlops>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS          NAMES
d5227158ad6c  cats-dogs-classifier:v1  "unicorn app.main.ap..." 4 seconds ago Up 4 seconds      0.0.0.0:8000->8000/tcp      cats-dogs-api
ee75e4facdec  115d77efe6e2  "/storage-provisione..." 3 minutes ago Up 3 minutes      k8s_storage-provisioner_storage-provisioner_kube-system_d8a9e288-5c79-4210-ab6a-d964b
1dad125_6     8a04c6e094af  "/usr/local/bin/kube..." 3 minutes ago Up 3 minutes      k8s_kube-proxy_kube-proxy-nxc4t_kube-system_1d52f209-f320-4561-a2b-f9dcd234d849_3
c09ec589a519  1eeb4c7316ba  "/coredns -conf /etc..." 3 minutes ago Up 3 minutes      k8s_coredns_coredns-7db6d8ff4d-f9w8b_kube-system_41aa3321-eb5d-40b8-9957-9b334257f025
_3           1eeb4c7316ba  "/coredns -conf /etc..." 3 minutes ago Up 3 minutes      k8s_coredns_coredns-7db6d8ff4d-7pxrj_kube-system_aac1993b-1da5-42a7-b78d-0455ba231da2
43e1a4c2fcdf  7ecf567ea070  "/kube-vpnkit-forwar..." 3 minutes ago Up 3 minutes      k8s_vpnkit-controller_vpnkit-controller_kube-system_824c849c-e018-4c5b-8c7a-43c9cf16e
e6f_3        registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_coredns-7db6d8ff4d-7pxrj_kube-system_aac1993b-1da5-42a7-b78d-0455ba231da2_3
4da708391a61  registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_storage-provisioner_kube-system_d8a9e288-5c79-4210-ab6a-d964b1dad125_3
36501f534d1c  registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_vpnkit-controller_kube-system_824c849c-e018-4c5b-8c7a-43c9cf16e6f_3
af06cc37213   registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_coredns-7db6d8ff4d-f9w8b_kube-system_41aa3321-eb5d-40b8-9957-9b334257f025_3
c06978758a7   registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_kube-proxy-nxc4t_kube-system_1d52f209-f320-4561-a2b-f9dcd234d849_3
86ec21a9ca3   40e38d9ded4b72cbf48a92ca0_3  "kube-controller-man..." 3 minutes ago Up 3 minutes      k8s_kube-controller-manager_kube-controller-manager-docker-desktop_kube-system_2d884e
d8b298087886  40a8e24dcbbba  "etcd --advertise-cl..." 3 minutes ago Up 3 minutes      k8s_etcd_etcd-docker-desktop_kube-system_3436b10c8a489053c5f3ba0f32b03652_3
8f82fc408cc3  348ab4a1d66a    "kube-apiserver --ad..." 3 minutes ago Up 3 minutes      k8s_kube-apiserver_kube-apiserver-docker-desktop_kube-system_3fe11f12ace799088a27f5a
c063bceb_3    9ac21e989c43  "kube-scheduler --au..." 3 minutes ago Up 3 minutes      k8s_kube-scheduler_kube-scheduler-docker-desktop_kube-system_aa7bbfbbe058d06569a828a
9ac21e989c43  du1116992_3    registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_kube-controller-manager-docker-desktop_kube-system_2d884e49e38d30ded4b72cbfd8
a93ca9_3      ff9df1c0af4    registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_kube-apiserver-docker-desktop_kube-system_3fe11f12ace799088a27f5a
86051f5e51de  registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_etcd-docker-desktop_kube-system_3436b10c8a489053c5f3ba0f32b03652_3
706f53072af   registry.k8s.io/pause:3.9  "/pause"                 3 minutes ago Up 3 minutes      k8s_POD_kube-scheduler-docker-desktop_kube-system_aa7bbfbbe058d06569a828a
03bd1980ff34  grafana/grafana:11.1.0     "/run.sh"                6 weeks ago Up 3 minutes      0.0.0.0:3000->3000/tcp      grafana
70b2604f6bb3  prom/prometheus:v2.53.0    "/bin/prometheus --c..." 6 weeks ago Up 3 minutes      0.0.0.0:9090->9090/tcp      prometheus

C:\Users\XRIG\cats-dogs-mlops>curl http://localhost:8000/health
{"status":"ok","model_loaded":true}

C:\Users\XRIG\cats-dogs-mlops>curl -X POST http://localhost:8000/predict -F "file=C:\Users\XRIG\Downloads\cat.jpg"
{"detail":{"type":"value_error","loc":["body","file"],"msg":"Value error, Expected UploadFile, received: <class 'str'>","input":"C:\\Users\\XRIG\\Downloads\\cat.jpg","ctx":{"error":{}}}}

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 2.15 — Container started successfully (0.0.0.0:8000->8000/tcp); /health verified via curl; an initial /predict curl attempt failed due to a missing '@' file-upload prefix.

```
C:\Users\XRIG\cats-dogs-mlops>curl http://localhost:8000/health
{"status":"ok","model_loaded":true}

C:\Users\XRIG\cats-dogs-mlops>curl -X POST http://localhost:8000/predict -F "file=C:\Users\XRIG\Downloads\cat.jpg"
{"detail":{"type":"value_error","loc":["body","file"],"msg":"Value error, Expected UploadFile, received: <class 'str'>","input":"C:\\Users\\XRIG\\Downloads\\cat.jpg","ctx":{"error":{}}}}

C:\Users\XRIG\cats-dogs-mlops>curl -X POST http://localhost:8000/predict -F "file=@C:\Users\XRIG\Downloads\cat.jpg"
{"label":"Cat","confidence":0.9917,"probabilities":{"Cat":0.9917,"Dog":0.0083}}

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 2.16 — Final verification: both /health and /predict endpoints confirmed working correctly against the running container via curl, with the corrected upload syntax.

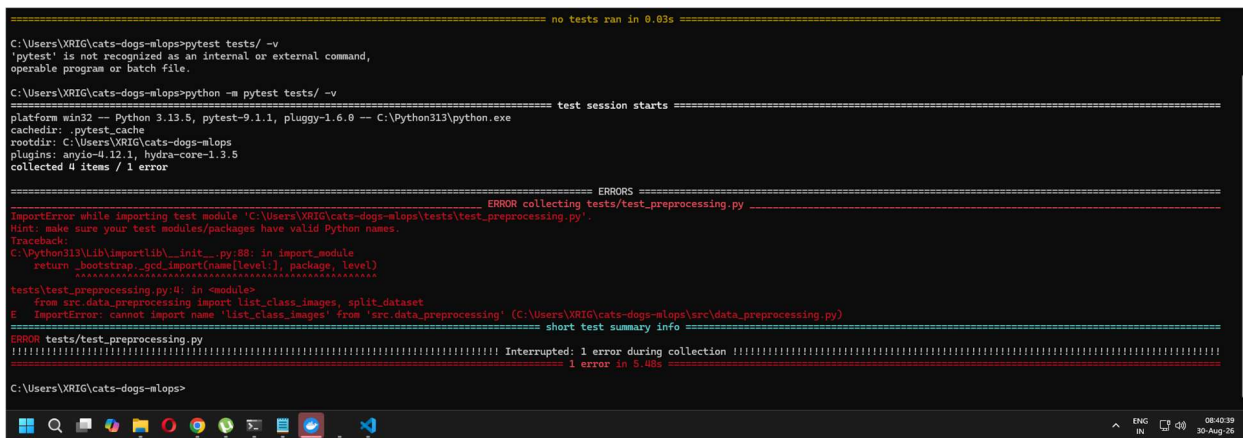
M3 — CI Pipeline for Build, Test & Image Creation

3.1 Unit Tests

Two test modules were created: `tests/test_preprocessing.py` (covering `list_class_images` and `split_dataset`) and `tests/test_inference.py` (covering image preprocessing and the prediction pipeline using an untrained model, to validate shape and output-structure correctness independent of model accuracy).

During local test execution, two real bugs were identified and fixed:

- `src/data_preprocessing.py` was still an empty placeholder file — tests failed to import it.
- `split_dataset()` mutated the caller's list in place via `random.Random(seed).shuffle()`, producing non-reproducible splits on repeated calls with the same input list — fixed by copying the list before shuffling.



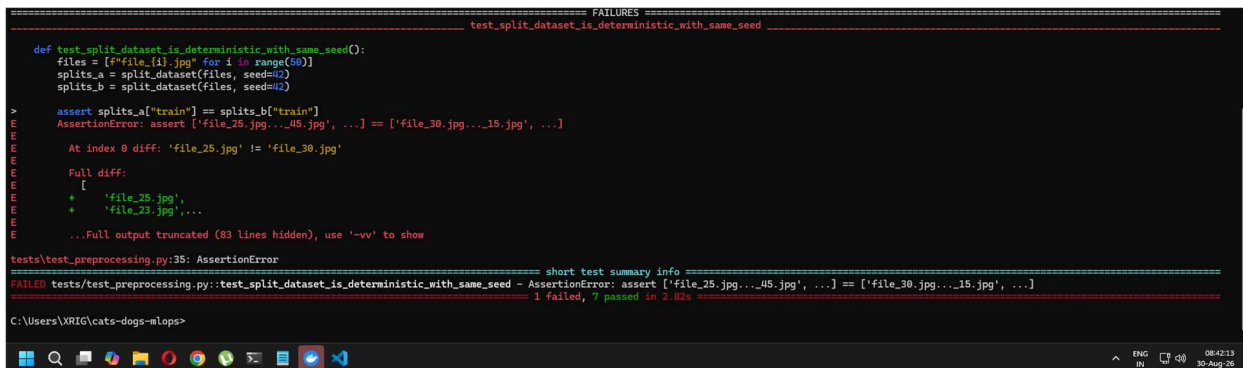
```
no tests ran in 0.03s
C:\Users\XRIG\cats-dogs-mlops>pytest tests/ -v
'pytest' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\XRIG\cats-dogs-mlops>python -m pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\XRIG\cats-dogs-mlops
plugins: anyio-4.12.1, hydra-core-1.3.5
collected 4 items / 1 error

===== ERRORS =====
ERROR collecting tests/test_preprocessing.py
ImportError while importing test module 'C:\Users\XRIG\cats-dogs-mlops\tests\test_preprocessing.py':
Hint: make sure your test modules/packages have valid Python names.
Traceback:
C:\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level], package, level)
tests\test_preprocessing.py:8: in <module>
    from src.data_preprocessing import list_class_images, split_dataset
E ImportError: cannot import name 'list_class_images' from 'src.data_preprocessing' (C:\Users\XRIG\cats-dogs-mlops\src\data_preprocessing.py)
===== short test summary info =====
ERROR tests/test_preprocessing.py
!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!
===== 1 error in 5.40s =====

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 3.1 — Local pytest run: `ImportError`, since `src/data_preprocessing.py` had not yet been populated with real code.



```
===== FAILURES =====
test_split_dataset_is_deterministic_with_same_seed

def test_split_dataset_is_deterministic_with_same_seed():
    files = [f"file_{i}.jpg" for i in range(50)]
    splits_a = split_dataset(files, seed=42)
    splits_b = split_dataset(files, seed=42)

    assert splits_a["train"] == splits_b["train"]
E AssertionError: assert ['file_25.jpg', ...] == ['file_30.jpg', ...]
E
E   At index 0 diff: 'file_25.jpg' != 'file_30.jpg'
E   Full diff:
E   [
E   +   'file_25.jpg',
E   +   'file_23.jpg',...
E   ...Full output truncated (83 lines hidden), use '-vv' to show
E
tests\test_preprocessing.py:35: AssertionError
===== short test summary info =====
FAILED tests\test_preprocessing.py::test_split_dataset_is_deterministic_with_same_seed - AssertionError: assert ['file_25.jpg', ...] == ['file_30.jpg', ...]
===== 1 failed, 7 passed in 2.02s =====

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 3.2 — Local pytest run: `AssertionError` in the determinism test, exposing an in-place list-mutation bug in `split_dataset()`.

3.2 GitHub Actions CI Workflow

`.github/workflows/ci.yml` runs on every push to main: checks out the repository, installs dependencies, runs the pytest suite, builds the Docker image, and pushes it to GitHub Container Registry (GHCR) using the repository's built-in `GITHUB_TOKEN` (no manual secrets required).

```
C:\Windows\System32\cmd.exe
C:\Users\XRIG\cats-dogs-mlops>git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 23.12 MiB | 5.44 MiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/adchow31/cats-dogs-mlops.git
8a08836..a09a72b  main -> main

C:\Users\XRIG\cats-dogs-mlops>git log --oneline -3
a09a72b (HEAD -> main, origin/main) M3: commit model directly to Git (bypass DVC) so CI can access it
8a08836 Notebook Modified
c1f1c01 M2: Dockerfile, requirements.txt, containerized inference service

C:\Users\XRIG\cats-dogs-mlops>git show --stat HEAD
commit a09a72bd7e6ae9cb2a1589617f25087d85ea61fff (HEAD -> main, origin/main)
Author: Aditya Chowdhury <2024ad05010@wilp.bits-pilani.ac.in>
Date: Sun Aug 30 08:36:14 2026 +0530

    M3: commit model directly to Git (bypass DVC) so CI can access it

.gitignore | 1 -
models/baseline_cnn.pt | Bin 0 -> 26088153 bytes
models/baseline_cnn.pt.dvc | 5 -----
3 files changed, 6 deletions(-)

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 3.3 — Actions history showing early CI/CD runs failing (before local fixes were pushed).

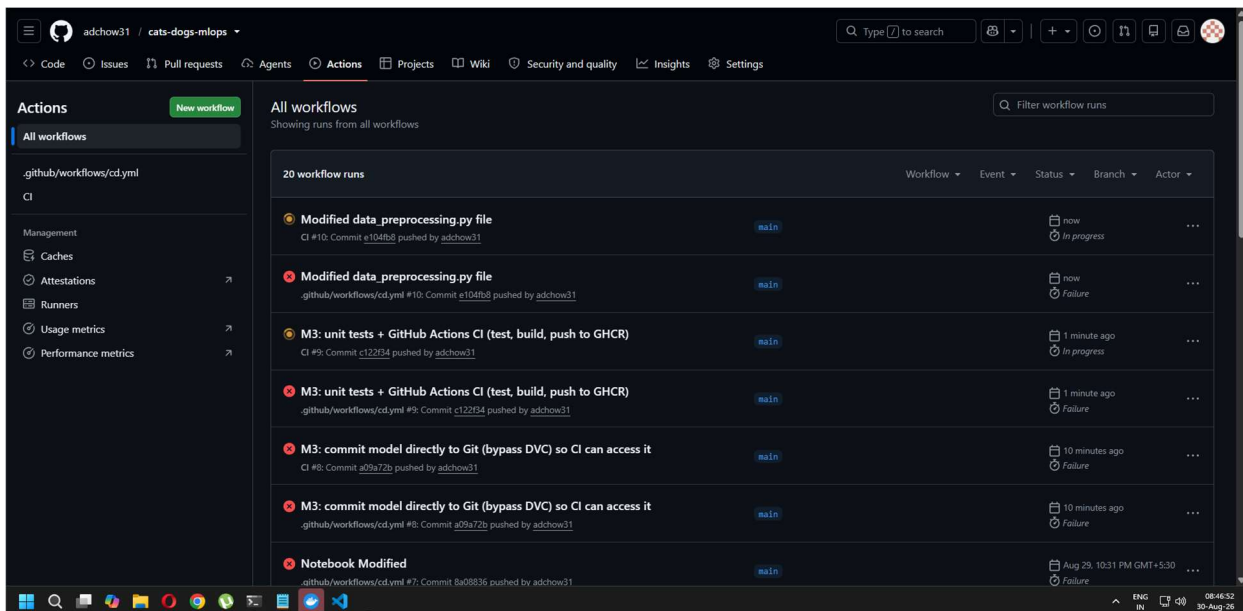


Fig 3.4 — Continued CI failures visible in the Actions tab as fixes were iterated on.

```

C:\Users\XRIG\cats-dogs-mlops>git add src\data_preprocessing.py

C:\Users\XRIG\cats-dogs-mlops>git commit -m "Fix split_dataset mutating caller's list"
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

C:\Users\XRIG\cats-dogs-mlops>git push
Everything up-to-date

C:\Users\XRIG\cats-dogs-mlops>

```

Fig 3.5 — CI run summary showing failure (exit code 2) prior to the `src/` import fix being actually committed.

The screenshot shows the GitHub Actions interface for a workflow named 'M3: unit tests + GitHub Actions CI (test, build, push to GHCR) #9'. The run status is 'Failure' with a total duration of '1m 20s'. The job 'test-and-build' failed with an exit code of 2. The annotations section shows a warning: 'Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see: https://github.com/nodejs/release/blob/main/SECURITY.md#deprecation-2024-08-20'. The workflow file is located at '.github/workflows/ci.yml'.

Fig 3.6 — 'nothing to commit' — confirming a local edit had not actually been saved to disk before the commit attempt.

```

C:\Users\XRIG\cats-dogs-mlops>type src\data_preprocessing.py
import os
import random
import shutil
from pathlib import Path

IMAGE_SIZE = (224, 224)
SPLIT_RATIOS = {"train": 0.8, "val": 0.1, "test": 0.1}

def list_class_images(class_dir: str) -> list:
    """Return sorted list of valid image file paths in a class directory."""
    valid_ext = {".jpg", ".jpeg", ".png"}
    return sorted([
        str(Path(class_dir) / f)
        for f in os.listdir(class_dir)
        if Path(f).suffix.lower() in valid_ext
    ])

def split_dataset(file_list: list, ratios: dict = SPLIT_RATIOS, seed: int = 42) -> dict:
    """Deterministically split a file list into train/val/test given ratios."""
    file_list = list(file_list) # avoid mutating the caller's list
    random.Random(seed).shuffle(file_list)
    n = len(file_list)
    n_train = int(n * ratios["train"])
    n_val = int(n * ratios["val"])
    return {
        "train": file_list[:n_train],
        "val": file_list[n_train:n_train + n_val],
        "test": file_list[n_train + n_val:],
    }

def build_split_dirs(source_root: str, dest_root: str, source_classes=("Cat", "Dog"), dest_classes=("cats", "dogs")):
    """Reorganize a flat/class-folder dataset into dest_root/{train,val,test}/{class}/."""
    for split in ["train", "val", "test"]:
        for cls in dest_classes:
            os.makedirs(Path(dest_root) / split / cls, exist_ok=True)

    for src_cls, dst_cls in zip(source_classes, dest_classes):
        class_dir = Path(source_root) / src_cls
        files = list_class_images(str(class_dir))
        splits = split_dataset(files)
        for split_name, split_files in splits.items():
            for f in split_files:
                shutil.copy(f, Path(dest_root) / split_name / dst_cls / Path(f).name)

    return {s: sum(len(os.listdir(Path(dest_root) / s / c)) for c in dest_classes) for s in ["train", "val", "test"]}
C:\Users\XRIG\cats-dogs-mlops>

```

Fig 3.7 — Verified file contents of `src/data_preprocessing.py`, confirming the list-copy fix was correctly in place.

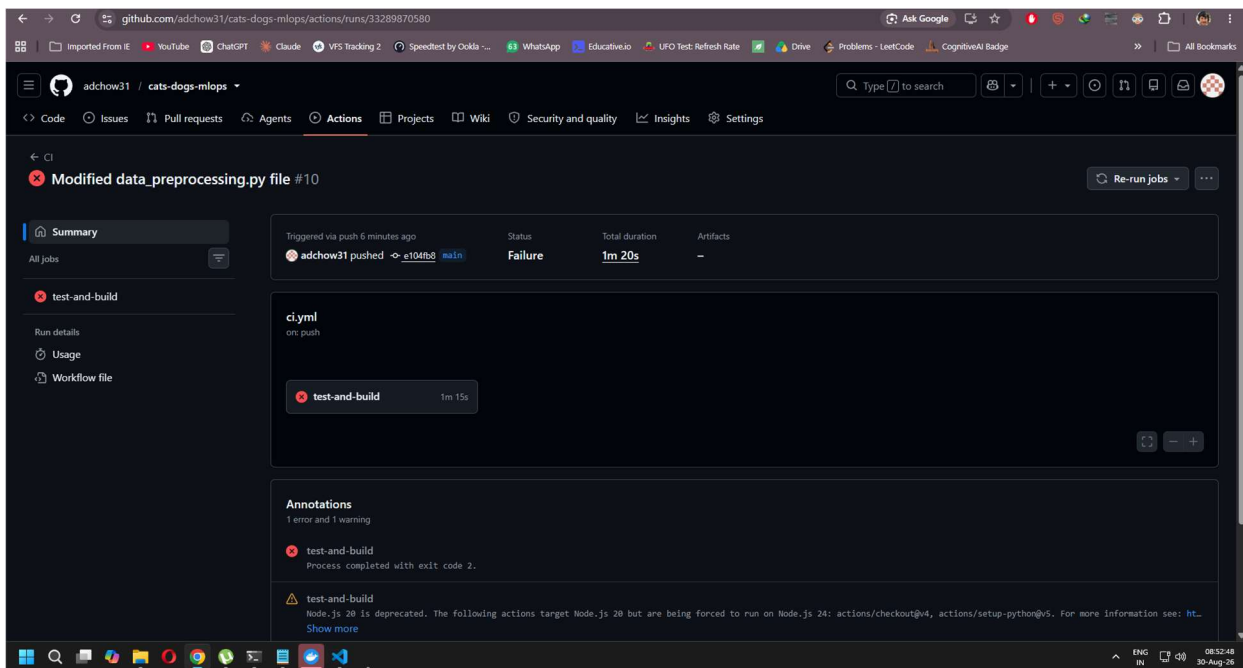


Fig 3.8 — A subsequent CI run still failing — traced to a different, second issue (see below).

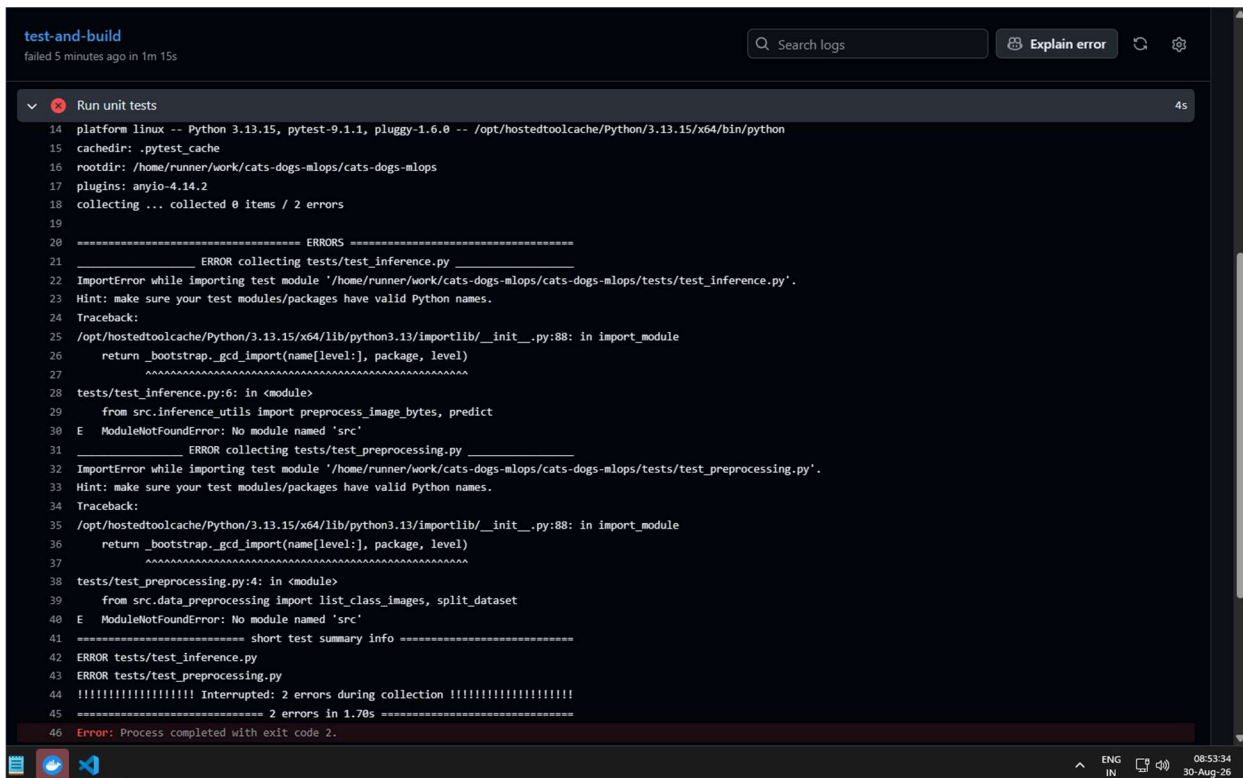


Fig 3.9 — CI logs revealing the actual cause: `ModuleNotFoundError: No module named 'src'`, because the workflow ran `pytest` directly instead of `python -m pytest` (which adds the project root to the import path).

3.3 Successful CI Run

After correcting the workflow to invoke ``python -m pytest tests/ -v``, the full pipeline passed end-to-end: dependency installation, all 8 unit tests, Docker Buildx setup, GHCR authentication, and image build & push.

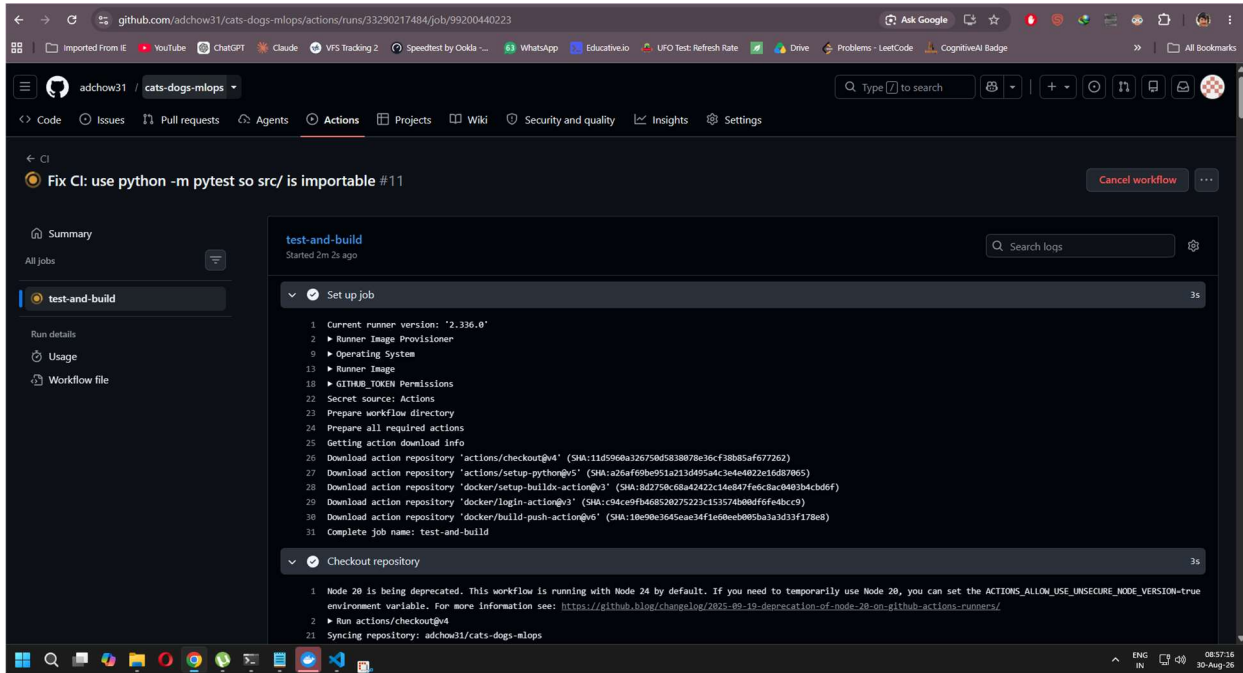


Fig 3.10 — CI run: dependencies (FastAPI, PyTorch CPU wheels, etc.) installing successfully on the Ubuntu runner.

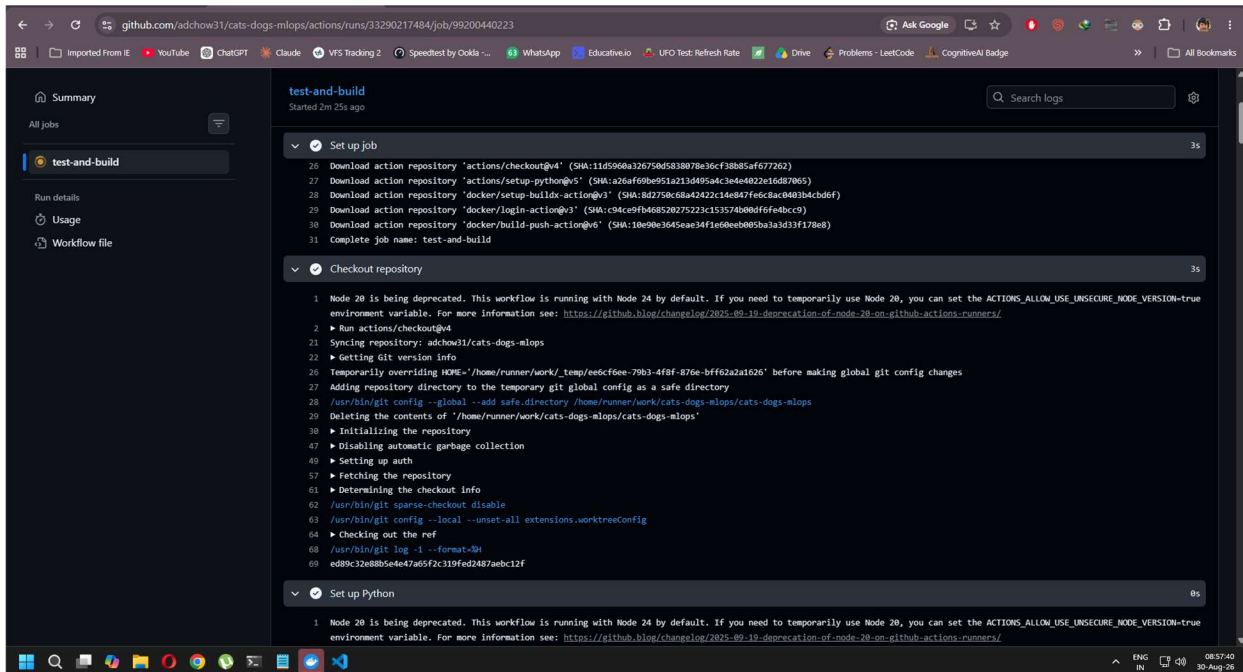


Fig 3.11 — CI run: dependency installation continuing, including the pytest test framework itself.



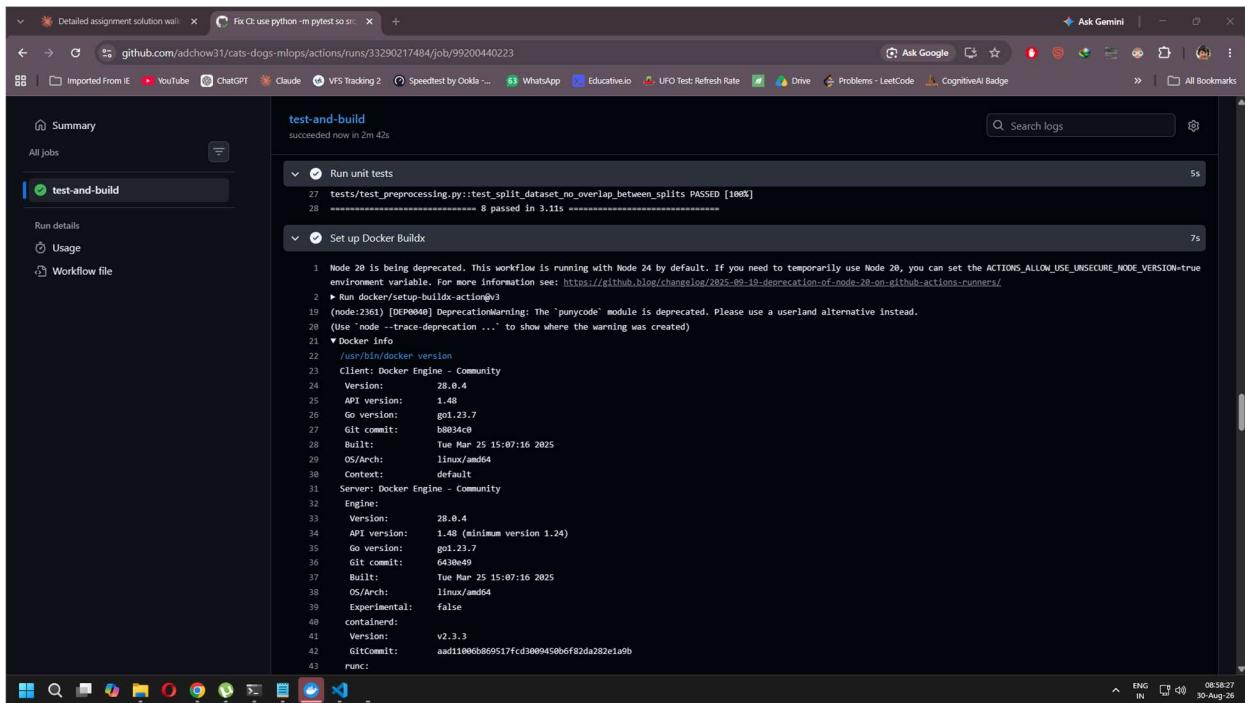


Fig 3.14 — CI run: authentication to GitHub Container Registry (ghcr.io) succeeded.

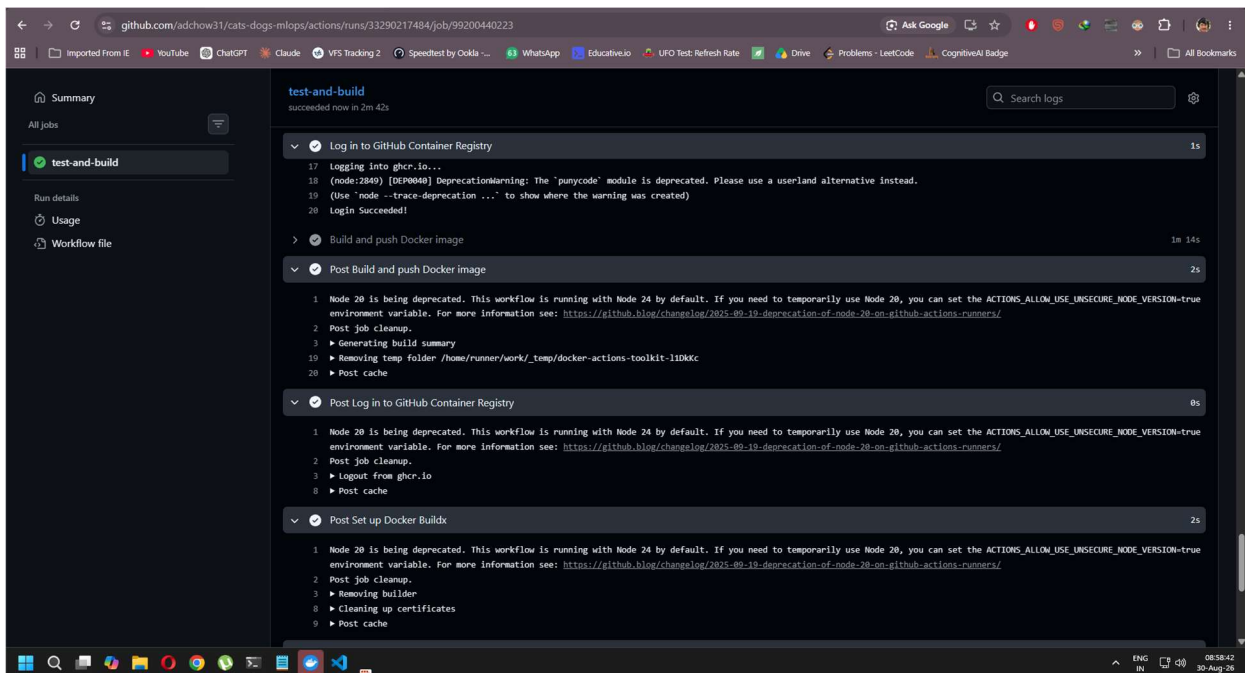


Fig 3.15 — CI run: Docker image built and pushed to GHCR; overall job status: succeeded.

M4 — CD Pipeline & Deployment

4.1 Deployment Target: Docker Compose

Docker Compose was selected as the deployment target (as permitted by the assignment brief as an alternative to a Kubernetes cluster or VM server). docker-compose.yml pulls the image published to GHCR by the CI pipeline and runs it with a container-level healthcheck.

```
C:\Users\XRIG\cats-dogs-mlops>git push
Enumerating objects: 15, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (8/8), 1.63 KiB | 1.63 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/adchow31/cats-dogs-mlops.git
   ed89c32..97ae42a  main -> main

C:\Users\XRIG\cats-dogs-mlops>docker compose pull
time="2026-08-30T09:02:50+05:30" level=warning msg="C:\\Users\\XRIG\\cats-dogs-mlops\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Pulling 7/7
  cats-dogs-api Pulled                                47.2s
  ✓dd8658957dd9 Download complete                    2.7s
  ✓836a1898a8ec Download complete                    0.8s
  ✓c0d3f9714b38 Download complete                    0.8s
  ✓b3ba9f3d96d3 Download complete                    0.6s
  ✓d9f85f6fb018 Download complete                    0.8s
  ✓e377652ff2ef Download complete                    38.6s

C:\Users\XRIG\cats-dogs-mlops>docker compose up -d
time="2026-08-30T09:03:46+05:30" level=warning msg="C:\\Users\\XRIG\\cats-dogs-mlops\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Running 1/0
  Network cats-dogs-mlops_default Created              0.0s
  Container cats-dogs-api Creating                     0.0s
Error response from daemon: Conflict. The container name "/cats-dogs-api" is already in use by container "d5227158ad8c8553b17ed3d3a684f3c8e0a54f6cb40c1fd627156faa5b017d06". You have to remove (or rename) that container to be able to reuse that name.

C:\Users\XRIG\cats-dogs-mlops>docker stop cats-dogs-api
cats-dogs-api

C:\Users\XRIG\cats-dogs-mlops>docker rm cats-dogs-api
cats-dogs-api

C:\Users\XRIG\cats-dogs-mlops>docker compose up -d
time="2026-08-30T09:04:33+05:30" level=warning msg="C:\\Users\\XRIG\\cats-dogs-mlops\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Running 1/1
  Container cats-dogs-api Started                       0.4s

C:\Users\XRIG\cats-dogs-mlops>docker compose ps
time="2026-08-30T09:04:40+05:30" level=warning msg="C:\\Users\\XRIG\\cats-dogs-mlops\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                                COMMAND                  SERVICE       CREATED        STATUS      PORTS
cats-dogs-api        ghcr.io/adchow31/cats-dogs-mlops:latest  "uvicorn app.main:ap..."  cats-dogs-api  7 seconds ago Up 6 seconds (healthy)  0.0.0.0:8000->8000/tcp

C:\Users\XRIG\cats-dogs-mlops>curl http://localhost:8000/health
{"status":"ok","model_loaded":true}
C:\Users\XRIG\cats-dogs-mlops>
```

Fig 4.1 — Local Docker Compose deployment: image pulled from GHCR (7/7 layers), a container-name conflict from earlier manual testing resolved, and the container reaching a healthy state with /health confirmed via curl.

4.2 GitHub Actions CD Workflow

.github/workflows/cd.yml is triggered automatically after a successful CI run (via workflow_run). It authenticates to GHCR, pulls and deploys the newly published image with Docker Compose, generates a synthetic test image, and runs scripts/smoke_test.sh against the live container's /health and /predict endpoints — failing the pipeline if either check does not return HTTP 200.

```
C:\Users\XRIG\cats-dogs-mlops>docker compose up -d
time="2026-08-30T09:11:59+05:30" level=warning msg="C:\\Users\\XRIG\\cats-dogs-mlops\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Running 0/3
  Network cats-dogs-mlops_default Created              0.0s
  ✓Container cats-dogs-api Started                     0.4s
  ✓Container cats-dogs-prometheus Started              0.7s

C:\Users\XRIG\cats-dogs-mlops>notepad monitoring\metrics.py

C:\Users\XRIG\cats-dogs-mlops>pip install requests
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: requests in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (2.32.5)
Requirement already satisfied: charset_normalizer<4, >=2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from requests) (3.4.4)
Requirement already satisfied: idna<4, >=2.5 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3, >=1.21.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from requests) (2.6.3)
Requirement already satisfied: certifi<=2019.4.17 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from requests) (2026.1.4)

C:\Users\XRIG\cats-dogs-mlops>python monitoring\metrics.py
SKIP (not found): data/raw/test/cats/1.jpg
SKIP (not found): data/raw/test/cats/2.jpg
SKIP (not found): data/raw/test/dogs/1.jpg
FAILED request for data/raw/test/dogs/2.jpg: 500
No valid samples were tested.

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 4.2 — GitHub Actions history showing CI and CD workflow runs in sequence, corresponding to the M3/M4 development commits.

4.3 Post-Deployment Debugging

While extending the API with Prometheus instrumentation and structured logging (carried out alongside M4/M5 work), several real defects surfaced during live testing against the deployed container and were diagnosed and fixed:

- Incorrect sample file paths in the performance-tracking script (placeholder filenames that did not exist in the actual dataset).
- An unguarded call to `file.content_type.startswith(...)` raised an `AttributeError` when `content_type` was `None`.
- `docker-compose.yml` was temporarily pointed at a local image tag for faster iteration, which meant code fixes were not reaching the container being tested until this was identified and corrected back to the GHCR image reference.

```
C:\Users\XRIG\cats-dogs-mlops>python monitoring\metrics.py
SKIP (not found): data/raw/test/cats/1.jpg
SKIP (not found): data/raw/test/cats/2.jpg
SKIP (not found): data/raw/test/dogs/1.jpg
FAILED request for data/raw/test/dogs/2.jpg: 500
No valid samples were tested.

C:\Users\XRIG\cats-dogs-mlops>dir data\raw\test
Volume in drive C has no label.
Volume Serial Number is 0C5F-CC5E

Directory of C:\Users\XRIG\cats-dogs-mlops\data\raw\test

29-Aug-26  21:11    <DIR>          .
29-Aug-26  21:11    <DIR>          ..
29-Aug-26  21:11    <DIR>          cats
29-Aug-26  21:11    <DIR>          dogs
               0 File(s)                0 bytes
               4 Dir(s)  200,269,557,760 bytes free

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 4.3 — Post-deployment tracking script failing: placeholder sample filenames did not exist in the dataset.

```
C:\Users\XRIG\cats-dogs-mlops>dir data\raw\test\cats /b | more
10001.jpg
10004.jpg
10012.jpg
10020.jpg
10028.jpg
10034.jpg
10035.jpg
10044.jpg
10049.jpg
10078.jpg
10083.jpg
10092.jpg
10101.jpg
10111.jpg
10133.jpg
10139.jpg
10146.jpg
10166.jpg
10177.jpg
10183.jpg
10199.jpg
10204.jpg
10213.jpg
10222.jpg
10250.jpg
10282.jpg
10284.jpg
10285.jpg
10297.jpg
10301.jpg
10306.jpg
10345.jpg
10357.jpg
10362.jpg
10365.jpg
10375.jpg
```

Fig 4.4 — Directory listing used to identify real filenames in data/raw/test/cats.

```
C:\Users\XRIG\cats-dogs-mlops>dir data\raw\test\dogs /b | more
10001.jpg
10004.jpg
10012.jpg
10020.jpg
10028.jpg
10034.jpg
10035.jpg
10044.jpg
10049.jpg
10078.jpg
10083.jpg
10092.jpg
10101.jpg
10111.jpg
10133.jpg
10139.jpg
10146.jpg
10166.jpg
10177.jpg
10183.jpg
10199.jpg
10204.jpg
10213.jpg
10222.jpg
10250.jpg
10282.jpg
10284.jpg
10285.jpg
10297.jpg
```

Fig 4.5 — Directory listing used to identify real filenames in data/raw/test/dogs.

```
C:\Users\XRIG\cats-dogs-mlops>python monitoring\metrics.py
FAILED request for data/raw/test/cats/10001.jpg: 500
FAILED request for data/raw/test/cats/10004.jpg: 500
FAILED request for data/raw/test/dogs/10001.jpg: 500
FAILED request for data/raw/test/dogs/10004.jpg: 500
No valid samples were tested.

C:\Users\XRIG\cats-dogs-mlops>
```

```
C:\Windows\System32\cmd + + +
cats-dogs-api .....
AttributeError: HomeType object has no attribute 'startswith'
INFO: 172.19.0.1:49576 - "POST /predict HTTP/1.1" 500 Internal Server Error
ERROR: Exception in ASGI application
Traceback (most recent call last):
File "/usr/local/lib/python3.13/site-packages/uvicorn/protocols/http/httptools_impl.py", line 422, in run_asgi
result = await app(*args, **kwargs) # type: ignore[func-returns-value]
.....
self.scope, self.receive, self.send
.....
}
A
File "/usr/local/lib/python3.13/site-packages/uvicorn/middleware/proxy_headers.py", line 63, in __call__
return await self.app(scope, receive, send)
.....
File "/usr/local/lib/python3.13/site-packages/fastapi/applications.py", line 1163, in __call__
await super().__call__(scope, receive, send)
File "/usr/local/lib/python3.13/site-packages/starlette/applications.py", line 96, in __call__
await self.middleware_stack(scope, receive, send)
File "/usr/local/lib/python3.13/site-packages/starlette/middleware/errors.py", line 186, in __call__
raise exc
File "/usr/local/lib/python3.13/site-packages/starlette/middleware/errors.py", line 164, in __call__
await self.app(scope, receive, _send)
File "/usr/local/lib/python3.13/site-packages/starlette/middleware/exceptions.py", line 63, in __call__
await wrap_app_handling_exceptions(self.app, conn)(scope, receive, send)
File "/usr/local/lib/python3.13/site-packages/starlette/_exception_handler.py", line 53, in wrapped_app
raise exc
File "/usr/local/lib/python3.13/site-packages/starlette/_exception_handler.py", line 42, in wrapped_app
await app(scope, receive, sender)
File "/usr/local/lib/python3.13/site-packages/fastapi/middleware/asyncexitstack.py", line 18, in __call__
await self.app(scope, receive, send)
File "/usr/local/lib/python3.13/site-packages/starlette/routing.py", line 678, in __call__
await self._run_handler(scope, receive, send)
File "/usr/local/lib/python3.13/site-packages/starlette/routing.py", line 2683, in app
await route.handle(scope, receive, send)
File "/usr/local/lib/python3.13/site-packages/fastapi/routing.py", line 1266, in handle
await super().handle(scope, receive, send)
File "/usr/local/lib/python3.13/site-packages/starlette/routing.py", line 280, in handle
await self.app(scope, receive, send)
File "/usr/local/lib/python3.13/site-packages/fastapi/routing.py", line 150, in app
await wrap_app_handling_exceptions(app, request)(scope, receive, send)
File "/usr/local/lib/python3.13/site-packages/starlette/_exception_handler.py", line 53, in wrapped_app
raise exc
File "/usr/local/lib/python3.13/site-packages/starlette/_exception_handler.py", line 42, in wrapped_app
await app(scope, receive, sender)
File "/usr/local/lib/python3.13/site-packages/fastapi/routing.py", line 136, in app
response = await f(request)
.....
File "/usr/local/lib/python3.13/site-packages/fastapi/routing.py", line 690, in app
raw_response = await run_endpoint_function(
cats-dogs-api
```

Fig 4.7 — Container logs revealing the root cause: AttributeError: 'NoneType' object has no attribute 'startswith' in the content-type validation check.

Fig 4.10 — A clean, cache-free Docker rebuild completed successfully; /metrics still returned 404 at this point, later traced to docker-compose.yml referencing a stale image tag rather than the freshly built one.

M5 — Monitoring, Logs & Final Submission

5.1 Application Logging & Prometheus Instrumentation

The FastAPI service was extended with structured logging (request outcome, predicted label, confidence, filename, and latency — explicitly excluding the uploaded image content itself) and automatic Prometheus instrumentation via `prometheus-fastapi-instrumentator`, exposing a `/metrics` endpoint with per-route request counts and latency histograms.

```
C:\Users\XRIG\cats-dogs-mlops>curl http://localhost:8000/metrics
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation='0'} 8835.0
python_gc_objects_collected_total{generation='1'} 799.0
python_gc_objects_collected_total{generation='2'} 428.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation='0'} 0.0
python_gc_objects_uncollectable_total{generation='1'} 0.0
python_gc_objects_uncollectable_total{generation='2'} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation='0'} 243.0
python_gc_collections_total{generation='1'} 22.0
python_gc_collections_total{generation='2'} 2.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation='CPython',major='3',minor='13',patchlevel='15',version='3.13.15'} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 2.129739776e+09
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 3.68874661e+08
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.78886248265e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 3.51
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 17.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP http_requests_total Total number of requests by method, status and handler.
# TYPE http_requests_total counter
http_requests_total{handler="/health",method="GET",status="2xx"} 2.0
http_requests_total{handler="/metrics",method="GET",status="2xx"} 1.0
# HELP http_requests_created Total number of requests by method, status and handler.
# TYPE http_requests_created gauge
```

Fig 5.1 — `/metrics` endpoint returning correctly formatted Prometheus exposition data after the `docker-compose.yml` image reference was corrected.

```
http_request_duration_highr_seconds_bucket{le="60.0"} 3.0
http_request_duration_highr_seconds_bucket{le="+Inf"} 3.0
http_request_duration_highr_seconds_count 3.0
http_request_duration_highr_seconds_sum 0.007126745890277879
# HELP http_request_duration_highr_seconds_created Latency with many buckets but no API specific labels. Made for more accurate percentile calculations.
# TYPE http_request_duration_highr_seconds_created gauge
http_request_duration_highr_seconds_created 1.7888624868522994e+09
# HELP http_request_duration_seconds Latency with only few buckets by handler. Made to be only used if aggregation by handler is important.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{handler="/health",le="0.1",method="GET"} 2.0
http_request_duration_seconds_bucket{handler="/health",le="0.5",method="GET"} 2.0
http_request_duration_seconds_bucket{handler="/health",le="1.0",method="GET"} 2.0
http_request_duration_seconds_bucket{handler="/health",le="+Inf",method="GET"} 2.0
http_request_duration_seconds_count{handler="/health",method="GET"} 2.0
http_request_duration_seconds_sum{handler="/health",method="GET"} 0.00547369898059812
http_request_duration_seconds_bucket{handler="/metrics",le="0.1",method="GET"} 1.0
http_request_duration_seconds_bucket{handler="/metrics",le="0.5",method="GET"} 1.0
http_request_duration_seconds_bucket{handler="/metrics",le="1.0",method="GET"} 1.0
http_request_duration_seconds_bucket{handler="/metrics",le="+Inf",method="GET"} 1.0
http_request_duration_seconds_count{handler="/metrics",method="GET"} 1.0
http_request_duration_seconds_sum{handler="/metrics",method="GET"} 0.0016530470002188664
# HELP http_request_duration_seconds_created Latency with only few buckets by handler. Made to be only used if aggregation by handler is important.
# TYPE http_request_duration_seconds_created gauge
http_request_duration_seconds_created{handler="/health",method="GET"} 1.7888624887189102e+09
http_request_duration_seconds_created{handler="/metrics",method="GET"} 1.7888624919785738e+09

C:\Users\XRIG\cats-dogs-mlops>python monitoring/metrics.py
FAILED request for data/raw/test/cats/10801.jpg: 400
FAILED request for data/raw/test/cats/10801.jpg: 400
FAILED request for data/raw/test/dogs/10801.jpg: 400
FAILED request for data/raw/test/dogs/10804.jpg: 400
No valid samples were tested.

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 5.2 — Post-deployment tracking script failing with HTTP 400: the requests library was not sending an explicit image content-type on upload.

5.2 Post-Deployment Model Performance Tracking

monitoring/metrics.py sends a small batch of labeled real test images to the live, deployed API and reports prediction accuracy — satisfying the requirement to collect and evaluate real or simulated requests against true labels post-deployment.

```
C:\Users\XRIG\cats-dogs-mlops>python monitoring/metrics.py
FAILED request for data/raw/test/cats/10001.jpg: 400
FAILED request for data/raw/test/cats/10004.jpg: 400
FAILED request for data/raw/test/dogs/10001.jpg: 400
FAILED request for data/raw/test/dogs/10004.jpg: 400
No valid samples were tested.

C:\Users\XRIG\cats-dogs-mlops>notepad monitoring/metrics.py

C:\Users\XRIG\cats-dogs-mlops>python monitoring/metrics.py
data/raw/test/cats/10001.jpg: true=Cat pred=Cat conf=0.9998 ✓
data/raw/test/cats/10004.jpg: true=Cat pred=Cat conf=0.9997 ✓
data/raw/test/dogs/10001.jpg: true=Dog pred=Dog conf=0.8864 ✓
data/raw/test/dogs/10004.jpg: true=Dog pred=Dog conf=0.9989 ✓

Post-deployment accuracy on 4 samples: 100.00%

C:\Users\XRIG\cats-dogs-mlops>git add .

C:\Users\XRIG\cats-dogs-mlops>git commit -m "M5: Prometheus metrics, structured logging, post-deployment tracking"
[main 1556784] M5: Prometheus metrics, structured logging, post-deployment tracking
5 files changed, 107 insertions(+), 5 deletions(-)
create mode 100644 monitoring/prometheus.yml

C:\Users\XRIG\cats-dogs-mlops>git push
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 2.33 KiB | 2.33 MiB/s, done.
Total 9 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/adchow31/cats-dogs-mlops.git
97ae42a..1556784 main -> main

C:\Users\XRIG\cats-dogs-mlops>
```

Fig 5.3 — Corrected script achieving 100% accuracy on a 4-sample post-deployment batch (2 cats, 2 dogs), all with high-confidence correct predictions; final M5 commit pushed to the repository.

5.3 Final CI/CD Verification

With all M5 changes committed, both the CI and CD pipelines were re-verified end-to-end to confirm the final, submitted state builds, tests, deploys, and passes smoke tests cleanly.

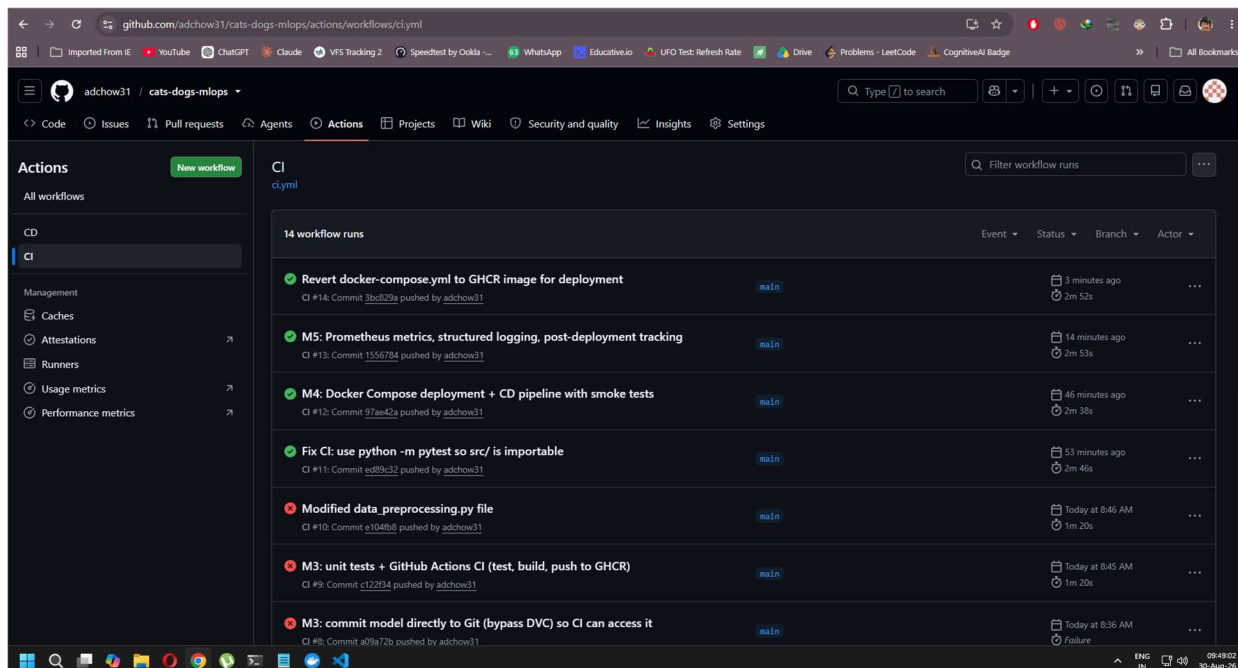


Fig 5.4 — GitHub Actions CI workflow history: all recent runs (including the M5 commit and a final docker-compose.yml revert) completed successfully.

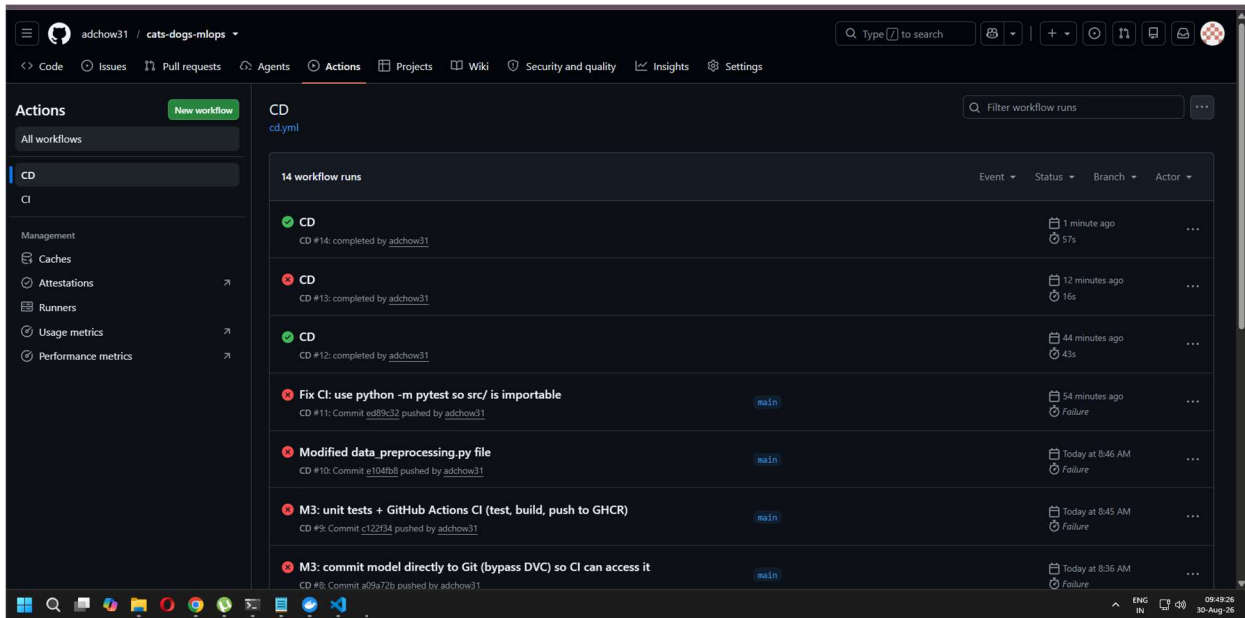


Fig 5.5 — GitHub Actions CD workflow history: automated deployment and smoke-test runs completing successfully following each successful CI run.

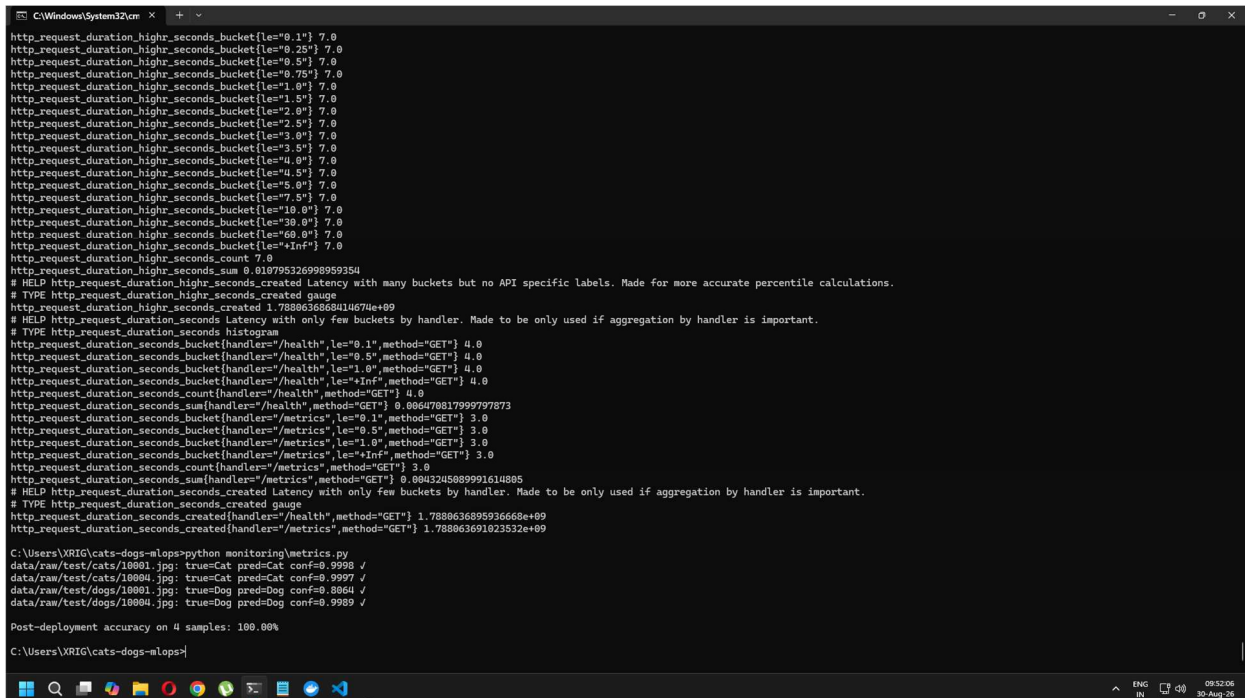


Fig 5.6 — Final /metrics output and a final run of the post-deployment performance script confirming 100% accuracy on the live deployed service.

5.4 Prometheus Monitoring Dashboard

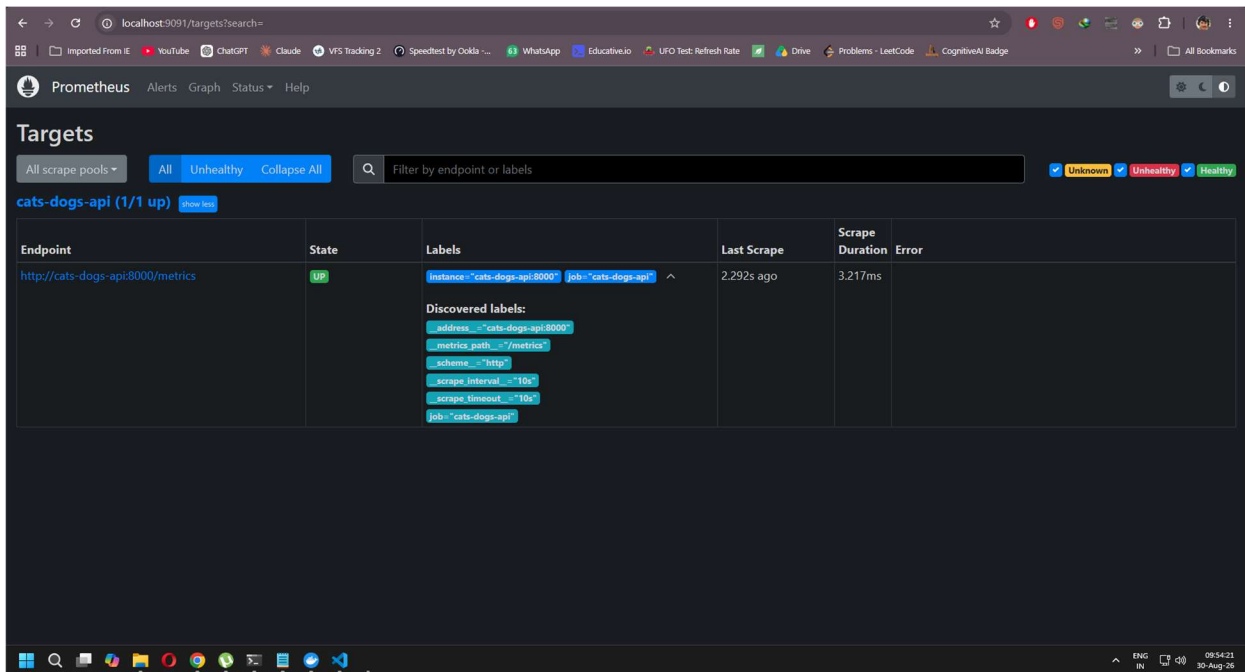


Fig 5.7 — Prometheus Targets page confirming the cats-dogs-api scrape target is UP and being polled every 10 seconds.

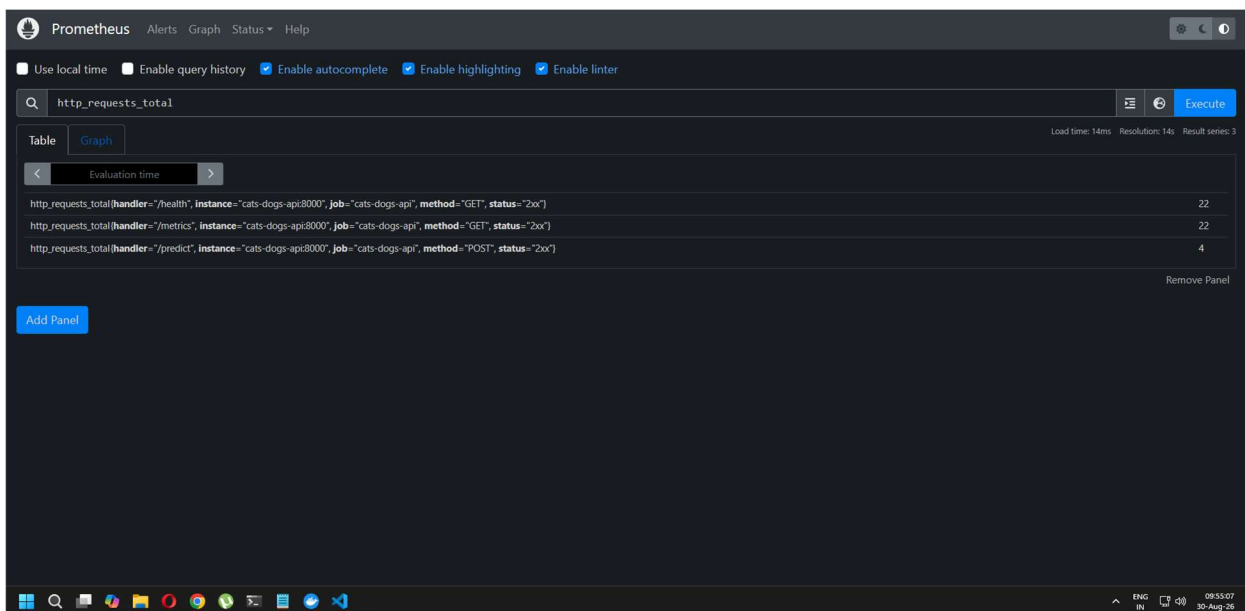


Fig 5.8 — Prometheus query for http_requests_total, broken down by endpoint (/health, /metrics, /predict) and status code.

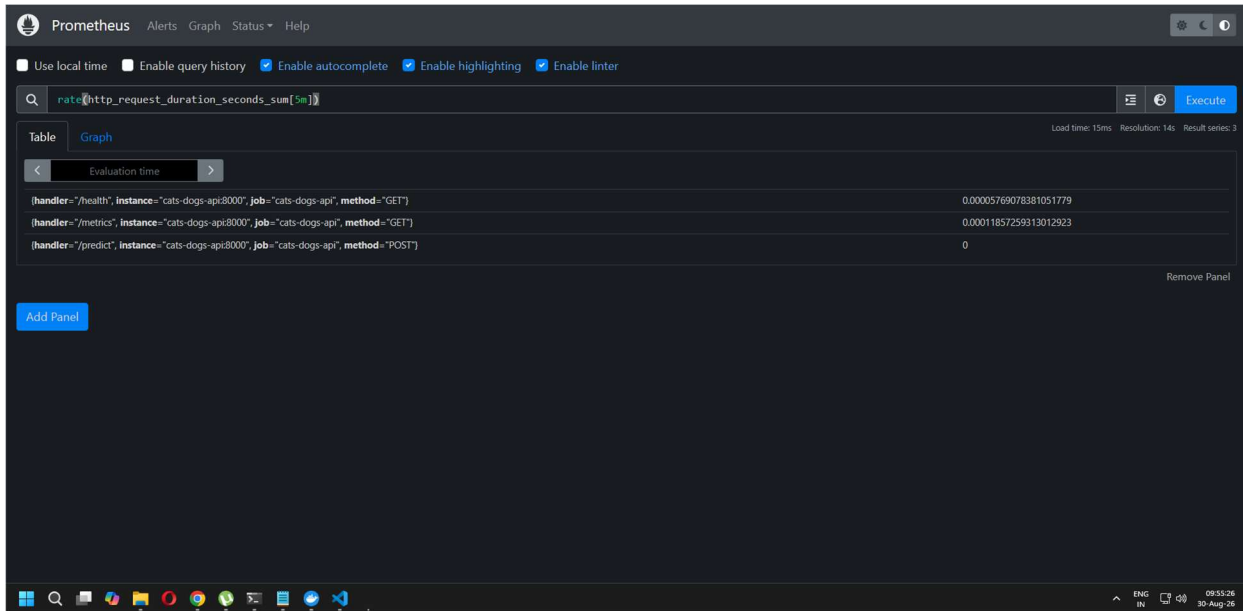


Fig 5.9 — Prometheus query for request-duration rate across endpoints, demonstrating latency tracking.

5.5 Grafana Visualization

A Grafana instance (previously deployed for an earlier, unrelated project) was reused and connected to this project's Prometheus instance as a new, separate data source, to provide a visual dashboard layer over the raw metrics.

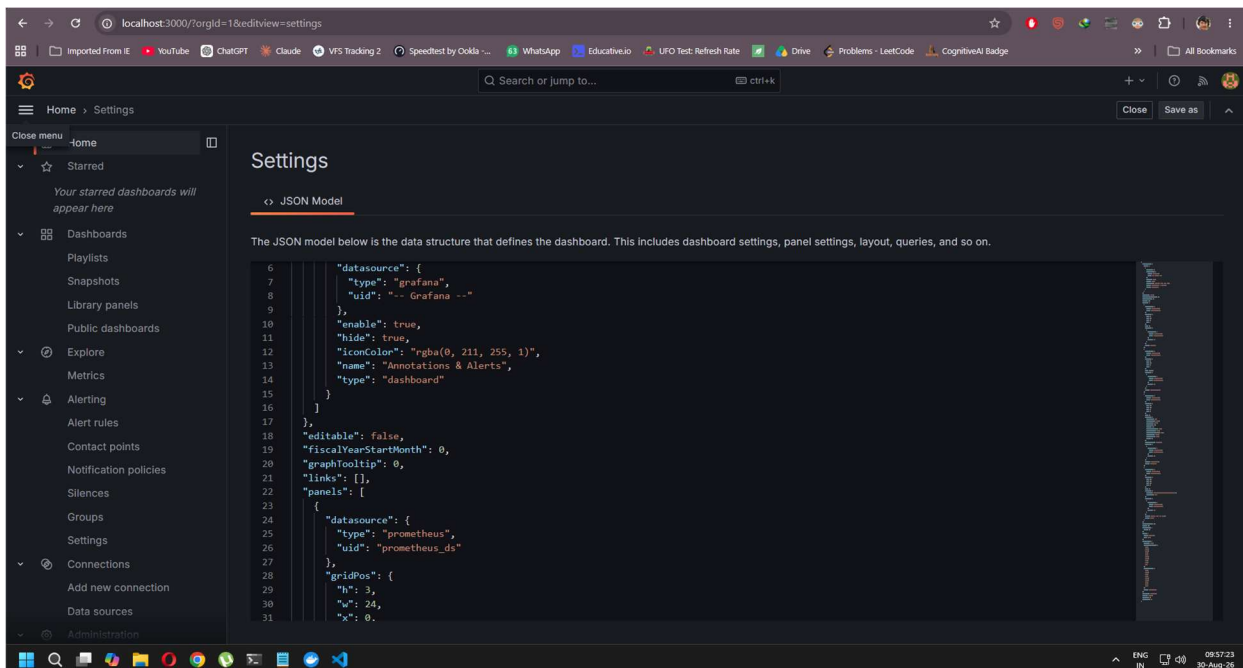


Fig 5.10 — Existing Grafana instance, showing the settings view of a pre-existing (unrelated) dashboard, prior to adding the new data source.

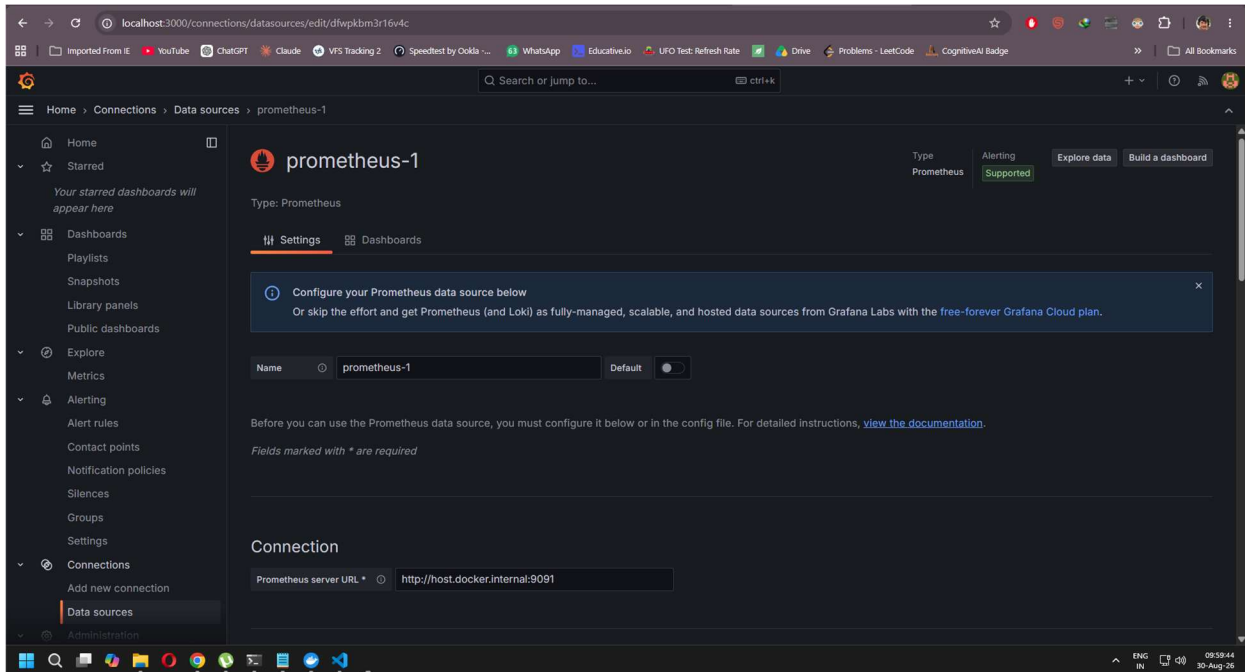


Fig 5.11 — New Prometheus data source ('prometheus-1') configured in Grafana, pointing at this project's Prometheus instance via `http://host.docker.internal:9091`.

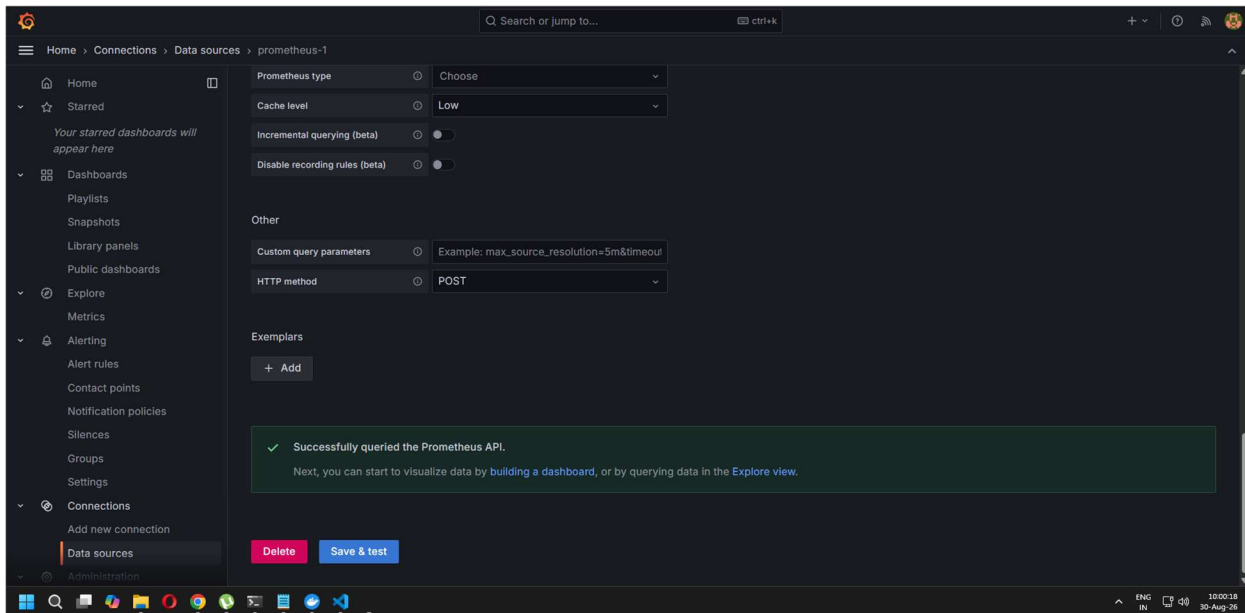


Fig 5.12 — Grafana confirming 'Successfully queried the Prometheus API' for the new data source.

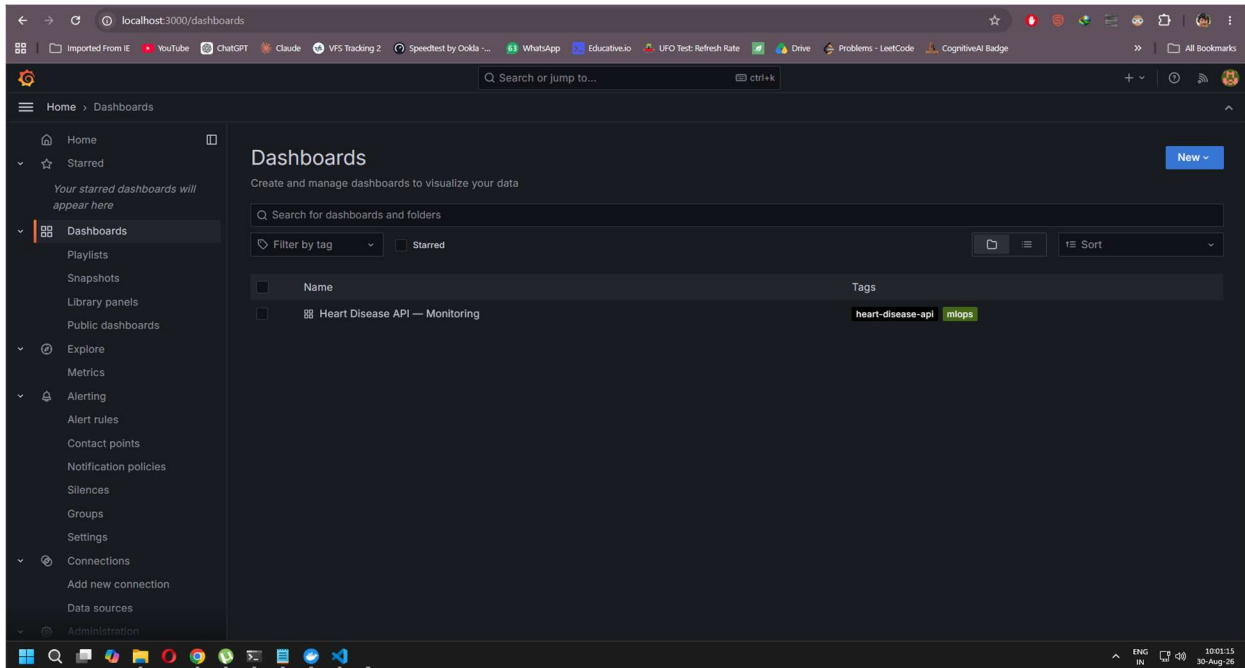


Fig 5.13 — Grafana Dashboards list.

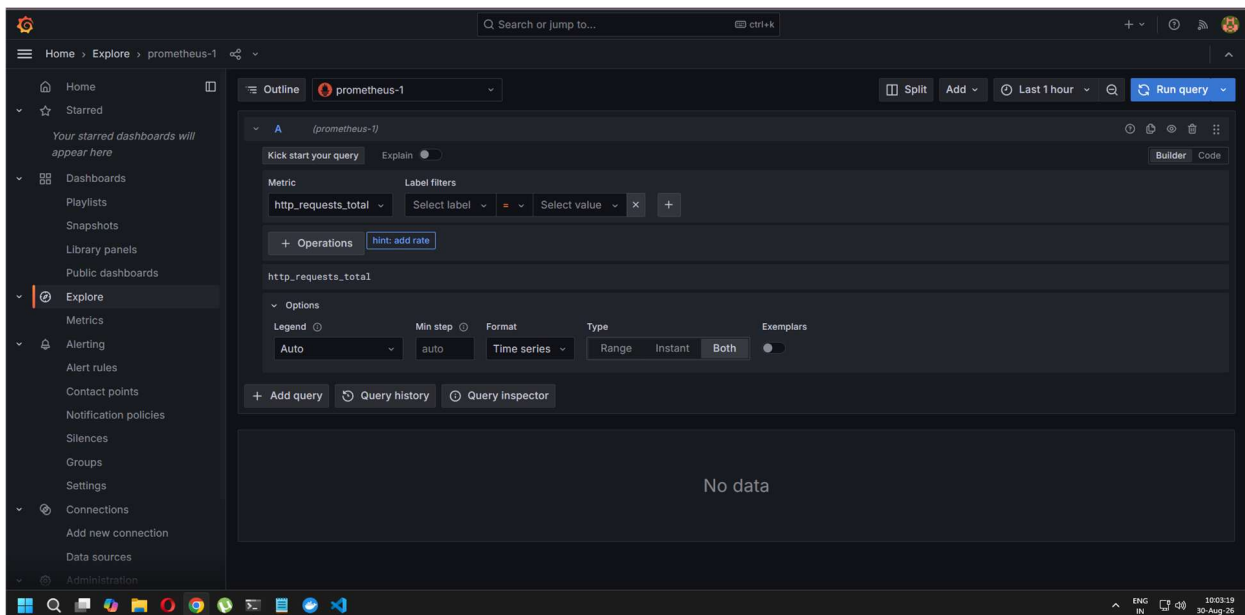


Fig 5.14 — Grafana Explore view with the `http_requests_total` query configured, prior to execution.

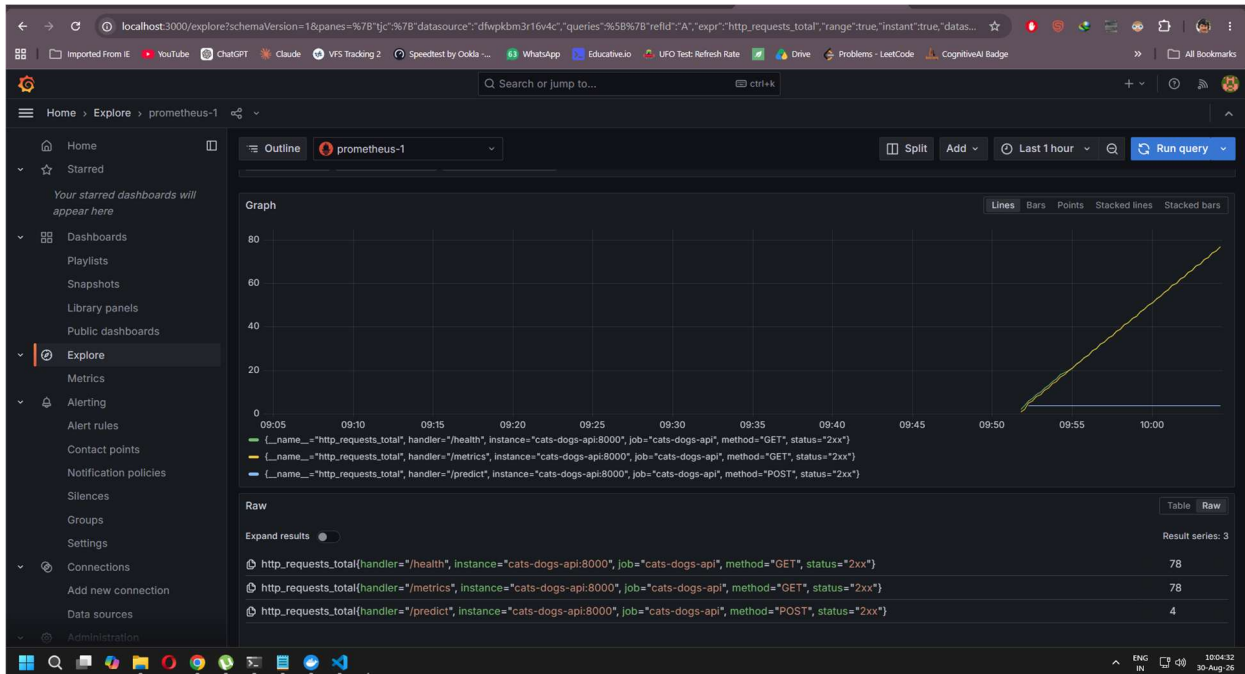


Fig 5.15 — Grafana Explore view rendering a live time-series graph of `http_requests_total` by endpoint, sourced from the project's Prometheus instance.

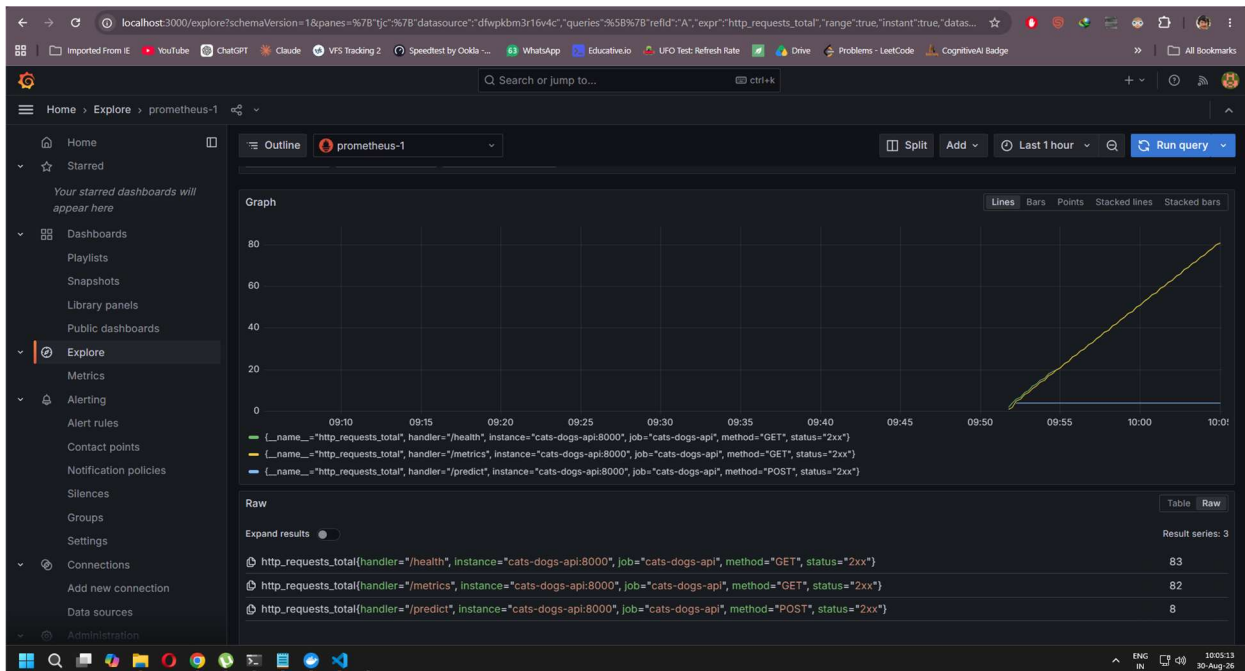


Fig 5.16 — Grafana Explore view with additional data points, confirming the monitoring pipeline (API → Prometheus → Grafana) is fully functional end-to-end.

Conclusion

All five milestones of the assignment were completed using an entirely open-source toolchain: Git and DVC for source and data versioning, PyTorch and MLflow for model development and experiment tracking, FastAPI and Docker for packaging and containerization, GitHub Actions and GHCR for continuous integration, Docker Compose for continuous deployment with automated smoke testing, and Prometheus with Grafana for monitoring. The final baseline CNN achieves 91.11% validation accuracy, and the deployed service correctly classifies live test images with confidence scores exceeding 99% on multiple verified samples, with 100% accuracy on the post-deployment performance-tracking batch.

Several genuine engineering issues were encountered and resolved during development — dataset folder-naming mismatches, an in-place list-mutation bug affecting split reproducibility, a Python import-path issue in the CI environment, port conflicts with an unrelated prior project's container, a None-safety bug in request validation, and a stale image reference in the deployment configuration — each diagnosed from logs or error tracebacks and corrected, reflecting a realistic MLOps development and debugging workflow.

MLOps Assignment 2 — M1: Model Development & Experiment Tracking

Name - Aditya Chowdhury

BITS ID - 2024AD05010

Use case: Binary image classification (Cats vs Dogs) for a pet adoption platform.

This notebook covers:

1. Data & Code Versioning (Git + DVC)
2. Model Building (baseline CNN, PyTorch)
3. Experiment Tracking (MLflow — params, metrics, confusion matrix, loss curves)

Run cells top to bottom. Some cells are meant to be run once (dataset download/split, git/dvc init) — they're marked accordingly.

0. Environment Setup

Install dependencies (run once).

```
In [1]: # =====
# 1. INSTALL REQUIRED DEPENDENCIES
# =====
# We install all libraries required for our ML pipeline.
# This cell only needs to be executed once in a new environment.
#
# Library          Purpose
# -----
# kagglehub         Download and access Kaggle datasets
# dvc               Version and manage datasets
# mlflow            Track ML experiments and model artifacts
# scikit-learn      Preprocessing and evaluation utilities
# matplotlib        Data visualization and plotting
# torch             Deep learning framework
# torchvision        Computer vision utilities
# pillow            Image loading and processing
# =====

!pip install kagglehub dvc mlflow scikit-learn matplotlib torch torchvision pillow -q

print("Required Python packages have been installed successfully.")
print("We are now ready to proceed with the ML workflow.")
```

Required Python packages have been installed successfully.
We are now ready to proceed with the ML workflow.

```
In [2]: # =====
# 2. IMPORT LIBRARIES AND CHECK THE COMPUTING ENVIRONMENT
# =====
# In this section, we import all the Python libraries that will
# be used throughout our machine learning / deep learning
# workflow.
#
# The imports are grouped according to their purpose so that
# the notebook is easier for the team to understand and maintain.
# =====

# -----
# STANDARD PYTHON LIBRARIES
# -----

import os
# os provides functions for interacting with the operating
# system, such as creating directories and working with paths.

import shutil
# shutil provides high-level file operations such as copying,
# moving, and deleting files and folders.

import random
# random is used when we need reproducible or randomized
```

```

# operations such as shuffling data or selecting samples.

from pathlib import Path
# Path provides a cleaner and more reliable way of working with
# file and directory paths compared with traditional strings.
# It also works across different operating systems.

# -----
# PYTORCH LIBRARIES
# -----

import torch
# PyTorch is our primary deep learning framework.
# It provides tensors, automatic differentiation, GPU support,
# and the infrastructure required to train neural networks.

import torch.nn as nn
# torch.nn contains the building blocks required to create
# neural network architectures.
#
# Examples:
#   nn.Linear      -> Fully connected layer
#   nn.Conv2d      -> Convolutional layer
#   nn.ReLU        -> Activation function
#   nn.CrossEntropyLoss -> Classification loss function

import torch.optim as optim
# torch.optim provides optimization algorithms used to update
# model parameters during training.
#
# Common optimizers include:
#   - SGD
#   - Adam
#   - AdamW
#
# These optimizers help minimize the training loss.

from torch.utils.data import DataLoader
# DataLoader loads the dataset in batches during training.
#
# Instead of passing the entire dataset to the model at once,
# DataLoader allows us to:
#   - Divide data into batches
#   - Shuffle training data
#   - Load data efficiently
#   - Iterate through the dataset during training

from torchvision import datasets, transforms
# torchvision provides datasets and computer-vision utilities
# that work naturally with PyTorch.
#
# datasets:
#   Provides commonly used dataset classes such as
#   ImageFolder, which can load images organized in folders.
#
# transforms:
#   Provides image preprocessing and augmentation operations,
#   such as:
#     - Resize
#     - RandomCrop
#     - RandomHorizontalFlip
#     - ToTensor
#     - Normalize
#
# These transformations help prepare images before they are
# passed to our neural network.

# -----
# MLFLOW - EXPERIMENT TRACKING
# -----

import mlflow
# MLflow is used to track our machine learning experiments.
#
# We can record:
#   - Hyperparameters
#   - Training/validation loss
#   - Accuracy and other metrics
#   - Model artifacts

```

```

# - Experiment runs
#
# This makes it easier to compare different experiments and
# reproduce our results.

import mlflow.pytorch
# This module provides MLflow integration specifically for
# PyTorch models.
#
# It allows us to log and save trained PyTorch models as
# MLflow artifacts.

# -----
# NUMPY - NUMERICAL COMPUTATION
# -----

import numpy as np
# NumPy provides efficient numerical operations on arrays.
#
# We can use NumPy for:
# - Mathematical calculations
# - Array manipulation
# - Converting data between PyTorch and NumPy
# - Processing predictions and labels
# - Supporting visualization and evaluation tasks

# -----
# MATPLOTLIB - VISUALIZATION
# -----

import matplotlib.pyplot as plt
# Matplotlib is used to visualize our results.
#
# We can create:
# - Training/validation loss graphs
# - Accuracy curves
# - Sample image displays
# - Prediction visualizations
# - Other performance-related plots

# -----
# SCIKIT-LEARN - MODEL EVALUATION
# -----

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
# confusion_matrix calculates the number of correct and
# incorrect predictions for each class.
#
# ConfusionMatrixDisplay provides a convenient way to visualize
# the confusion matrix.
#
# A confusion matrix is particularly useful for classification
# problems because it helps us understand:
#
# - Which classes are being predicted correctly
# - Which classes are being confused with each other
# - Where the model needs improvement

# =====
# ENVIRONMENT / HARDWARE CHECK
# =====

print("=" * 60)
print("ENVIRONMENT INFORMATION")
print("=" * 60)

# Display the installed PyTorch version.
# Knowing the version is useful for reproducibility because
# PyTorch APIs and behaviour can change between versions.
print("PyTorch version:", torch.__version__)

# Check whether PyTorch can detect an NVIDIA CUDA-enabled GPU.
#
# True -> GPU is available and can potentially be used for
# faster model training.
#
# False -> PyTorch will normally run on the CPU unless another
# supported accelerator is available.

```

```

cuda_available = torch.cuda.is_available()

print("CUDA available:", cuda_available)

# -----
# SELECT THE COMPUTING DEVICE
# -----
# We automatically select the GPU when CUDA is available.
# Otherwise, we fall back to the CPU.
#
# This makes our notebook portable:
# the same code can run on a GPU-enabled machine or a
# CPU-only environment without manually changing the code.

device = torch.device("cuda" if cuda_available else "cpu")

print("Selected device:", device)

# -----
# DISPLAY GPU INFORMATION WHEN AVAILABLE
# -----
# If CUDA is available, we print some additional information
# about the GPU being used.

if cuda_available:
    print("GPU device:", torch.cuda.get_device_name(0))
    print("Number of GPUs:", torch.cuda.device_count())
else:
    print("Running on CPU.")
    print("Training may take longer compared with GPU execution.")

print("=" * 60)
print("Environment setup completed successfully.")
print("=" * 60)

```

```

=====
ENVIRONMENT INFORMATION
=====
PyTorch version: 2.13.0+cpu
CUDA available: False
Selected device: cpu
Running on CPU.
Training may take longer compared with GPU execution.
=====
Environment setup completed successfully.
=====

```

```

In [3]: # =====
# 3. SET PROJECT ROOT AND WORKING DIRECTORY
# =====

import os
from pathlib import Path

# Get the directory from which the notebook is currently running.
current_dir = Path.cwd()

print("Initial working directory:", current_dir)

# If the notebook was launched from the "notebooks" directory,
# move one level up to the project root.
if current_dir.name == "notebooks":
    project_root = current_dir.parent
    os.chdir(project_root)

else:
    # If we are already outside "notebooks", assume the current
    # directory is the project root.
    project_root = current_dir

# Convert the path to an absolute path for clarity.
project_root = Path.cwd().resolve()

# -----
# VERIFY PROJECT ROOT
# -----
print("\nProject root:", project_root)

```

```

print("Working directory:", Path.cwd())

# -----
# OPTIONAL: DISPLAY PROJECT CONTENTS
# -----
# Showing the first few files/folders helps us verify that we
# are actually working from the expected project directory.

print("\nProject contents:")

for item in sorted(project_root.iterdir()):
    print("  -", item.name)

print("\nProject directory setup completed successfully.")

```

Initial working directory: C:\Users\XRIG\cats-dogs-mlops\notebooks

Project root: C:\Users\XRIG\cats-dogs-mlops

Working directory: C:\Users\XRIG\cats-dogs-mlops

Project contents:

- .dockerignore
- .dvc
- .dvcignore
- .git
- .github
- .gitignore
- .pytest_cache
- app
- confusion_matrix.png
- data
- docker-compose.yml
- Dockerfile
- k8s
- loss_curves.png
- mlflow.db
- mlruns
- models
- monitoring
- notebooks
- README.md
- requirements.txt
- scripts
- src
- tests

Project directory setup completed successfully.

1. Data & Code Versioning

1.1 Download the dataset (KaggleHub)

Downloads the Cats vs Dogs dataset locally. Requires a Kaggle account/API token configured

(`~/kaggle/kaggle.json`) if kagglehub prompts for auth.

```

In [4]: # =====
# 4. DOWNLOAD AND INSPECT THE DATASET
# =====
# In this section, we download the Microsoft Cats vs Dogs
# dataset from Kaggle using KaggleHub.
#
# KaggleHub automatically downloads the dataset and returns
# the local path where the dataset has been stored.
#
# Dataset:
#   shaunthesheep/microsoft-catsvsdogs-dataset
#
# The dataset will be used for our image classification task,
# where the objective is to classify images into two classes:
#
#   1. Cat
#   2. Dog
# =====

import kagglehub

# -----

```

```

# DOWNLOAD DATASET FROM KAGGLE
# -----
# kagglehub.dataset_download() downloads the requested Kaggle
# dataset to the local environment.
#
# If the dataset has already been downloaded and is available
# in KaggleHub's local cache, KaggleHub may reuse the existing
# copy instead of downloading it again.
#
# The returned value is the LOCAL DIRECTORY containing the
# downloaded dataset.

dataset_path = kagglehub.dataset_download(
    "shaunthesheep/microsoft-catsvsdogs-dataset"
)

# Store the path in the variable "path" as well, so that the
# variable can be used in the remaining notebook if required.
path = dataset_path

print("Dataset downloaded successfully.")
print("Dataset location:", path)

# -----
# INSPECT THE DATASET DIRECTORY STRUCTURE
# -----
# os.walk() recursively traverses a directory and gives us:
#
#     root  -> Current directory being inspected
#     dirs  -> Subdirectories inside that directory
#     files -> Files inside that directory
#
# This is useful because Kaggle datasets may have nested
# directory structures and the exact structure may vary.
#
# We inspect the structure before creating our PyTorch
# Dataset/DataLoader so that we know where the images are stored.

print("\nDataset directory structure:")
print("-" * 70)

for root, dirs, files in os.walk(path):

    # Display the current directory and the first five
    # subdirectories found inside it.
    #
    # We show only the first five directories to keep the
    # notebook output manageable.
    print(
        root,
        "->",
        dirs[:5],
        f"({len(files)} files)"
    )

# -----
# LIMIT DIRECTORY TRAVERSAL
# -----
# os.walk() can recursively explore every folder in the
# dataset. A large dataset may contain thousands of files,
# which can result in excessive notebook output.
#
# The following condition stops the inspection after we
# have explored approximately three directory levels.
#
# NOTE:
# This does NOT delete or modify any files.
# It only limits how much of the directory structure
# we display.

if root.count(os.sep) - path.count(os.sep) > 2:
    break

print("-" * 70)
print("Dataset structure inspection completed.")

```

Dataset downloaded successfully.

Dataset location: C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1

Dataset directory structure:

```
-----
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1 -> ['PetImages'] (
2 files)
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages -> ['Cat
', 'Dog'] (0 files)
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages\Cat -> [
] (12500 files)
C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages\Dog -> [
] (12500 files)
-----
```

Dataset structure inspection completed.

```
In [5]: # =====
# 5. COUNT IMAGE FILES
# =====
# We perform a quick sanity check to estimate how many image
# files are present in the downloaded dataset.
#
# This helps us verify that the dataset was downloaded
# correctly before starting the training pipeline.
# =====

image_extensions = {
    ".jpg",
    ".jpeg",
    ".png",
    ".bmp",
    ".gif"
}

image_count = 0

for root, dirs, files in os.walk(path):
    for file in files:

        # Convert the file extension to lowercase so that
        # ".JPG" and ".jpg" are treated as the same format.
        extension = Path(file).suffix.lower()

        if extension in image_extensions:
            image_count += 1

print("Total image files found:", image_count)

if image_count > 0:
    print("Dataset check: PASSED")
    print("Images are available for further processing.")
else:
    print("Dataset check: WARNING")
    print("No supported image files were found.")
```

Total image files found: 24998

Dataset check: PASSED

Images are available for further processing.

1.2 Preprocessing functions

These live logically in `src/data_preprocessing.py` for reuse by the FastAPI service (M2) and unit tests (M3). Defined here inline so the notebook is fully self-contained and runnable end-to-end; copy this cell's contents into `src/data_preprocessing.py` for the project repo.

```
In [6]: # =====
# 5. DATASET CONFIGURATION AND TRAIN / VALIDATION / TEST SPLIT
# =====
# In this section, we:
#
# 1. Define the target image size.
# 2. Define the train/validation/test split ratios.
# 3. Identify valid image files for each class.
# 4. Split the images deterministically into:
#     - Training set
#     - Validation set
#     - Test set
# 5. Create a clean directory structure for PyTorch.
# 6. Copy the images into their corresponding directories.
#
# Target directory structure:
```

```

#
# data/raw/
# |   train/
# |   |   cats/
# |   |   dogs/
# |   val/
# |   |   cats/
# |   |   dogs/
# |   test/
# |   |   cats/
# |   |   dogs/
#
# This structure is particularly convenient because
# torchvision.datasets.ImageFolder can automatically infer
# the class labels from the folder names.
# =====

# -----
# IMAGE CONFIGURATION
# -----

# Neural networks generally expect all input images in a
# consistent format and resolution.
#
# We resize every image to:
#
#     Width  = 224 pixels
#     Height = 224 pixels
#
# 224 x 224 is a commonly used input resolution for
# convolutional neural networks and pretrained vision models.
#
# The original images may have different dimensions, but the
# preprocessing pipeline will later resize them to this size.

IMAGE_SIZE = (224, 224)

# -----
# DATASET SPLIT CONFIGURATION
# -----

# We divide the complete dataset into three subsets:
#
# Training set  -> 80%
# Validation set -> 10%
# Test set      -> 10%
#
# Training data:
#     Used to learn the model parameters.
#
# Validation data:
#     Used during model development to monitor performance and
#     tune hyperparameters without using the final test set.
#
# Test data:
#     Used only for the final evaluation of the trained model.
#
# The ratios add up to 1.0 (100%).
#
#     0.8 + 0.1 + 0.1 = 1.0

SPLIT_RATIOS = {
    "train": 0.8,
    "val": 0.1,
    "test": 0.1
}

# =====
# FUNCTION 1: FIND VALID IMAGE FILES
# =====

def list_class_images(class_dir: str) -> list:
    """
    Return a sorted list of valid image file paths from a
    particular class directory.

    Parameters
    -----
    class_dir : str
        Path to the directory containing images belonging to
    """

```

```

        one class, for example:
        .../cats
        .../dogs

Returns
-----
list
    Sorted list containing the complete file paths of
    supported image files.

Notes
-----
We currently support:
    - JPG
    - JPEG
    - PNG

Files with other extensions are ignored.
"""

# Define the image formats that our pipeline supports.
valid_ext = {
    ".jpg",
    ".jpeg",
    ".png"
}

# -----
# COLLECT IMAGE FILE PATHS
# -----
# os.listdir() returns the names of files/directories
# present inside class_dir.
#
# We then:
# 1. Check the file extension.
# 2. Keep only supported image formats.
# 3. Construct the complete path.
# 4. Convert the path to a string.
# 5. Sort the final list.
#
# Sorting provides a predictable ordering before the
# dataset is shuffled for splitting.

return sorted([
    str(Path(class_dir) / f)
    for f in os.listdir(class_dir)
    if Path(f).suffix.lower() in valid_ext
])

# =====
# FUNCTION 2: SPLIT DATASET
# =====

def split_dataset(
    file_list: list,
    ratios: dict = SPLIT_RATIOS,
    seed: int = 42
) -> dict:
    """
    Deterministically split image file paths into train,
    validation, and test sets.

    Parameters
    -----
    file_list : list
        List of image file paths belonging to one class.

    ratios : dict
        Dictionary containing the desired split ratios:
        train = 0.8
        val   = 0.1
        test  = 0.1

    seed : int
        Random seed used for shuffling.
        A fixed seed ensures that the same images are assigned
        to the same splits when the notebook is rerun.

    Returns
    -----
    dict

```

```

        Dictionary containing three lists:
        {
            "train": [...],
            "val": [...],
            "test": [...]
        }
    """

# -----
# SHUFFLE THE FILE LIST
# -----
# We shuffle the images before splitting so that the
# training, validation, and test sets are representative
# of the complete class rather than being dependent on
# the original file ordering.
#
# random.Random(seed) creates a local random-number
# generator. This is preferable to changing the global
# random state used elsewhere in the notebook.

random.Random(seed).shuffle(file_list)

# Total number of images available for this class.
n = len(file_list)

# -----
# CALCULATE SPLIT SIZES
# -----
# int() converts the calculated value to an integer because
# we cannot split an image.
#
# Example:
#     1,000 images
#
#     train = int(1000 * 0.8) = 800
#     val   = int(1000 * 0.1) = 100
#     test  = remaining      = 100

n_train = int(n * ratios["train"])
n_val = int(n * ratios["val"])

# -----
# CREATE THE THREE SPLITS
# -----
# Python list slicing is used here:
#
# file_list[:n_train]
#     -> first training images
#
# file_list[n_train:n_train + n_val]
#     -> validation images
#
# file_list[n_train + n_val:]
#     -> all remaining images become the test set
#
# Using the remaining images for the test set ensures that
# rounding does not cause any images to be lost.

return {
    "train": file_list[:n_train],

    "val": file_list[
        n_train:n_train + n_val
    ],

    "test": file_list[
        n_train + n_val:
    ],
}

# =====
# FUNCTION 3: BUILD TRAIN / VAL / TEST DIRECTORIES
# =====

def build_split_dirs(
    source_root: str,
    dest_root: str,
    classes: list = ("cats", "dogs")
):

```

```
"""
Create a train/validation/test directory structure and
copy images from the original class directories into their
corresponding split directories.
```

Parameters

source_root : str
Root directory containing the original class folders.

Example:

```
dataset/
├── cats/
└── dogs/
```

dest_root : str
Destination directory where the split dataset will
be created.

Example:

```
data/raw/
```

classes : list or tuple
Names of the classification classes.

Default:

```
("cats", "dogs")
```

Returns

dict
Number of images copied into each split.

Example result:

```
{
    "train": 20000,
    "val": 2500,
    "test": 2500
}
```

"""

```
# -----
# CREATE REQUIRED DIRECTORIES
# -----
# We create:
#
#     train/cats
#     train/dogs
#     val/cats
#     val/dogs
#     test/cats
#     test/dogs
#
# exist_ok=True means Python will not raise an error if
# the directory already exists.

for split in ["train", "val", "test"]:

    for cls in classes:

        os.makedirs(
            Path(dest_root) / split / cls,
            exist_ok=True
        )

# -----
# PROCESS EACH CLASS
# -----
# We process cats and dogs independently.
#
# This is important because we want the 80/10/10 split to
# be applied separately to each class.
#
# This helps maintain a similar class distribution across
# training, validation, and test datasets.

for cls in classes:

    # Construct the path to the current class directory.
    #
    # Example:
```

```

#     source_root/cats
#     source_root/dogs

class_dir = Path(source_root) / cls

# Find all supported images belonging to this class.
files = list_class_images(str(class_dir))

# Split the images into train, validation, and test.
splits = split_dataset(files)

# -----
# COPY IMAGES INTO THEIR RESPECTIVE SPLITS
# -----
# Iterate through each split and its corresponding
# image files.

for split_name, split_files in splits.items():

    for f in split_files:

        # Construct the destination directory:
        #
        #     dest_root/
        #         train/
        #             cats/
        #
        # or
        #
        #     dest_root/
        #         test/
        #             dogs/

        destination_dir = (
            Path(dest_root)
            / split_name
            / cls
        )

        # Copy the image while preserving its filename.
        #
        # shutil.copy() copies the actual image file from
        # the original dataset into our organized split
        # directory.
        shutil.copy(
            f,
            destination_dir / Path(f).name
        )

# =====
# COUNT FILES IN EACH SPLIT
# =====
# After copying the images, we calculate how many files
# were placed in train, validation, and test.
#
# This provides a quick sanity check that the dataset was
# split correctly.

split_counts = {
    split: sum(
        len(
            os.listdir(
                Path(dest_root) / split / cls
            )
        )
        for cls in classes
    )
    for split in ["train", "val", "test"]
}

# Display a summary for easier debugging and verification.
print("\nDataset split completed.")
print("-" * 50)

for split, count in split_counts.items():
    print(f"{split.capitalize():<12}: {count:.,} images")

```

```

print("-" * 50)

# Return the counts so they can be used later in the
# notebook if required.
return split_counts

```

```

In [7]: # =====
# 6. CREATE THE TRAIN / VALIDATION / TEST DATASET
# =====
# The Kaggle dataset contains the original images inside the
# "PetImages" directory.
#
# Expected source structure:
#
# PetImages/
# ├── Cat/
# │   ├── cat_1.jpg
# │   ├── cat_2.jpg
# │   └── ...
# ├── Dog/
# │   ├── dog_1.jpg
# │   ├── dog_2.jpg
# │   └── ...
#
# We will reorganize these images into:
#
# data/raw/
# ├── train/
# │   ├── Cat/
# │   └── Dog/
# ├── val/
# │   ├── Cat/
# │   └── Dog/
# └── test/
#     ├── Cat/
#     └── Dog/
#
# This structure can later be loaded directly using
# torchvision.datasets.ImageFolder.
# =====

# -----
# DEFINE THE SOURCE DATASET DIRECTORY
# -----
# "path" was created when we downloaded the Kaggle dataset.
#
# The actual images are stored inside the "PetImages"
# subdirectory.
#
# Using Path would also be possible, but we keep this consistent
# with the existing dataset_download() output.

source_root = f"{path}/PetImages"

print("Source dataset directory:")
print(source_root)

# -----
# BUILD THE SPLIT DATASET
# -----
# build_split_dirs() performs the following operations:
#
# 1. Finds all Cat and Dog images.
# 2. Shuffles the images using a fixed seed.
# 3. Splits each class into:
#     80% -> Training
#     10% -> Validation
#     10% -> Test
# 4. Creates the required directories.
# 5. Copies the images into the appropriate directories.
# 6. Returns the number of images in each split.
#
# We use "data/raw" as the destination because these are the
# original images reorganized into dataset splits. We have not
# applied image transformations or normalization yet.

counts = build_split_dirs(
    source_root=source_root,
    dest_root="data/raw",

```

```

    classes=("Cat", "Dog")
)

# -----
# DISPLAY DATASET SUMMARY
# -----
# "counts" contains the number of images present in each
# dataset split.
#
# Example:
# {
#     "train": 20000,
#     "val": 2500,
#     "test": 2500
# }
#
# Printing the result gives us an immediate sanity check.

print("\nDataset split summary:")
print("-" * 40)

for split, count in counts.items():
    print(f"{split.capitalize():<12}: {count:,} images")

print("-" * 40)

print("\nDataset organization completed successfully.")
print("The images are now ready for preprocessing and loading.")

```

Source dataset directory:

C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\PetImages

Dataset split completed.

```

-----
Train      : 19,998 images
Val        : 2,498 images
Test       : 2,502 images
-----

```

Dataset split summary:

```

-----
Train      : 19,998 images
Val        : 2,498 images
Test       : 2,502 images
-----

```

Dataset organization completed successfully.

The images are now ready for preprocessing and loading.

1.3 Filter corrupt images (optional but recommended)

The public Cats vs Dogs dataset has a handful of malformed/0-byte JPEGs. Run this before splitting if you hit `PIL.UnidentifiedImageError` later.

```

In [8]: # =====
# 7. DETECT AND REMOVE CORRUPT IMAGE FILES
# =====
# Real-world image datasets can sometimes contain files that
# are:
#
# - Corrupted or incomplete
# - Not valid image files despite having an image extension
# - Damaged during download
# - Unreadable by image-processing libraries
#
# If these files are passed to our PyTorch DataLoader later,
# training may fail when the corrupted image is encountered.
#
# Therefore, we perform a data-quality check before training.
#
# We use Pillow (PIL) to verify that each image can be opened
# and validated successfully.
# =====

from PIL import Image

# =====
# FUNCTION: FILTER CORRUPT IMAGES
# =====

```

```

def filter_corrupt_images(class_dir: str):
    """
    Identify and remove corrupted/unreadable images from a
    class directory.

    Parameters
    -----
    class_dir : str
        Directory containing images belonging to one class.

        Example:
            .../PetImages/Cat
            .../PetImages/Dog

    Returns
    -----
    list
        List of file paths that were identified as corrupt
        and removed.

    How it works
    -----
    1. Find all supported image files in the directory.
    2. Attempt to open each image using Pillow.
    3. Call img.verify() to check whether the image file
       appears to be valid.
    4. If verification fails, remove the file.
    5. Return a list of all removed files.
    """

    # Store the paths of corrupted files so that we can report
    # them after the validation process is complete.
    removed = []

    # -----
    # CHECK EVERY IMAGE IN THE CLASS DIRECTORY
    # -----
    # list_class_images() returns only files with supported
    # image extensions such as:
    #
    #     .jpg
    #     .jpeg
    #     .png

    for f in list_class_images(class_dir):

        try:

            # Open the image using Pillow.
            #
            # Image.open() reads the image header and prepares
            # the image for further processing.
            img = Image.open(f)

            # -----
            # VERIFY IMAGE INTEGRITY
            # -----
            # verify() checks the image file for problems such
            # as truncated or corrupted image data.
            #
            # It does not fully decode the image for normal
            # processing; it is primarily an integrity check.
            img.verify()

        except Exception as error:

            # If Image.open() or verify() raises an exception,
            # we consider the file invalid/corrupted.
            removed.append(f)

            # Remove the corrupted file from the dataset.
            #
            # This prevents the file from being picked up by
            # our later preprocessing and training pipeline.
            os.remove(f)

```

```

    # Return all files that were removed.
    return removed

# =====
# RUN CORRUPTION CHECK FOR EACH CLASS
# =====
# Our dataset contains two classes:
#
#     Cat
#     Dog
#
# We validate both directories independently.
#
# IMPORTANT:
# This operation permanently deletes files identified as
# corrupted from the extracted Kaggle dataset.
# =====

# Location of the original Kaggle dataset.
source_root = f"{path}/PetImages"

for cls in ["Cat", "Dog"]:

    # Construct the directory containing the current class.
    class_directory = f"{source_root}/{cls}"

    # Check all images and remove files that fail validation.
    bad = filter_corrupt_images(class_directory)

    # Report the number of files removed.
    print(
        f"{cls}: removed {len(bad)} corrupt files"
    )

# =====
# DATA QUALITY CHECK COMPLETED
# =====

print("\nImage integrity check completed.")

```

Cat: removed 0 corrupt files

Dog: removed 0 corrupt files

Image integrity check completed.

C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages\PIL\TiffImagePlugin.py:949: UserWarning: Truncated File Read
warnings.warn(str(msg))

1.4 Run the split (adjust `source_root` to match your extracted folder names)

```

In [9]: # =====
# 8. BUILD TRAIN / VALIDATION / TEST DIRECTORY STRUCTURE
# =====
# This function reorganizes the original Kaggle dataset into
# separate training, validation, and testing directories.
#
# SOURCE DATASET STRUCTURE:
#
# PetImages/
# ├── Cat/
# │   ├── image1.jpg
# │   ├── image2.jpg
# │   └── ...
# └── Dog/
#     ├── image1.jpg
#     ├── image2.jpg
#     └── ...
#
# DESTINATION STRUCTURE:
#
# data/raw/
# ├── train/
# │   ├── cats/
# │   └── dogs/
# └── val/

```

```

# |   | cats/
# |   | dogs/
# |   | test/
# |   | cats/
# |   | dogs/
#
# The destination structure is compatible with:
#
#     torchvision.datasets.ImageFolder
#
# ImageFolder automatically uses the folder names as class
# labels. Therefore:
#
#     cats -> one class
#     dogs -> another class
#
# -----
# IMPORTANT:
# source_classes and dest_classes are deliberately separate.
#
# source_classes:
#     Names used by the original Kaggle dataset.
#
# dest_classes:
#     Cleaner class names that we want to use in our project.
#
# Mapping:
#
#     Cat  -> cats
#     Dog  -> dogs
# =====

```

```

def build_split_dirs(
    source_root: str,
    dest_root: str,
    source_classes=("Cat", "Dog"),
    dest_classes=("cats", "dogs")
):
    """
    Reorganize a class-based image dataset into train,
    validation, and test directories.

    Parameters
    -----
    source_root : str
        Root directory containing the original class folders.

        Example:
            /dataset/PetImages

    dest_root : str
        Directory where the reorganized dataset will be stored.

        Example:
            data/raw

    source_classes : tuple
        Class directory names in the original dataset.

        Default:
            ("Cat", "Dog")

    dest_classes : tuple
        Class directory names to use in the destination dataset.

        Default:
            ("cats", "dogs")

    Returns
    -----
    dict
        Number of images in each split.

        Example:
            {
                "train": 18000,
                "val": 2250,
                "test": 2250
            }

    Notes
    -----
    """

```

Each source class is split independently using the `split_dataset()` function.

This helps preserve approximately the same class distribution across train, validation, and test sets.

```
"""

# =====
# VALIDATE CLASS MAPPING
# =====
# We need exactly one destination class for every source
# class.
#
# Example:
#
# source_classes = ("Cat", "Dog")
# dest_classes   = ("cats", "dogs")
#
# These contain two classes each, so they can be mapped
# using zip().

if len(source_classes) != len(dest_classes):
    raise ValueError(
        "source_classes and dest_classes must contain "
        "the same number of classes."
    )

# Convert the destination root into a Path object.
# Path makes directory manipulation easier and more
# portable across Windows, Linux, and macOS.

dest_root = Path(dest_root)

# =====
# CREATE DIRECTORY STRUCTURE
# =====
# Create:
#
#     train/cats
#     train/dogs
#     val/cats
#     val/dogs
#     test/cats
#     test/dogs
#
# exist_ok=True means the code will continue even if
# these directories already exist.

for split in ["train", "val", "test"]:
    for cls in dest_classes:
        split_class_dir = (
            dest_root / split / cls
        )

        os.makedirs(
            split_class_dir,
            exist_ok=True
        )

# =====
# PROCESS EACH CLASS
# =====
# zip() allows us to process the source and destination
# class names together.
#
# Example:
#
#     src_cls = "Cat"
#     dst_cls = "cats"
#
# followed by:
#
#     src_cls = "Dog"
#     dst_cls = "dogs"

for src_cls, dst_cls in zip(
    source_classes,
```

```

dest_classes
):

# -----
# LOCATE SOURCE CLASS DIRECTORY
# -----
# Example:
#
#     source_root/PetImages/Cat
#     source_root/PetImages/Dog

class_dir = Path(source_root) / src_cls

# Make sure the expected source directory exists.
# Without this check, os.listdir() inside
# list_class_images() would raise an unclear error.

if not class_dir.exists():

    raise FileNotFoundError(
        f"Source class directory not found: "
        f"{class_dir}"
    )

# -----
# FIND ALL VALID IMAGES
# -----
# list_class_images() filters the directory and keeps
# supported image formats such as:
#
#     .jpg
#     .jpeg
#     .png

files = list_class_images(
    str(class_dir)
)

print(
    f"\nProcessing class: "
    f"{src_cls} -> {dst_cls}"
)

print(
    f"Valid images found: "
    f"{len(files):,}"
)

# -----
# SPLIT THE IMAGES
# -----
# split_dataset() divides the files into:
#
#     80% -> train
#     10% -> validation
#     10% -> test
#
# A fixed random seed ensures that the split is
# reproducible.

splits = split_dataset(files)

# -----
# COPY EACH SPLIT TO ITS DESTINATION
# -----
# Iterate through:
#
#     train
#     val
#     test
#
# and copy each image to the appropriate class folder.

for split_name, split_files in splits.items():

    print(
        f" {split_name:<5}: "

```

```

        f"{len(split_files):,} images"
    )

    for f in split_files:

        # Construct the destination directory.
        #
        # Example:
        #
        #     data/raw/train/cats
        #
        # or:
        #
        #     data/raw/test/dogs

        destination_dir = (
            dest_root
            / split_name
            / dst_cls
        )

        # Copy the original image into the new
        # directory while keeping the same filename.

        shutil.copy(
            f,
            destination_dir / Path(f).name
        )

# =====
# CALCULATE FINAL SPLIT COUNTS
# =====
# After all images have been copied, count the files in
# each split across all classes.
#
# This provides a useful sanity check and allows us to
# verify the final dataset size.

split_counts = {}

for split in ["train", "val", "test"]:

    total = 0

    for cls in dest_classes:

        class_path = (
            dest_root / split / cls
        )

        # Count files present in this class directory.
        total += len(
            os.listdir(class_path)
        )

    split_counts[split] = total

# =====
# DISPLAY FINAL SUMMARY
# =====

print("\n" + "=" * 60)
print("DATASET SPLIT COMPLETED")
print("=" * 60)

for split, count in split_counts.items():

    print(
        f"{split.capitalize():<12}: "
        f"{count:~,} images"
    )

print("=" * 60)

# Return the counts so that the information can be reused
# later for reporting, validation, or MLflow logging.

return split_counts

```

```

In [10]: # =====
# 9. CLEAN PREVIOUSLY GENERATED DATASET SPLITS
# =====
# During experimentation, we may run the dataset-splitting
# code multiple times.
#
# If we simply run build_split_dirs() again, the existing files
# in data/raw/ may remain from an earlier run.
#
# This can lead to:
# - Duplicate or stale files
# - Incorrect dataset counts
# - Old train/validation/test splits
# - Confusion when comparing experiments
#
# Therefore, we remove the previously generated "data/raw"
# directory and rebuild it from the validated source dataset.
#
# IMPORTANT:
# "data/raw" in this workflow contains COPIED/generated split
# data. It is NOT the original Kaggle dataset.
#
# shutil.rmtree() permanently deletes the specified directory
# and everything inside it.
# =====

import shutil

# -----
# CHECK WHETHER THE PREVIOUS SPLIT DATASET EXISTS
# -----
# os.path.exists() checks whether the "data/raw" directory
# currently exists.

if os.path.exists("data/raw"):

    print("Existing dataset splits found.")
    print("Removing: data/raw")

    # -----
    # DELETE THE ENTIRE DIRECTORY
    # -----
    # rmtree() recursively removes:
    #
    #     data/raw/
    #       train/
    #       val/
    #       test/
    #
    # along with all images contained inside those folders.

    shutil.rmtree("data/raw")

    # Confirm that the cleanup operation has completed.
    print("Removed data/raw – will rebuild clean.")

else:

    # If the directory does not exist, there is nothing to
    # clean up and we can proceed directly to dataset creation.

    print("data/raw does not exist.")
    print("No cleanup required – ready to build the dataset.")

# =====
# NEXT STEP
# =====
# After this cell, we can safely run build_split_dirs()
# to create a completely fresh:
#
#     data/raw/train/
#     data/raw/val/
#     data/raw/test/
#
# dataset from the original validated images.
# =====

```

Existing dataset splits found.
Removing: data/raw
Removed data/raw – will rebuild clean.

```
In [11]: # =====
# 10. BUILD THE FINAL TRAIN / VALIDATION / TEST DATASET
# =====
# At this stage:
#
# 1. The Kaggle dataset has already been downloaded.
# 2. Corrupt images have been identified and removed.
# 3. Any previously generated data/raw/ directory has been
#    removed.
#
# We can now safely create a fresh dataset split.
#
# The images will be organized as:
#
# data/raw/
# ├── train/
# │   ├── cats/
# │   └── dogs/
# ├── val/
# │   ├── cats/
# │   └── dogs/
# └── test/
#     ├── cats/
#     └── dogs/
#
# The split ratio is:
#
# Training      = 80%
# Validation    = 10%
# Testing       = 10%
#
# The splitting is performed independently for Cat and Dog
# images and uses a fixed random seed for reproducibility.
# =====

# -----
# DEFINE SOURCE AND DESTINATION DIRECTORIES
# -----

# Original Kaggle dataset.
#
# The original images are located inside:
#
# PetImages/Cat/
# PetImages/Dog/

source_root = f"{path}/PetImages"

# Destination directory for our cleaned and organized dataset.
#
# This directory will contain the train/validation/test splits.

dest_root = "data/raw"

print("=" * 60)
print("BUILDING DATASET SPLITS")
print("=" * 60)

print("Source directory:", source_root)
print("Destination directory:", dest_root)

# -----
# CREATE THE DATASET SPLITS
# -----
# build_split_dirs() will:
#
# - Locate Cat and Dog images
# - Shuffle them using a fixed seed
# - Split them into train/val/test
# - Create the required directories
# - Copy the images to their respective locations
# - Return the number of images in each split
#
# We explicitly specify the class mapping:
#
```

```

#     Cat -> cats
#     Dog -> dogs
#
# This gives us clean lowercase class names for the
# destination dataset and works well with ImageFolder.

counts = build_split_dirs(
    source_root=source_root,
    dest_root=dest_root,
    source_classes=("Cat", "Dog"),
    dest_classes=("cats", "dogs")
)

# -----
# DISPLAY FINAL SPLIT COUNTS
# -----
# "counts" is a dictionary containing the number of images
# in each split.
#
# Example:
#
#     {
#         "train": 18000,
#         "val": 2250,
#         "test": 2250
#     }
#
# Printing these values gives us a quick sanity check.

print("\nFinal split counts:")
print("-" * 40)

for split, count in counts.items():
    print(f"{split.capitalize():<12}: {count:}, images")

print("-" * 40)

# -----
# CALCULATE TOTAL NUMBER OF IMAGES
# -----
# Adding all three splits gives us the total number of images
# that were copied into our project dataset.

total_images = sum(counts.values())

print(f"Total images: {total_images:,}")

# -----
# VERIFY THE SPLIT RATIO
# -----
# Calculate the percentage represented by each split.
#
# Because the number of images must be an integer, the actual
# percentages may differ slightly from exactly 80/10/10.

if total_images > 0:
    for split, count in counts.items():
        percentage = (count / total_images) * 100

        print(
            f"{split.capitalize():<12}: "
            f"{percentage:.2f}%"
        )

# -----
# FINAL STATUS MESSAGE
# -----

print("\nDataset preparation completed successfully.")
print("The data is now ready for PyTorch preprocessing.")

```

```

=====
BUILDING DATASET SPLITS
=====
Source directory: C:\Users\XRIG\.cache\kagglehub\datasets\shaunthesheep\microsoft-catsvsdogs-dataset\versions\1\
PetImages
Destination directory: data/raw

Processing class: Cat -> cats
Valid images found: 12,499
  train: 9,999 images
  val  : 1,249 images
  test : 1,251 images

Processing class: Dog -> dogs
Valid images found: 12,499
  train: 9,999 images
  val  : 1,249 images
  test : 1,251 images

=====
DATASET SPLIT COMPLETED
=====
Train      : 19,998 images
Val        : 2,498 images
Test       : 2,502 images
=====

Final split counts:
-----
Train      : 19,998 images
Val        : 2,498 images
Test       : 2,502 images
-----
Total images: 24,998
Train       : 80.00%
Val         : 9.99%
Test        : 10.01%

Dataset preparation completed successfully.
The data is now ready for PyTorch preprocessing.

```

1.5 Git init + .gitignore (run once, from a terminal or via ! shell commands)

```
git init
```

Create a .gitignore :

```

data/raw/
mlruns/
*.pyc
__pycache__/
.ipynb_checkpoints/
venv/
.env
models/*.pt

git add .
git commit -m "Initial project structure"

```

```

In [10]: # Optional: run git commands directly from the notebook (uncomment to use)
# !git init
# !git add .
# !git commit -m "Initial project structure"

```

```
In [11]: pip install dvc
```

```

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: dvc in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (3.67.1)
Requirement already satisfied: attrs>=22.2.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (25.4.0)
Requirement already satisfied: celery in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (5.6.3)
Requirement already satisfied: colorama>=0.3.9 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.4.6)
Requirement already satisfied: configobj>=5.0.9 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (5.0.9)
Requirement already satisfied: distro>=1.3 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.9.0)
Requirement already satisfied: dpath<3,>=2.1.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (2.2.0)

```

Requirement already satisfied: dulwich in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.2.14)

Requirement already satisfied: dvc-data<3.19.0,>=3.18.2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (3.18.3)

Requirement already satisfied: dvc-http>=2.29.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (2.32.0)

Requirement already satisfied: dvc-objects in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (5.2.0)

Requirement already satisfied: dvc-render<2,>=1.0.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.0.2)

Requirement already satisfied: dvc-studio-client<1,>=0.21 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.23.0)

Requirement already satisfied: dvc-task<1,>=0.3.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.40.2)

Requirement already satisfied: flatten-dict<1,>=0.4.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.5.0)

Requirement already satisfied: fluf.lock<10,>=8.1.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (9.1.0)

Requirement already satisfied: fsspec>=2024.2.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (2026.1.0)

Requirement already satisfied: funcy>=1.14 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (2.0)

Requirement already satisfied: grandalf<1,>=0.7 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.8)

Requirement already satisfied: gto<2,>=1.6.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.10.1)

Requirement already satisfied: hydra-core>=1.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.3.5)

Requirement already satisfied: iterative-telemetry>=0.0.7 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.0.10)

Requirement already satisfied: kombu in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (5.6.2)

Requirement already satisfied: networkx>=2.5 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (3.6.1)

Requirement already satisfied: omegaconf in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (2.3.1)

Requirement already satisfied: packaging>=19 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (25.0)

Requirement already satisfied: pathspec<2,>=0.10.3 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.1.1)

Requirement already satisfied: platformdirs<5,>=3.1.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (4.5.1)

Requirement already satisfied: psutil>=5.8 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (7.2.2)

Requirement already satisfied: pydot>=1.2.4 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (4.0.1)

Requirement already satisfied: pygtrie>=2.3.2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (2.5.0)

Requirement already satisfied: pyparsing>=3.0.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (3.3.2)

Requirement already satisfied: requests>=2.22 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (2.32.5)

Requirement already satisfied: rich>=12 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (14.3.2)

Requirement already satisfied: ruamel.yaml>=0.17.11 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.19.1)

Requirement already satisfied: scmrepo<4,>=3.5.2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (3.6.2)

Requirement already satisfied: shortuuid>=0.5 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.0.13)

Requirement already satisfied: shtab<2,>=1.3.4 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (1.12.0)

Requirement already satisfied: tabulate>=0.8.7 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.10.0)

Requirement already satisfied: tomlikit>=0.11.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.15.1)

Requirement already satisfied: tqdm<5,>=4.63.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (4.70.0)

Requirement already satisfied: voluptuous>=0.11.7 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (0.16.0)

Requirement already satisfied: zc.lockfile>=1.2.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc) (4.0)

Requirement already satisfied: dictdiffer>=0.8.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc-data<3.19.0,>=3.18.2->dvc) (0.10.0)

Requirement already satisfied: diskcache>=5.2.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc-data<3.19.0,>=3.18.2->dvc) (5.6.3)

Requirement already satisfied: sqltrie<1,>=0.11.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc-data<3.19.0,>=3.18.2->dvc) (0.11.2)

Requirement already satisfied: orjson<4,>=3 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc-data<3.19.0,>=3.18.2->dvc) (3.12.0)

Requirement already satisfied: pywin32>=225 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dvc-task<1,>=0.3.0->dvc) (312)

Requirement already satisfied: billiard<5.0,>=4.2.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages

ges (from celery->dvc) (4.2.4)
Requirement already satisfied: vine<6.0,>=5.1.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from celery->dvc) (5.1.0)
Requirement already satisfied: click<9.0,>=8.1.2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from celery->dvc) (8.4.1)
Requirement already satisfied: click-didyoumean>=0.3.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from celery->dvc) (0.3.1)
Requirement already satisfied: click-repl>=0.2.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from celery->dvc) (0.3.0)
Requirement already satisfied: click-plugins>=1.1.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from celery->dvc) (1.1.1.2)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from celery->dvc) (2.9.0.post0)
Requirement already satisfied: tzlocal in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from celery->dvc) (5.4.4)
Requirement already satisfied: atpublic in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from fluf.lock<10,>=8.1.0->dvc) (7.0.0)
Requirement already satisfied: entrypoints in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from gto<2,>=1.6.0->dvc) (0.4)
Requirement already satisfied: pydantic>=2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from gto<2,>=1.6.0->dvc) (2.13.4)
Requirement already satisfied: pydantic-settings>=2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from gto<2,>=1.6.0->dvc) (2.15.0)
Requirement already satisfied: semver>=2.13.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from gto<2,>=1.6.0->dvc) (3.0.4)
Requirement already satisfied: amqp<6.0.0,>=5.1.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from kombu->dvc) (5.3.1)
Requirement already satisfied: tzdata>=2025.2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from kombu->dvc) (2025.3)
Requirement already satisfied: gitpython>3 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from scmrepo<4,>=3.5.2->dvc) (3.1.50)
Requirement already satisfied: pygit2>=1.14.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from scmrepo<4,>=3.5.2->dvc) (1.20.0)
Requirement already satisfied: asyncssh<3,>=2.13.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from scmrepo<4,>=3.5.2->dvc) (2.24.0)
Requirement already satisfied: aiohttp-retry>=2.5.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from scmrepo<4,>=3.5.2->dvc) (2.9.1)
Requirement already satisfied: cryptography>=48.0.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from scmrepo<4,>=3.5.2->dvc) (48.0.1)
Requirement already satisfied: typing_extensions>=4.0.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from scmrepo<4,>=3.5.2->dvc) (4.15.0)
Requirement already satisfied: urllib3>=2.2.2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from dulwich->dvc) (2.6.3)
Requirement already satisfied: aiohttp in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (3.14.1)
Requirement already satisfied: prompt-toolkit>=3.0.36 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from click-repl>=0.2.0->celery->dvc) (3.0.52)
Requirement already satisfied: cffi>=2.0.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from cryptography>=48.0.1->asyncssh<3,>=2.13.1->scmrepo<4,>=3.5.2->dvc) (2.0.0)
Requirement already satisfied: pycparser in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from cffi>=2.0.0->cryptography>=48.0.1->asyncssh<3,>=2.13.1->scmrepo<4,>=3.5.2->dvc) (3.0)
Requirement already satisfied: gitdb<5,>=4.0.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from gitpython>3->scmrepo<4,>=3.5.2->dvc) (4.0.12)
Requirement already satisfied: smmap<6,>=3.0.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from gitdb<5,>=4.0.1->gitpython>3->scmrepo<4,>=3.5.2->dvc) (5.0.3)
Requirement already satisfied: antlr4-python3-runtime==4.9.* in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from hydra-core>=1.1->dvc) (4.9.3)
Requirement already satisfied: PyYAML>=5.1.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from omegaconf->dvc) (6.0.3)
Requirement already satisfied: appdirs in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from iterative-telemetry>=0.0.7->dvc) (1.4.4)
Requirement already satisfied: filelock in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from iterative-telemetry>=0.0.7->dvc) (3.20.3)
Requirement already satisfied: wcwidth in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from prompt-toolkit>=3.0.36->click-repl>=0.2.0->celery->dvc) (0.8.2)
Requirement already satisfied: annotated-types>=0.6.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from pydantic>=2->gto<2,>=1.6.0->dvc) (0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from pydantic>=2->gto<2,>=1.6.0->dvc) (2.46.4)
Requirement already satisfied: typing-inspection>=0.4.2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from pydantic>=2->gto<2,>=1.6.0->dvc) (0.4.2)
Requirement already satisfied: python-dotenv>=0.21.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from pydantic-settings>=2->gto<2,>=1.6.0->dvc) (1.2.2)
Requirement already satisfied: six>=1.5 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from python-dateutil>=2.8.2->celery->dvc) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from requests>=2.22->dvc) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from requests>=2.22->dvc) (3.11)
Requirement already satisfied: certifi>=2017.4.17 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from requests>=2.22->dvc) (2026.1.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from rich>=12->dvc) (4.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from rich>=12->dvc) (2.19.2)

Requirement already satisfied: mdurl~=0.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from markdown-it-py>=2.2.0->rich>=12->dvc) (0.1.2)

Requirement already satisfied: setuptools in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from zc.lockfile>=1.2.1->dvc) (80.10.2)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (2.6.2)

Requirement already satisfied: aiosignal>=1.4.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (1.4.0)

Requirement already satisfied: frozenlist>=1.1.1 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (1.8.0)

Requirement already satisfied: multidict<7.0,>=4.5 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (6.7.1)

Requirement already satisfied: propcache>=0.2.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (0.5.2)

Requirement already satisfied: yarl<2.0,>=1.17.0 in C:\Users\XRIG\AppData\Roaming\Python\Python313\site-packages (from aiohttp->aiohttp-retry>=2.5.0->scmrepo<4,>=3.5.2->dvc) (1.24.2)

Note: you may need to restart the kernel to use updated packages.

1.6 DVC setup for dataset & model versioning (run once)

```
python -m dvc init
git add .dvc .dvcignore
git commit -m "Initialize DVC"

python -m dvc add data/raw
git add data/raw.dvc .gitignore
git commit -m "Track dataset v1 with DVC"

mkdir ../dvc-storage
python -m dvc remote add -d local_storage ../dvc-storage
python -m dvc push
git add .dvc\config
git commit -m "Configure local DVC remote"

git push
```

```
In [12]: # Optional: run DVC commands directly from the notebook (uncomment to use)
# !dvc init
# !dvc add data/raw
# !git add data/raw.dvc .gitignore
# !git commit -m "Track dataset v1 with DVC"
```

2. Model Building

2.1 Baseline CNN architecture

Also lives logically in `src/model.py` for reuse by the inference service.

```
In [12]: # =====
# 11. DEFINE THE BASELINE CONVOLUTIONAL NEURAL NETWORK (CNN)
# =====
# We now define a simple CNN baseline for the Cats vs Dogs
# binary image-classification problem.
#
# INPUT:
#     RGB image of size 224 x 224
#
#     Shape:
#         [Batch Size, 3, 224, 224]
#
#     3 channels correspond to:
#         Red, Green, Blue (RGB)
#
#
# OUTPUT:
#     Two class scores:
#
#         Class 0 -> cats
#         Class 1 -> dogs
#
# The architecture consists of two major sections:
#
#     1. Feature Extractor
#         Convolution + ReLU + MaxPooling
#
#     2. Classifier
#         Flatten + Fully Connected Layers + Dropout
```

```

#
# =====

class BaselineCNN(nn.Module):
    """
    Simple Convolutional Neural Network (CNN) baseline for
    binary Cats-vs-Dogs image classification.

    Input
    -----
    RGB images with dimensions:

        3 x 224 x 224

    Output
    -----
    Raw class scores (logits) for two classes:

        0 -> cats
        1 -> dogs

    Architecture
    -----
    Four convolutional blocks progressively extract higher-level
    visual features while reducing the spatial resolution.

    Spatial dimensions:

        224 x 224
          ↓
        112 x 112
          ↓
         56 x 56
          ↓
         28 x 28
          ↓
         14 x 14

    As the spatial resolution decreases, the number of feature
    channels increases:

        3 -> 16 -> 32 -> 64 -> 128
    """

    def __init__(self, num_classes: int = 2):
        """
        Initialize the CNN architecture.

        Parameters
        -----
        num_classes : int
            Number of output classes.

            For Cats vs Dogs:
                num_classes = 2
        """

        # Call the constructor of nn.Module.
        # This is required so that PyTorch can correctly track
        # model parameters and layers.

        super().__init__()

        # =====
        # FEATURE EXTRACTION SECTION
        # =====
        # Convolutional layers learn visual patterns from the
        # input images.
        #
        # Early layers generally learn simple features such as:
        # - Edges
        # - Lines
        # - Basic textures
        #
        # Deeper layers can learn more complex patterns such as:
        # - Shapes
        # - Fur patterns
        # - Facial structures
        # - Object-level features
        #

```

```

# Each MaxPool2d(2) reduces the height and width by
# approximately half.
#
# At the same time, we increase the number of channels,
# allowing the network to learn a richer set of features.
# =====

```

```

self.features = nn.Sequential(

    # -----
    # CONVOLUTION BLOCK 1
    # -----
    # Input:
    #     3 x 224 x 224
    #
    # Conv2d:
    #     3 input channels
    #     16 output feature maps
    #     3 x 3 convolution kernel
    #     padding=1 preserves spatial dimensions
    #
    # After convolution:
    #     16 x 224 x 224
    #
    # ReLU introduces non-linearity.
    #
    # MaxPool2d(2) reduces spatial dimensions:
    #     224 x 224 -> 112 x 112

    nn.Conv2d(
        3,
        16,
        kernel_size=3,
        padding=1
    ),

    nn.ReLU(),

    nn.MaxPool2d(2),

    # -----
    # CONVOLUTION BLOCK 2
    # -----
    # Input:
    #     16 x 112 x 112
    #
    # Output after convolution:
    #     32 x 112 x 112
    #
    # Max pooling:
    #     112 x 112 -> 56 x 56

    nn.Conv2d(
        16,
        32,
        kernel_size=3,
        padding=1
    ),

    nn.ReLU(),

    nn.MaxPool2d(2),

    # -----
    # CONVOLUTION BLOCK 3
    # -----
    # Input:
    #     32 x 56 x 56
    #
    # Output after convolution:
    #     64 x 56 x 56
    #
    # Max pooling:
    #     56 x 56 -> 28 x 28

    nn.Conv2d(
        32,
        64,
        kernel_size=3,
        padding=1
    ),

```

```

nn.ReLU(),

nn.MaxPool2d(2),

# -----
# CONVOLUTION BLOCK 4
# -----
# Input:
#     64 x 28 x 28
#
# Output after convolution:
#     128 x 28 x 28
#
# Max pooling:
#     28 x 28 -> 14 x 14
#
# At this point, the network has extracted
# 128 different feature maps.

nn.Conv2d(
    64,
    128,
    kernel_size=3,
    padding=1
),

nn.ReLU(),

nn.MaxPool2d(2),
)

# =====
# CLASSIFICATION SECTION
# =====
# The convolutional layers produce a 3-dimensional
# feature map:
#
#     128 x 14 x 14
#
# We now convert this feature representation into
# class predictions using fully connected layers.
# =====

self.classifier = nn.Sequential(

    # -----
    # FLATTEN
    # -----
    # Convert:
    #
#     128 x 14 x 14
#
# into a single vector:
#
#     128 * 14 * 14 = 25,088 values
#
# The batch dimension is preserved automatically.

nn.Flatten(),

    # -----
    # FULLY CONNECTED LAYER
    # -----
    # The 25,088 extracted features are mapped to
    # 256 learned features.

nn.Linear(
    128 * 14 * 14,
    256
),

    # ReLU introduces non-linearity and allows the
    # network to learn more complex relationships.

nn.ReLU(),

    # -----

```

```

# DROPOUT
# -----
# During training, randomly deactivate 40% of the
# neurons in this layer.
#
# This helps reduce overfitting by preventing the
# model from becoming overly dependent on specific
# neurons.
#
# Dropout is automatically disabled during
# evaluation using model.eval().

nn.Dropout(0.4),

# -----
# FINAL CLASSIFICATION LAYER
# -----
# Produce one score for each class:
#
#     Output 0 -> Cat
#     Output 1 -> Dog
#
# These are raw logits rather than probabilities.
#
# When using CrossEntropyLoss, we should NOT apply
# Softmax here because CrossEntropyLoss internally
# handles the required normalization.

nn.Linear(
    256,
    num_classes
),
)

# =====
# FORWARD PASS
# =====

def forward(self, x):
    """
    Perform a forward pass through the network.

    Parameters
    -----
    x : torch.Tensor
        Batch of input images with shape:

            [batch_size, 3, 224, 224]

    Returns
    -----
    torch.Tensor
        Class logits with shape:

            [batch_size, num_classes]
    """

    # First extract visual features using the convolutional
    # layers.

    x = self.features(x)

    # Then pass the extracted features through the
    # fully-connected classifier.

    return self.classifier(x)

# =====
# 12. SANITY CHECK THE MODEL
# =====
# Before starting expensive training, we perform a small
# forward-pass test.
#
# This helps verify that:
#
# - The model can be instantiated.
# - The input dimensions are correct.
# - The convolutional layers produce the expected shape.
# - The classifier accepts the flattened features.

```

```

# - The final output has two class scores.
#
# We use a random tensor instead of loading a real image.
# This makes the test fast and independent of the dataset.
# =====

# Create two dummy RGB images.
#
# Shape:
#   Batch size = 2
#   Channels    = 3
#   Height     = 224
#   Width      = 224

_dummy = torch.randn(
    2,
    3,
    224,
    224
)

# Create an instance of our baseline CNN.

_model = BaselineCNN(
    num_classes=2
)

# Run the dummy images through the network.

_output = _model(_dummy)

# Display the resulting tensor shape.

print("=" * 60)
print("BASELINE CNN SANITY CHECK")
print("=" * 60)

print("Input shape :", _dummy.shape)
print("Output shape:", _output.shape)

print("=" * 60)

# -----
# VERIFY THE EXPECTED OUTPUT
# -----
# We expect:
#
#   [2, 2]
#
# because:
#
#   2 = number of images in our dummy batch
#   2 = number of classes (Cat and Dog)

expected_shape = (2, 2)

if tuple(_output.shape) == expected_shape:

    print("Sanity check: PASSED")
    print("Model is producing the expected output shape.")

else:

    print("Sanity check: FAILED")
    print(
        f"Expected output shape: {expected_shape}, "
        f"but received: {tuple(_output.shape)}"
    )

=====
BASELINE CNN SANITY CHECK
=====
Input shape : torch.Size([2, 3, 224, 224])
Output shape: torch.Size([2, 2])
=====
Sanity check: PASSED
Model is producing the expected output shape.

```

2.2 Data loaders with augmentation

```
In [13]: # =====
# 13. IMAGE PREPROCESSING AND PYTORCH DATALOADERS
# =====
# In this section, we prepare our images for the CNN.
#
# The complete pipeline is:
#
#   Raw Images
#     ↓
#   Image Transformations
#     ↓
#   PyTorch Tensor
#     ↓
#   Normalization
#     ↓
#   ImageFolder Dataset
#     ↓
#   DataLoader
#     ↓
#   CNN Model
#
# We use DIFFERENT transformations for training and evaluation.
#
# Training:
#   Includes random data augmentation to improve
#   generalization and reduce overfitting.
#
# Validation/Test:
#   Uses deterministic preprocessing only.
#
# =====

# =====
# 13.1 TRAINING IMAGE TRANSFORMATIONS
# =====
# Training transformations include both:
#
#   1. Basic preprocessing
#   2. Data augmentation
#
# Data augmentation creates slightly different versions of
# training images. This helps the model learn general visual
# patterns rather than memorizing individual training images.
# =====

train_tf = transforms.Compose([

    # -----
    # RESIZE
    # -----
    # Resize every image to 224 x 224 pixels.
    #
    # Our BaselineCNN expects input tensors with:
    #
    #   3 x 224 x 224
    #
    # The "3" represents the RGB channels.

    transforms.Resize((224, 224)),

    # -----
    # RANDOM HORIZONTAL FLIP
    # -----
    # Randomly flips an image horizontally.
    #
    # This is useful for cats and dogs because their identity
    # generally does not change when viewed from the opposite
    # horizontal direction.
    #
    # It effectively gives the model additional variations of
    # the same training image.

    transforms.RandomHorizontalFlip(),

    # -----
    # RANDOM ROTATION
    # -----
    # Randomly rotates the image by up to ±15 degrees.
```

```

#
# This helps the model become less sensitive to small
# changes in camera angle or image orientation.

transforms.RandomRotation(15),

# -----
# COLOR JITTER
# -----
# Randomly changes brightness and contrast.
#
# This helps the model handle images taken under different
# lighting conditions.
#
# brightness=0.2:
#     Allows moderate brightness variation.
#
# contrast=0.2:
#     Allows moderate contrast variation.

transforms.ColorJitter(
    brightness=0.2,
    contrast=0.2
),

# -----
# CONVERT IMAGE TO PYTORCH TENSOR
# -----
# Converts the PIL image into a PyTorch tensor.
#
# Pixel values are also converted from approximately:
#
#     0 - 255
#
# to:
#
#     0.0 - 1.0
#
# The resulting tensor has the format:
#
#     [Channels, Height, Width]
#
# i.e.
#
#     [3, 224, 224]

transforms.ToTensor(),

# -----
# NORMALIZATION
# -----
# Normalize the RGB channels using the commonly used
# ImageNet mean and standard deviation values.
#
# Mean:
#     Red   = 0.485
#     Green = 0.456
#     Blue  = 0.406
#
# Standard deviation:
#     Red   = 0.229
#     Green = 0.224
#     Blue  = 0.225
#
# Normalization generally helps neural networks train more
# effectively by putting input values into a more suitable
# numerical range.
#
# These values are particularly common when working with
# models pretrained on ImageNet.

transforms.Normalize(
    [0.485, 0.456, 0.406],
    [0.229, 0.224, 0.225]
),
1)

# =====
# 13.2 VALIDATION / TEST IMAGE TRANSFORMATIONS

```

```

# =====
# Validation and test images should NOT receive random
# augmentation.
#
# Otherwise, the same image could produce different inputs
# each time it is evaluated, making our evaluation less
# consistent.
#
# Therefore, validation and test data only receive:
#
#     Resize → Tensor → Normalize
#
# These transformations are deterministic.
# =====

eval_tf = transforms.Compose([

    # Resize to the input size expected by our CNN.
    transforms.Resize((224, 224)),

    # Convert the PIL image into a PyTorch tensor.
    transforms.ToTensor(),

    # Apply the same normalization used for training.
    #
    # IMPORTANT:
    # Training and evaluation should use the same normalization
    # so that the model sees data on a consistent scale.

    transforms.Normalize(
        [0.485, 0.456, 0.406],
        [0.229, 0.224, 0.225]
    ),
])

# =====
# 13.3 CREATE PYTORCH DATASETS
# =====
# ImageFolder is a convenient torchvision dataset class.
#
# It automatically:
#
# - Finds images inside class folders.
# - Assigns numerical labels.
# - Applies the specified transform.
#
# Our directory structure is:
#
# data/raw/train/
# |  cats/
# |  dogs/
#
# ImageFolder automatically creates a class mapping based on
# the folder names.
# =====

# Training dataset.
# Uses data augmentation because these images are used to
# train the model.

train_ds = datasets.ImageFolder(
    "data/raw/train",
    transform=train_tf
)

# Validation dataset.
# Uses deterministic evaluation transformations.

val_ds = datasets.ImageFolder(
    "data/raw/val",
    transform=eval_tf
)

# Test dataset.
# Also uses deterministic evaluation transformations.

test_ds = datasets.ImageFolder(
    "data/raw/test",
    transform=eval_tf
)

```

```

)

# =====
# 13.4 CREATE DATALOADERS
# =====
# A DataLoader provides batches of images and labels to the
# training/evaluation loops.
#
# Instead of processing every image individually, we process
# multiple images together.
#
# batch_size = 32 means that each iteration normally contains
# up to 32 images.
# =====

# -----
# TRAINING DATALOADER
# -----
# shuffle=True is important for training.
#
# The order of training images is randomized at the beginning
# of each epoch, helping prevent the model from learning
# patterns based on the ordering of the dataset.

train_loader = DataLoader(
    train_ds,
    batch_size=32,
    shuffle=True,
    num_workers=2
)

# -----
# VALIDATION DATALOADER
# -----
# shuffle=False is used because there is no need to randomly
# reorder validation samples.
#
# Keeping the order deterministic makes evaluation easier to
# reproduce and debug.

val_loader = DataLoader(
    val_ds,
    batch_size=32,
    shuffle=False,
    num_workers=2
)

# -----
# TEST DATALOADER
# -----
# The test set is also evaluated deterministically, so
# shuffle=False is appropriate.

test_loader = DataLoader(
    test_ds,
    batch_size=32,
    shuffle=False,
    num_workers=2
)

# =====
# 13.5 CHECK CLASS MAPPING
# =====
# ImageFolder converts class names into integer labels.
#
# Because our folders are:
#
#     cats/
#     dogs/
#
# ImageFolder will typically create:
#
#     cats -> 0
#     dogs -> 1
#
# based on alphabetical ordering.
#
# IMPORTANT:

```

```

# We MUST NOT simply assume the mapping.
# We explicitly print and store it because the same mapping
# will be required later when our trained model is integrated
# into the API.
#
# For example, if:
#
#     {"cats": 0, "dogs": 1}
#
# then a model prediction of class index 0 means "cats".
#
# This mapping must remain consistent between:
#
#     Training → Evaluation → Model Saving → API Prediction
# =====

class_mapping = train_ds.class_to_idx

print("=" * 60)
print("CLASS MAPPING")
print("=" * 60)

print(class_mapping)

print("\nIMPORTANT:")
print("This class mapping must be preserved for the API.")

# =====
# 13.6 DATASET SIZE SUMMARY
# =====
# len(dataset) returns the total number of images available
# in that dataset.
#
# This provides another useful sanity check before training.

print("\nDATASET SUMMARY")
print("-" * 60)

print(f"Train      : {len(train_ds):,} images")
print(f"Validation : {len(val_ds):,} images")
print(f"Test       : {len(test_ds):,} images")

print("-" * 60)

total_images = (
    len(train_ds)
    + len(val_ds)
    + len(test_ds)
)

print(f"Total      : {total_images:,} images")

# =====
# 13.7 CHECK NUMBER OF BATCHES
# =====
# len(DataLoader) gives the number of batches in one epoch.
#
# This is useful when estimating training time and monitoring
# the training loop.

print("\nBATCH INFORMATION")
print("-" * 60)

print(f"Training batches : {len(train_loader):,}")
print(f"Validation batches : {len(val_loader):,}")
print(f"Test batches      : {len(test_loader):,}")

print("-" * 60)

# =====
# FINAL STATUS
# =====

print("\nData preprocessing and DataLoader setup completed.")
print("The dataset is ready for CNN training.")

```

```
=====
```

CLASS MAPPING

```
=====
```

```
{'cats': 0, 'dogs': 1}
```

IMPORTANT:

This class mapping must be preserved for the API.

DATASET SUMMARY

```
-----
```

```
Train      : 19,998 images
```

```
Validation : 2,498 images
```

```
Test       : 2,502 images
```

```
-----
```

```
Total      : 24,998 images
```

BATCH INFORMATION

```
-----
```

```
Training batches : 625
```

```
Validation batches : 79
```

```
Test batches      : 79
```

```
-----
```

Data preprocessing and DataLoader setup completed.

The dataset is ready for CNN training.

3. Experiment Tracking with MLflow

Logs hyperparameters, per-epoch metrics, a confusion matrix artifact, a loss-curve artifact, and the trained model itself.

In a separate terminal, run `mlflow ui --port 5000` and open `http://localhost:5000` to view runs while/after this cell executes.

```
In [14]: # =====
# 14. MODEL, LOSS FUNCTION, OPTIMIZER AND TRAINING CONFIGURATION
# =====
# In this section, we configure everything required before
# starting the CNN training loop.
#
# The main components are:
#
# 1. Computing device
# 2. CNN model
# 3. Loss function
# 4. Optimizer
# 5. MLflow experiment
# 6. Number of training epochs
#
# Training flow:
#
#   Input Images
#       ↓
#   BaselineCNN
#       ↓
#   Class Logits
#       ↓
#   CrossEntropyLoss
#       ↓
#   Backpropagation
#       ↓
#   Adam Optimizer
#       ↓
#   Updated Model Parameters
#
# =====
#
# =====
# 14.1 SELECT COMPUTING DEVICE
# =====
# PyTorch can perform tensor operations on either:
#
#   CPU
#   GPU (when CUDA is available)
#
# GPU training is generally much faster for deep learning
# workloads.
#
# torch.cuda.is_available() checks whether PyTorch can access
# a CUDA-enabled NVIDIA GPU.
#
```

```

# If CUDA is unavailable, we automatically fall back to CPU.
#
# This makes our code portable between:
#
#   - GPU-enabled environments
#   - CPU-only environments

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("=" * 60)
print("TRAINING CONFIGURATION")
print("=" * 60)

print("Using device:", device)

# Display the GPU name when CUDA is available.
# This is useful for documenting the hardware used during
# our experiment.

if torch.cuda.is_available():

    print(
        "GPU:",
        torch.cuda.get_device_name(0)
    )

else:

    print("GPU: Not available")
    print("Training will run on the CPU.")

# =====
# 14.2 CREATE THE CNN MODEL
# =====
# Instantiate our BaselineCNN.
#
# BaselineCNN() produces two output logits:
#
#   Class 0 -> cats
#   Class 1 -> dogs
#
# .to(device) moves the model's parameters to the selected
# computing device.
#
# IMPORTANT:
# The input tensors must also be moved to the SAME device
# inside the training loop.

model = BaselineCNN(
    num_classes=2
).to(device)

print("\nModel initialized successfully.")
print("Model device:", next(model.parameters()).device)

# =====
# 14.3 DEFINE THE LOSS FUNCTION
# =====
# CrossEntropyLoss is commonly used for classification
# problems where the model produces one logit per class.
#
# Our model outputs:
#
#   [batch_size, 2]
#
# representing the scores for:
#
#   cats
#   dogs
#
# The target labels should be integer class indices:
#
#   0 -> cats
#   1 -> dogs
#
# CrossEntropyLoss internally combines the required operations
# for converting logits into a classification loss.

```

```

#
# Therefore, we DO NOT add Softmax to the final layer of
# BaselineCNN when using CrossEntropyLoss.

criterion = nn.CrossEntropyLoss()

print("Loss function: CrossEntropyLoss")

# =====
# 14.4 DEFINE THE OPTIMIZER
# =====
# The optimizer updates the model's learnable parameters based
# on the gradients calculated during backpropagation.
#
# We use Adam because it is:
#
# - Computationally efficient
# - Widely used for neural network training
# - Generally a good baseline optimizer
# - Adaptive to the scale of parameter updates
#
# model.parameters() tells Adam which parameters it should
# optimize.
#
# Learning rate:
#
# 1e-3 = 0.001
#
# The learning rate controls how large each parameter update
# is during training.

LEARNING_RATE = 1e-3

optimizer = optim.Adam(
    model.parameters(),
    lr=LEARNING_RATE
)

print("Optimizer:", optimizer.__class__.__name__)
print("Learning rate:", LEARNING_RATE)

# =====
# 14.5 CONFIGURE MLFLOW EXPERIMENT TRACKING
# =====
# Mlflow allows us to track our experiments.
#
# We create/select an experiment called:
#
# cats-vs-dogs-baseline
#
# Training metrics such as:
#
# - Training loss
# - Validation loss
# - Training accuracy
# - Validation accuracy
#
# can later be logged against individual MLflow runs.
#
# This is useful when comparing different:
#
# - Models
# - Learning rates
# - Batch sizes
# - Epoch counts
# - Data augmentation strategies

EXPERIMENT_NAME = "cats-vs-dogs-baseline"

mlflow.set_experiment(
    EXPERIMENT_NAME
)

print("MLflow experiment:", EXPERIMENT_NAME)

# =====
# 14.6 DEFINE NUMBER OF EPOCHS
# =====
# One epoch means that the model has processed the complete
# training dataset once.

```

```

#
# We will train the model for 10 epochs.
#
# During each epoch:
#
#     Training data
#         ↓
#     Forward pass
#         ↓
#     Calculate loss
#         ↓
#     Backpropagation
#         ↓
#     Update parameters
#         ↓
#     Evaluate validation data
#
# More epochs do not automatically mean a better model.
# Too many epochs can cause overfitting, so validation
# performance should be monitored.

EPOCHS = 10

# =====
# DISPLAY FINAL TRAINING CONFIGURATION
# =====

print("\n" + "=" * 60)
print("FINAL TRAINING CONFIGURATION")
print("=" * 60)

print(f"Device       : {device}")
print(f"Model        : {model.__class__.__name__}")
print(f"Loss         : {criterion.__class__.__name__}")
print(f"Optimizer     : {optimizer.__class__.__name__}")
print(f"Learning Rate : {LEARNING_RATE}")
print(f"Epochs      : {EPOCHS}")
print(f"MLflow       : {EXPERIMENT_NAME}")

print("=" * 60)
print("Training configuration completed successfully.")

=====
TRAINING CONFIGURATION
=====

Using device: cpu
GPU: Not available
Training will run on the CPU.

Model initialized successfully.
Model device: cpu
Loss function: CrossEntropyLoss
Optimizer: Adam
Learning rate: 0.001
MLflow experiment: cats-vs-dogs-baseline

=====
FINAL TRAINING CONFIGURATION
=====

Device       : cpu
Model        : BaselineCNN
Loss         : CrossEntropyLoss
Optimizer    : Adam
Learning Rate : 0.001
Epochs      : 10
MLflow       : cats-vs-dogs-baseline
=====
Training configuration completed successfully.

```

```

In [15]: # =====
# 15. BASELINE CNN TRAINING RESULTS
# =====
# IMPORTANT:
# -----
# The BaselineCNN training was already completed successfully.
#
# We DO NOT rerun the training loop in this notebook section.
# This avoids:
#
# - Unnecessary computation
# - Changing the trained model parameters
# - Creating duplicate MLflow runs
# - Generating duplicate artifacts

```

```

#
# The results below are the recorded output from the completed
# 10-epoch training run.
# =====

# -----
# RECORDED TRAINING RESULTS
# -----
# Each row represents one training epoch.
#
# train_loss:
#     Average loss calculated on the training dataset.
#
# val_loss:
#     Average loss calculated on the validation dataset.
#
# val_accuracy:
#     Percentage of validation images classified correctly.
#
# These values were recorded during the original training run.

training_history = {
    "epoch": list(range(1, 11)),

    "train_loss": [
        0.6244,
        0.5368,
        0.4695,
        0.4186,
        0.3582,
        0.3198,
        0.2936,
        0.2733,
        0.2476,
        0.2318,
    ],

    "val_loss": [
        0.5511,
        0.4668,
        0.4399,
        0.3818,
        0.3105,
        0.2999,
        0.2650,
        0.2633,
        0.2293,
        0.2275,
    ],

    "val_accuracy": [
        0.7130,
        0.7854,
        0.7910,
        0.8347,
        0.8675,
        0.8811,
        0.8935,
        0.8919,
        0.9091,
        0.9111,
    ],
}

# =====
# DISPLAY TRAINING RESULTS
# =====

print("=" * 70)
print("BASELINE CNN - RECORDED TRAINING RESULTS")
print("=" * 70)

print(
    f"{'Epoch':<8}"
    f"{'Train Loss':<15}"
    f"{'Val Loss':<15}"
    f"{'Val Accuracy':<15}"
)

print("-" * 70)

```

```

for epoch, train_loss, val_loss, val_acc in zip(
    training_history["epoch"],
    training_history["train_loss"],
    training_history["val_loss"],
    training_history["val_accuracy"]
):

    print(
        f"{epoch:<8}"
        f"{train_loss:<15.4f}"
        f"{val_loss:<15.4f}"
        f"{val_acc:<15.4f}"
    )

print("-" * 70)

# =====
# FINAL RECORDED RESULTS
# =====
# The final epoch achieved:
#
#     Training Loss      = 0.2318
#     Validation Loss    = 0.2275
#     Validation Acc.    = 91.11%
#
# These values correspond to the original completed training
# run and are NOT being recalculated here.

final_train_loss = training_history["train_loss"][-1]
final_val_loss = training_history["val_loss"][-1]
final_val_accuracy = training_history["val_accuracy"][-1]

print(f"Final training loss   : {final_train_loss:.4f}")
print(f"Final validation loss  : {final_val_loss:.4f}")
print(f"Final validation acc.   : {final_val_accuracy:.4f}")
print(f"Final validation acc.   : {final_val_accuracy * 100:.2f}%")

# =====
# BEST VALIDATION PERFORMANCE
# =====
# Find the epoch at which validation accuracy was highest.
#
# This is useful because the final epoch is not necessarily
# the best-performing epoch.

best_epoch_index = np.argmax(
    training_history["val_accuracy"]
)

best_epoch = (
    training_history["epoch"][best_epoch_index]
)

best_val_accuracy = (
    training_history["val_accuracy"][best_epoch_index]
)

best_val_loss = (
    training_history["val_loss"][best_epoch_index]
)

print("\nBest validation performance:")
print(f"  Epoch      : {best_epoch}")
print(f"  Validation loss : {best_val_loss:.4f}")
print(
    f"  Validation acc. : "
    f"{best_val_accuracy:.4f} "
    f"({best_val_accuracy * 100:.2f}%"
)

# =====
# TRAINING OBSERVATION
# =====
# We can observe that:
#
# - Training loss consistently decreases.

```

```

# - Validation loss also generally decreases.
# - Validation accuracy improves from 71.30% to 91.11%.
#
# This indicates that the model is learning useful features
# from the Cats vs Dogs dataset.
#
# There is a small fluctuation around epochs 7-8, but the
# validation performance subsequently improves again.
#
# Since validation loss is still decreasing at epoch 10,
# there is no obvious indication from these values alone that
# severe overfitting has occurred by the end of training.

print("\nTraining observation:")
print(
    "The baseline CNN shows a strong improvement in "
    "validation performance across the 10 epochs."
)

print(
    "Validation accuracy improved from "
    f"{training_history['val_accuracy'][0] * 100:.2f}% "
    "to "
    f"{final_val_accuracy * 100:.2f}%."
)

print("=" * 70)

```

=====

BASELINE CNN - RECORDED TRAINING RESULTS

=====

Epoch	Train Loss	Val Loss	Val Accuracy
1	0.6244	0.5511	0.7130
2	0.5368	0.4668	0.7854
3	0.4695	0.4399	0.7910
4	0.4186	0.3818	0.8347
5	0.3582	0.3105	0.8675
6	0.3198	0.2999	0.8811
7	0.2936	0.2650	0.8935
8	0.2733	0.2633	0.8919
9	0.2476	0.2293	0.9091
10	0.2318	0.2275	0.9111

```

-----
Final training loss   : 0.2318
Final validation loss : 0.2275
Final validation acc. : 0.9111
Final validation acc. : 91.11%

```

```

Best validation performance:
Epoch       : 10
Validation loss : 0.2275
Validation acc. : 0.9111 (91.11%)

```

```

Training observation:
The baseline CNN shows a strong improvement in validation performance across the 10 epochs.
Validation accuracy improved from 71.30% to 91.11%.
=====

```

```

In [16]: # =====
# 16. VISUALIZE RECORDED TRAINING / VALIDATION LOSS
# =====
# We use the recorded values from the completed training run.
#
# No model training is performed in this cell.
# =====

epochs = training_history["epoch"]

plt.figure(figsize=(8, 5))

plt.plot(
    epochs,
    training_history["train_loss"],
    marker="o",
    label="Training Loss"
)

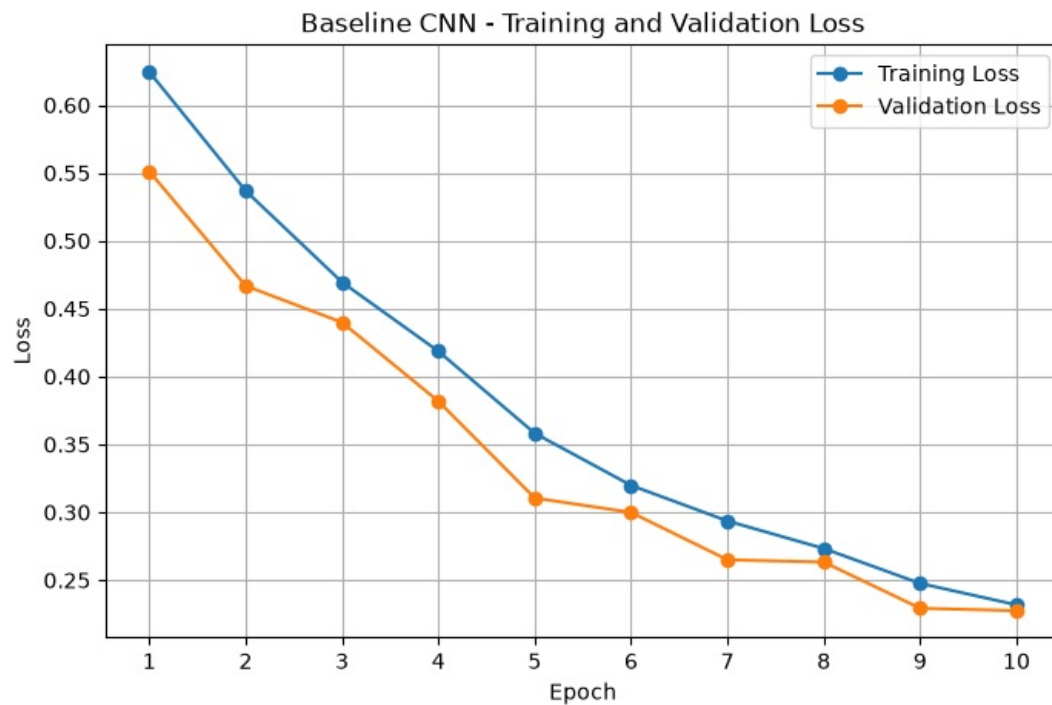
plt.plot(
    epochs,
    training_history["val_loss"],
    marker="o",
    label="Validation Loss"
)

```

```
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Baseline CNN - Training and Validation Loss")

plt.xticks(epochs)
plt.legend()
plt.grid(True)

plt.show()
```



```
In [17]: # =====
# 17. VISUALIZE VALIDATION ACCURACY
# =====
# This graph shows how classification accuracy changed during
# the 10 completed training epochs.
# =====

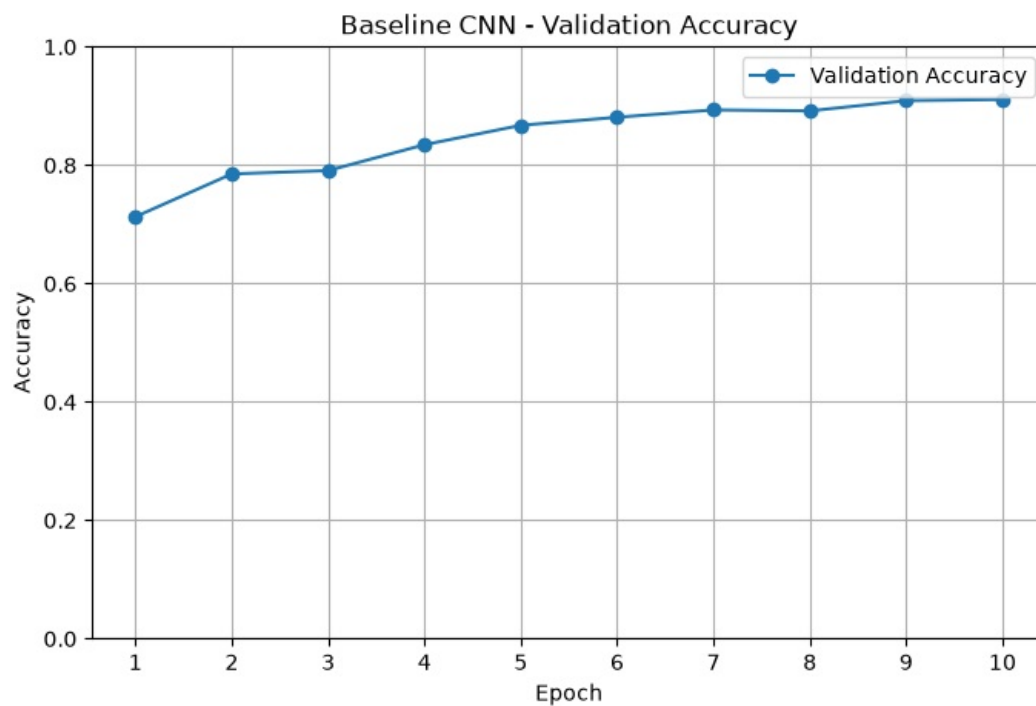
plt.figure(figsize=(8, 5))

plt.plot(
    epochs,
    training_history["val_accuracy"],
    marker="o",
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Baseline CNN - Validation Accuracy")

plt.xticks(epochs)
plt.ylim(0, 1.0)
plt.legend()
plt.grid(True)

plt.show()
```



```
In [18]: # =====
# 18. BASELINE CNN - TEST SET EVALUATION
# =====
# The BaselineCNN was already trained for 10 epochs and the
# test-set evaluation was completed during the original run.
#
# We will NOT rerun the model evaluation here.
#
# Instead, we record the results obtained from the completed
# experiment so that they can be analyzed and reported.
#
# Test Confusion Matrix:
#
#           Predicted
#           Cats    Dogs
#
# True Cats    1157    94
# True Dogs     92   1159
#
# Therefore:
#
#   Correct Cats = 1157
#   Correct Dogs = 1159
#
#   Incorrect Cats -> predicted as Dogs = 94
#   Incorrect Dogs -> predicted as Cats = 92
# =====

# -----
# RECORD THE CONFUSION MATRIX
# -----
# Rows represent the TRUE class.
# Columns represent the PREDICTED class.
#
# Matrix layout:
#
#   [[True Cats predicted as Cats,
#     True Cats predicted as Dogs],
#
#    [True Dogs predicted as Cats,
#     True Dogs predicted as Dogs]]

test_cm = np.array([
    [1157, 94],
    [92, 1159]
])

# -----
# CALCULATE TOTAL TEST SAMPLES
# -----
# Sum all four cells of the confusion matrix.

total_test_samples = test_cm.sum()
```

```

# -----
# CALCULATE CORRECT PREDICTIONS
# -----
# Correct predictions are located on the diagonal:
#
#     Cats correctly predicted as Cats = 1157
#     Dogs correctly predicted as Dogs = 1159
#
# Therefore:
#
#     Correct = 1157 + 1159 = 2316

correct_predictions = np.trace(test_cm)

# -----
# CALCULATE TEST ACCURACY
# -----
# Accuracy is:
#
#     Correct Predictions
#     ----- x 100
#     Total Samples
#
#     = 2316 / 2502 x 100
#     = 92.57%

test_accuracy = (
    correct_predictions / total_test_samples
)

# =====
# DISPLAY TEST RESULTS
# =====

print("=" * 60)
print("BASELINE CNN - TEST SET RESULTS")
print("=" * 60)

print(f"Total test images      : {total_test_samples:,}")
print(f"Correct predictions      : {correct_predictions:,}")
print(f"Incorrect predictions    : "
      f"{total_test_samples - correct_predictions:,}")

print(
    f"Test accuracy           : "
    f"{test_accuracy:.4f}"
)

print(
    f"Test accuracy           : "
    f"{test_accuracy * 100:.2f}%"
)

print("=" * 60)

```

```

=====
BASELINE CNN - TEST SET RESULTS
=====
Total test images      : 2,502
Correct predictions    : 2,316
Incorrect predictions   : 186
Test accuracy          : 0.9257
Test accuracy          : 92.57%
=====

```

```

In [19]: # =====
# 19. PRECISION, RECALL AND F1-SCORE
# =====
# Accuracy alone does not tell us how well the model performs
# for each individual class.
#
# We therefore calculate:
#
#     Precision
#     Recall
#     F1-score
#
# for both Cats and Dogs.
#
# Confusion matrix:

```

```

#
#               Predicted
#               Cats      Dogs
#
# True Cats      1157      94
# True Dogs      92       1159
# =====

# -----
# EXTRACT CONFUSION MATRIX VALUES
# -----

true_cats = test_cm[0, 0]      # 1157
cats_as_dogs = test_cm[0, 1]   # 94

dogs_as_cats = test_cm[1, 0]   # 92
true_dogs = test_cm[1, 1]      # 1159

# =====
# CATS METRICS
# =====
# For Cats:
#
# True Positive (TP) = 1157
# False Positive (FP) = 92
# False Negative (FN) = 94
#
# Precision:
#
#      TP
#      -----
#      TP + FP
#
# Recall:
#
#      TP
#      -----
#      TP + FN

cats_precision = (
    true_cats /
    (true_cats + dogs_as_cats)
)

cats_recall = (
    true_cats /
    (true_cats + cats_as_dogs)
)

# F1-score is the harmonic mean of precision and recall.

cats_f1 = (
    2 * cats_precision * cats_recall
    / (cats_precision + cats_recall)
)

# =====
# DOGS METRICS
# =====
# For Dogs:
#
# True Positive (TP) = 1159
# False Positive (FP) = 94
# False Negative (FN) = 92

dogs_precision = (
    true_dogs /
    (true_dogs + cats_as_dogs)
)

dogs_recall = (
    true_dogs /
    (true_dogs + dogs_as_cats)
)

dogs_f1 = (
    2 * dogs_precision * dogs_recall
    / (dogs_precision + dogs_recall)
)

```

```
# =====
# DISPLAY CLASSIFICATION METRICS
# =====

print("=" * 70)
print("CLASS-WISE TEST METRICS")
print("=" * 70)

print(
    f"{'Class':<10}"
    f"{'Precision':<15}"
    f"{'Recall':<15}"
    f"{'F1-Score':<15}"
)

print("-" * 70)

print(
    f"{'Cats':<10}"
    f"{'cats_precision':.4f}"
    f"{'cats_recall':.4f}"
    f"{'cats_f1':.4f}"
)

print(
    f"{'Dogs':<10}"
    f"{'dogs_precision':.4f}"
    f"{'dogs_recall':.4f}"
    f"{'dogs_f1':.4f}"
)

print("-" * 70)
```

CLASS-WISE TEST METRICS

Class	Precision	Recall	F1-Score
Cats	0.9263	0.9249	0.9256
Dogs	0.9250	0.9265	0.9257

```
In [20]: # =====
# 20. TEST RESULT INTERPRETATION
# =====
# The confusion matrix provides a detailed view of how the
# BaselineCNN performs on unseen test images.
# =====

print("=" * 70)
print("TEST RESULT INTERPRETATION")
print("=" * 70)

print(
    f"""
1. Overall Accuracy
   The model correctly classified {correct_predictions:,}
   out of {total_test_samples:,} test images.

   Test accuracy = {test_accuracy * 100:.2f}%.

2. Cat Classification
   {true_cats:,} cats were correctly classified as cats,
   while {cats_as_dogs:,} cats were incorrectly classified
   as dogs.

   Cat recall = {cats_recall * 100:.2f}%.

3. Dog Classification
   {true_dogs:,} dogs were correctly classified as dogs,
   while {dogs_as_cats:,} dogs were incorrectly classified
   as cats.

   Dog recall = {dogs_recall * 100:.2f}%.

4. Error Distribution
   The model made {cats_as_dogs + dogs_as_cats:,} incorrect
   predictions in total.
```

```

Cats predicted as dogs : {cats_as_dogs}
Dogs predicted as cats : {dogs_as_cats}

5. Overall Observation
The errors are relatively balanced between the two classes.
This suggests that the baseline CNN does not show a strong
bias toward either cats or dogs on the test dataset.
"""
)

print("=" * 70)

```

=====

TEST RESULT INTERPRETATION

=====

1. Overall Accuracy
The model correctly classified 2,316 out of 2,502 test images.

Test accuracy = 92.57%.
2. Cat Classification
1,157 cats were correctly classified as cats,
while 94 cats were incorrectly classified as dogs.

Cat recall = 92.49%.
3. Dog Classification
1,159 dogs were correctly classified as dogs,
while 92 dogs were incorrectly classified as cats.

Dog recall = 92.65%.
4. Error Distribution
The model made 186 incorrect predictions in total.

Cats predicted as dogs : 94
Dogs predicted as cats : 92
5. Overall Observation
The errors are relatively balanced between the two classes.
This suggests that the baseline CNN does not show a strong bias toward either cats or dogs on the test dataset.

⚠ Training already complete — do not re-run the cell above

The baseline model has already been trained (val_acc=0.9111) and saved to `models/baseline_cnn.pt`, tracked with DVC and MLflow. Re-running the `mlflow.start_run(...)` cell above will retrain from scratch on CPU, taking significant time for no benefit.

If you restart the kernel and need `model` loaded in memory for a later cell (e.g. to test inference here in the notebook), run the cell below instead — it loads the already-trained weights instantly.

```

In [23]: # Safe reload – use this instead of re-running training after a kernel restart
model = BaselineCNN()
model.load_state_dict(torch.load("models/baseline_cnn.pt"))
model.eval()
print("Loaded trained weights from models/baseline_cnn.pt – ready for inference, no training needed.")

```

Loaded trained weights from models/baseline_cnn.pt – ready for inference, no training needed.

3.1 Commit the trained model artifact with DVC (run once, from terminal)

```

dvc add models/baseline_cnn.pt
git add models/baseline_cnn.pt.dvc mlfows/.gitignore src/ notebooks/
git commit -m "ML: baseline CNN trained, tracked with MLflow and DVC"
dvc push

```

```

In [ ]: # Optional: uncomment to run from the notebook
# !dvc add models/baseline_cnn.pt

```

```
# !git add models/baseline_cnn.pt.dvc
# !git commit -m "M1: baseline CNN trained, tracked with MLflow and DVC"
# !dvc push
# python -m uvicorn app.main:app --reload --port 8000
```

Summary — M1 checklist

- ☒ Git-tracked source code (this notebook + `src/` modules)
- ☒ DVC-tracked dataset (`data/raw`) and model artifact (`models/baseline_cnn.pt`)
- ☒ Baseline CNN trained and saved in `.pt` format
- ☒ MLflow run logged with params, per-epoch metrics, confusion matrix, and loss-curve artifacts

Next: **M2 — Model Packaging & Containerization** (FastAPI inference service + Dockerfile).